UNIVERSITÀ
di VERONA

Università degli Studi di Verona
Dipartimento di Informatica

Technical Report

Differentiable Physics Simulation: Methods for Cloth and Inverse Problems

---

Umberto Castellani
Matteo Boroni Grazioli

June 2026

Contents

1 Math Background **4**
- 1.1 Newton-Raphson for Root-Finding . . . 4

2 Implicit Simulator **5**
- 2.1 BDF1 Time-Discretization . . . 5
- 2.2 Single-Linearization Solve (Newton-Raphson 1-iteration) . . . 5
- 2.3 Newton-Raphson (k-iterations) . . . 7
- 2.4 Equivalent Formulations: Variational vs. Force Balance . . . 8

3 Gradient Computation via Adjoint Method **9**

4 Baraff-Witkin Cloth Simulation Model [6] **15**
- 4.1 Preliminaries . . . 15
- 4.2 Stretch . . . 17
- 4.3 Shear . . . 18
- 4.4 Dihedral Bending . . . 18

5 Descent Methods [9] **20**
- 5.1 Existing Methods as Descent Methods . . . 20
- 5.2 Jacobi-Preconditioned Gradient Descent with Chebyshev Acceleration . . . 21

6 Primal/Dual Descent Methods [4] **23**
- 6.1 Equations of Motion . . . 23
- 6.2 The Primal Objective . . . 24
- 6.3 Quadratic Potentials . . . 24
- 6.4 Newton Descent . . . 26
- 6.5 Collision and Friction . . . 26
  - 6.5.1 Collision . . . 26
  - 6.5.2 Friction . . . 27
- 6.6 Dual Objective . . . 28

7 XPBD [1] **31**
- 7.1 Constraints . . . 33
  - 7.1.1 Distance Constraint . . . 33
- 7.2 Differentiable XPBD through Explicit Adjoint Method . . . 34
  - 7.2.1 1-iteration Case . . . 34
  - 7.2.2 k-iterations Case . . . 39
  - 7.2.3 k-iterations Case (second method) . . . 43
  - 7.2.4 DiffXPBD [10] . . . 46
  - 7.2.5 General Case with Collision Response . . . 47

8 Collisions **48**
8.1 Vertex Projection . . . 48
8.1.1 Sphere . . . 50
8.1.2 Capsule . . . 51
8.1.3 Halfspace . . . 51
8.2 Primitive-Based Collision Handling . . . 52
8.3 Collision Detection via Continuous CCD . . . 53
8.4 Collision Response as a Constrained Quadratic Problem . . . 54
8.4.1 Solution via Non-Rigid Impact Zones . . . 56

9 Implicit Differentiation **58**
9.1 Linear Solve . . . 58
9.2 Quadratic Program with only Linear Inequality Constraints . . . 60
9.3 Generic Constrained Quadratic Program . . . 61
9.4 Linear Complementary Problem . . . 61

10 Differentiable Cloth Simulation for Inverse Problems [5] **63**

11 Projective Dynamics (PD) [2] **64**
11.1 Constraint-Based Energy Potentials . . . 64
11.2 The Augmented Problem . . . 64
11.3 Strain Constraints for Cloth . . . 65
11.4 PD with Dry Frictional Contact [13] . . . 66
11.4.1 Velocity-Based PD . . . 66
11.4.2 Signorini–Coulomb Law . . . 67
11.4.3 Local Update of the Frictional Contact Forces . . . 68
11.5 Differentiable PD . . . 69
11.5.1 Backpropagation Through Implicit Time Integration . . . 69
11.5.2 DiffPD: Exploiting PD Structure in Backpropagation [14] 70
11.5.3 DiffCloth: DiffPD with Dry Frictional Contact [15] . . . 71
11.6 Differentiable Mass-Spring Systems via PD . . . 73
11.6.1 Forward PD . . . 73
11.6.2 Backward PD . . . 74
11.6.3 Implementation . . . 75

12 Incremental Potential Contact (IPC) [3] **79**
12.1 Contact Barrier Energy . . . 81
12.1.1 Collision Pair Distance Functions . . . 82
12.2 Computing Energy Jacobians and SPD-Projected Hessians . . . 82
12.2.1 Elastic Energy: First and Second Derivatives . . . 82
12.2.2 Contact Barrier: First and Second Derivatives . . . 86

Abstract

Physically based simulation of deformable bodies offers a spectrum of methods, each trading off physical accuracy against speed and stability. This report surveys these methods with a focus on cloth and thin shells, a class of deformable body whose high contact area and frequent self-collisions make accurate collision handling particularly challenging in interactive settings. Our broader motivation is the *inverse* problem: recovering internal cloth material parameters from real-world 3D-captured dynamics of garments in motion. This goal shapes the survey in two ways.

First, since the recovered parameters are intended for use in interactive environments, we concentrate primarily on methods capable of real-time forward simulation, examining XPBD [1] and Projective Dynamics [2] in depth, while also considering computationally heavier but more accurate approaches such as Incremental Potential Contact [3]. Second, solving inverse problems by gradient-based optimization requires the simulator to be *differentiable*, that is, able to compute gradients of an objective with respect to chosen parameters by backpropagating through the timestep integrator. For several of these simulation methods we therefore derive the corresponding gradient computation, relying on analytical gradients of the update formulas, the adjoint method, and implicit differentiation.

A recurring difficulty is the differentiation of collision response, which is inherently discontinuous. We analyze several collision-handling strategies and whether each can be differentiated, including post-projection corrections, penalty forces and constraints [4], a constrained quadratic-program formulation [5], and smooth barrier potentials [3]. The goal is to map the trade-offs between physical fidelity, interactive performance, and differentiability.

Introduction

Chapter 1 — Math Background. Collects the general mathematical tools used throughout the report, independent of any specific physical model or algorithm.

Chapter 2 — Implicit Simulator. Introduces implicit (BDF1) time integration for particle systems and shows how the resulting nonlinear equations of motion are discretized. It then presents two practical solution strategies: a single-linearization solve and the full multi-iteration Newton–Raphson scheme.

Chapter 3 — Gradient Computation via Adjoint Method. Derives the adjoint method from first principles as a means of computing gradients of an objective with respect to a simulator's parameters. The method is then specialized to explicit time-stepping functions.

Chapter 4 — Baraff–Witkin Cloth Simulation Model. Describes the constraint-based formulation of the Baraff–Witkin cloth model and its associated elastic energies. All components needed for a single-linear solve, namely the energy Jacobians and Hessians, are derived for the three cloth constraints: stretch, shear, and dihedral bending.

Chapter 5 — Descent Methods. Provides a general overview of descent-based formulations of implicit time integration for deformable bodies. It recasts several existing simulation methods within a unified descent framework.

Chapter 6 — Primal/Dual Descent Methods. Explains the distinction between primal and dual descent formulations of implicit Euler integration, deriving the objective, algorithm, and Newton update for each. It also presents a collision and friction handling scheme tailored to these descent methods.

Chapter 7 — XPBD. Describes and derives the XPBD simulation method starting from the dual objective of the previous chapter. It then develops the differentiable version of the algorithm for both single- and multi-iteration solves, focusing on gradients with respect to the constraint compliance parameters.

Chapter 8 — Collisions. Surveys strategies a cloth simulator can use to handle collisions and friction, operating at both the vertex level and the primitive level. For each approach it covers detection, response, and the corresponding derivatives required for gradient computation.

Chapter 9 — Implicit Differentiation. Presents the mathematics of implicit differentiation as a general framework for backpropagating. The framework is then specialized to different problems: a linear solve, a QP with linear

inequality constraints, a generic constrained QP, and a linear complementarity problem.

Chapter 10 — Differentiable Cloth Simulation for Inverse Problems. Describes the forward and backward simulation pipeline adapted from [5].

Chapter 11 — Projective Dynamics (PD). Describes and derives the Projective Dynamics method, with strain constraints specialized to cloth. It then covers its extension to dry frictional contact and the differentiable counterparts of both the base method and the contact formulation for gradient computation.

Chapter 12 — Incremental Potential Contact (IPC). Provides an in-depth description of the IPC method, including its smooth contact barrier energy and collision culling. It details the computation of energy Jacobians and SPD-projected Hessians, with the Stable Neo-Hookean model worked out in full.

1 Math Background

1.1 Newton-Raphson for Root-Finding

Given a smooth function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^n$, we seek $\mathbf{x}^* \in \mathbb{R}^n$ such that $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Starting from an initial guess $\mathbf{x}^{(0)}$, Newton-Raphson iteratively refines the estimate by linearizing $\mathbf{f}$ around the current iterate. At iteration k, the first-order Taylor expansion gives:

$$\mathbf{f}(\mathbf{x}^{(k)} + \delta\mathbf{x}) \approx \mathbf{f}(\mathbf{x}^{(k)}) + \left.\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right|_{\mathbf{x}^{(k)}} \delta\mathbf{x} \tag{1}$$

Setting this to zero and solving for the correction $\delta\mathbf{x}$ yields the linear system:

$$\left.\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right|_{\mathbf{x}^{(k)}} \delta\mathbf{x} = -\mathbf{f}(\mathbf{x}^{(k)}) \tag{2}$$

The iterate is then updated as $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \delta\mathbf{x}$. The process repeats until $\|\mathbf{f}(\mathbf{x}^{(k)})\| < \epsilon$.

Algorithm 1 Newton-Raphson Root-Finding

function NewtonRaphson($\mathbf{f}$, $\mathbf{x}^{(0)}$, ϵ, N)
  $\mathbf{x} \leftarrow \mathbf{x}^{(0)}$
  for $k = 0, 1, \ldots, N$ **do**
    $\mathbf{r} \leftarrow \mathbf{f}(\mathbf{x})$
    if $\|\mathbf{r}\| < \epsilon$ **then**
      return $\mathbf{x}$
    $\mathbf{A} \leftarrow \left.\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right|_{\mathbf{x}}$
    Solve $\mathbf{A}\,\delta\mathbf{x} = -\mathbf{r}$
    $\mathbf{x} \leftarrow \mathbf{x} + \delta\mathbf{x}$
  return $\mathbf{x}$
end function

2 Implicit Simulator

We model a system of n particles. The simulation state at time t is:

$$\mathbf{q}_t = \begin{bmatrix} \mathbf{x}_t \\ \mathbf{v}_t \end{bmatrix}, \qquad \mathbf{x}_t,\, \mathbf{v}_t \in \mathbb{R}^{3n},$$

where $\mathbf{x}_t$ and $\mathbf{v}_t$ are the positions and velocities of all particles. The state evolves according to Newton's second law, written as a first-order ODE system:

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{v} \\ \dot{\mathbf{v}} = \mathbf{M}^{-1}\mathbf{f}(\mathbf{x}, \mathbf{v}) \end{cases} \tag{3}$$

where $\mathbf{M} \in \mathbb{R}^{3n\times 3n}$ is the diagonal mass matrix and $\mathbf{f} : \mathbb{R}^{3n} \times \mathbb{R}^{3n} \to \mathbb{R}^{3n}$ encodes all internal and external forces acting on the system.

2.1 BDF1 Time-Discretization

A single simulation step $\mathbf{q}_{t+1} = g(\mathbf{q}_t)$ advances the state by timestep h using implicit Euler (BDF1) integration, which approximates the time derivative at time t as:

$$\dot{\mathbf{y}}_t \approx \frac{\mathbf{y}_{t+1} - \mathbf{y}_t}{h}$$

and evaluates the right-hand side at the *next* timestep $t+1$. Applying this to (3):

$$\begin{cases} \mathbf{x}_{t+1} = \mathbf{x}_t + h\,\mathbf{v}_{t+1} \\ \mathbf{M}(\mathbf{v}_{t+1} - \mathbf{v}_t) = h\,\mathbf{f}(\mathbf{x}_{t+1},\, \mathbf{v}_{t+1}) \end{cases} \tag{4}$$

Substituting the first equation into the second and introducing the velocity increment $\Delta\mathbf{v} = \mathbf{v}_{t+1} - \mathbf{v}_t$ as the primary unknown, the BDF1 timestep reduces to finding $\Delta\mathbf{v}$ satisfying:

$$\mathbf{r}(\Delta\mathbf{v}) = \mathbf{M}\,\Delta\mathbf{v} - h\,\mathbf{f}\big(\mathbf{x}_t + h(\mathbf{v}_t + \Delta\mathbf{v}),\ \mathbf{v}_t + \Delta\mathbf{v}\big) = \mathbf{0} \tag{5}$$

This is a nonlinear equation in $\Delta\mathbf{v}$, since $\mathbf{f}$ is generally nonlinear. The state is then recovered as:

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \Delta\mathbf{v}, \qquad \mathbf{x}_{t+1} = \mathbf{x}_t + h\,\mathbf{v}_{t+1}$$

2.2 Single-Linearization Solve (Newton-Raphson 1-iteration)

Discretizing (3) and solving implicitly for $\Delta\mathbf{v} = \mathbf{v}_{t+1} - \mathbf{v}_t$ yields:

$$\begin{cases} \underbrace{\left[\mathbf{M} - h\dfrac{\partial \mathbf{f}}{\partial \mathbf{v}} - h^2\dfrac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]}_{\mathbf{A}(\mathbf{x}_t,\,\mathbf{v}_t)}\Delta\mathbf{v} = \underbrace{h\left[\mathbf{f}(\mathbf{x}_t, \mathbf{v}_t) + h\dfrac{\partial \mathbf{f}}{\partial \mathbf{x}}\mathbf{v}_t\right]}_{\mathbf{b}(\mathbf{x}_t,\,\mathbf{v}_t)} \\ \mathbf{v}_{t+1} = \mathbf{v}_t + \Delta\mathbf{v} \\ \mathbf{x}_{t+1} = \mathbf{x}_t + h\,(\mathbf{v}_t + \Delta\mathbf{v}) \end{cases} \tag{6}$$

The velocity increment $\Delta\mathbf{v}$ is obtained by solving the sparse linear system $\mathbf{A}\,\Delta\mathbf{v} = \mathbf{b}$, where $\mathbf{A} \in \mathbb{R}^{3n\times 3n}$ and $\mathbf{b} \in \mathbb{R}^{3n}$ depend only on the current state $(\mathbf{x}_t, \mathbf{v}_t)$ and are recomputed at each timestep.

We want to discretize the ODE system (3) using implicit Euler (BDF1). Implicit Euler approximates the time derivative of a quantity $\mathbf{y}$ at time t as:

$$\dot{\mathbf{y}}_t \approx \frac{\mathbf{y}_{t+1} - \mathbf{y}_t}{h}$$

and evaluates the right-hand side at the *next* timestep $t+1$, rather than the current one as explicit Euler would. Applying this to both equations in (3):

$$\begin{cases} \dfrac{\mathbf{x}_{t+1} - \mathbf{x}_t}{h} = \mathbf{v}_{t+1} \\ \dfrac{\mathbf{v}_{t+1} - \mathbf{v}_t}{h} = \mathbf{M}^{-1}\mathbf{f}(\mathbf{x}_{t+1}, \mathbf{v}_{t+1}) \end{cases}$$

The first equation gives immediately:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + h\,\mathbf{v}_{t+1} \tag{7}$$

The second equation, multiplied through by $\mathbf{M}$, gives:

$$\mathbf{M}(\mathbf{v}_{t+1} - \mathbf{v}_t) = h\,\mathbf{f}(\mathbf{x}_{t+1}, \mathbf{v}_{t+1}) \tag{8}$$

This is a nonlinear equation in the unknowns $\mathbf{x}_{t+1}$ and $\mathbf{v}_{t+1}$, since $\mathbf{f}$ is generally nonlinear. To make it tractable, we linearize $\mathbf{f}$ around the current state $(\mathbf{x}_t, \mathbf{v}_t)$ using a first-order Taylor expansion:

$$\mathbf{f}(\mathbf{x}_{t+1}, \mathbf{v}_{t+1}) \approx \mathbf{f}(\mathbf{x}_t, \mathbf{v}_t) + \frac{\partial \mathbf{f}}{\partial \mathbf{x}}(\mathbf{x}_{t+1} - \mathbf{x}_t) + \frac{\partial \mathbf{f}}{\partial \mathbf{v}}(\mathbf{v}_{t+1} - \mathbf{v}_t)$$

where $\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \in \mathbb{R}^{3n\times 3n}$ and $\frac{\partial \mathbf{f}}{\partial \mathbf{v}} \in \mathbb{R}^{3n\times 3n}$ are the Jacobians of the force with respect to positions and velocities, both evaluated at $(\mathbf{x}_t, \mathbf{v}_t)$. We now introduce the velocity increment $\Delta\mathbf{v} = \mathbf{v}_{t+1} - \mathbf{v}_t$ as the primary unknown. From (7):

$$\mathbf{x}_{t+1} - \mathbf{x}_t = h\,\mathbf{v}_{t+1} = h\,(\mathbf{v}_t + \Delta\mathbf{v})$$

Substituting both into the Taylor expansion:

$$\mathbf{f}(\mathbf{x}_{t+1}, \mathbf{v}_{t+1}) \approx \mathbf{f}(\mathbf{x}_t, \mathbf{v}_t) + h\frac{\partial \mathbf{f}}{\partial \mathbf{x}}(\mathbf{v}_t + \Delta\mathbf{v}) + \frac{\partial \mathbf{f}}{\partial \mathbf{v}}\Delta\mathbf{v}$$

Substituting this linearized force into (8):

$$\mathbf{M}\,\Delta\mathbf{v} = h\left[\mathbf{f}(\mathbf{x}_t, \mathbf{v}_t) + h\frac{\partial \mathbf{f}}{\partial \mathbf{x}}(\mathbf{v}_t + \Delta\mathbf{v}) + \frac{\partial \mathbf{f}}{\partial \mathbf{v}}\Delta\mathbf{v}\right]$$

Expanding and collecting all terms involving $\Delta\mathbf{v}$ on the left-hand side:

$$\mathbf{M}\,\Delta\mathbf{v} - h\frac{\partial\mathbf{f}}{\partial\mathbf{v}}\Delta\mathbf{v} - h^2\frac{\partial\mathbf{f}}{\partial\mathbf{x}}\Delta\mathbf{v} = h\,\mathbf{f}(\mathbf{x}_t,\mathbf{v}_t) + h^2\frac{\partial\mathbf{f}}{\partial\mathbf{x}}\mathbf{v}_t$$

Factoring $\Delta\mathbf{v}$ on the left:

$$\underbrace{\left[\mathbf{M} - h\frac{\partial\mathbf{f}}{\partial\mathbf{v}} - h^2\frac{\partial\mathbf{f}}{\partial\mathbf{x}}\right]}_{\mathbf{A}(\mathbf{x}_t,\mathbf{v}_t)}\Delta\mathbf{v} = \underbrace{h\left[\mathbf{f}(\mathbf{x}_t,\mathbf{v}_t) + h\frac{\partial\mathbf{f}}{\partial\mathbf{x}}\mathbf{v}_t\right]}_{\mathbf{b}(\mathbf{x}_t,\mathbf{v}_t)}$$

which is the linear system $\mathbf{A}\,\Delta\mathbf{v} = \mathbf{b}$ in (6). Once $\Delta\mathbf{v}$ is obtained, the state is updated as:

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \Delta\mathbf{v} \qquad \mathbf{x}_{t+1} = \mathbf{x}_t + h\,\mathbf{v}_{t+1} = \mathbf{x}_t + h\,(\mathbf{v}_t + \Delta\mathbf{v})$$

Note that $\mathbf{A}$ plays the role of an effective mass matrix: it reduces to $\mathbf{M}$ when forces are constant (zero Jacobians), and its off-diagonal structure encodes how forces couple positions and velocities across the system.

Equivalently, solving for the position increment $\Delta\mathbf{x} = \mathbf{x}_{t+1} - \mathbf{x}_t$ as the primary unknown, the same discretization yields:

$$\begin{cases} \underbrace{\left[\mathbf{M} - h\frac{\partial\mathbf{f}}{\partial\mathbf{v}} - h^2\frac{\partial\mathbf{f}}{\partial\mathbf{x}}\right]}_{\mathbf{A}(\mathbf{x}_t,\mathbf{v}_t)}\Delta\mathbf{x} = \underbrace{h\left[h\,\mathbf{f}(\mathbf{x}_t,\mathbf{v}_t) + \mathbf{M}\mathbf{v}_t - h\frac{\partial\mathbf{f}}{\partial\mathbf{v}}\mathbf{v}_t\right]}_{\mathbf{b}(\mathbf{x}_t,\mathbf{v}_t)} \\ \mathbf{x}_{t+1} = \mathbf{x}_t + \Delta\mathbf{x} \\ \mathbf{v}_{t+1} = \frac{\Delta\mathbf{x}}{h} \end{cases} \tag{9}$$

The system matrix $\mathbf{A}$ is identical in both formulations; only the right-hand side and the recovered state differ. The two forms are related by the substitution $\Delta\mathbf{x} = h\,\Delta\mathbf{v} + h\mathbf{v}_t$.

2.3 Newton-Raphson (k-iterations)

The single-linearization solve of the previous section is equivalent to one iteration of Newton-Raphson (Section 1.1) applied to the residual (5), with initial guess $\Delta\mathbf{v}^{(0)} = \mathbf{0}$. This approximates the true BDF1 solution, but the linearization error can be significant for large timesteps or highly nonlinear forces. Running multiple Newton-Raphson iterations recovers the exact implicit Euler solution up to solver tolerance. The method is shown in Algorithm 2.

Initialization. The choice of initial guess $\Delta\mathbf{v}^{(0)}$ affects the number of iterations needed for convergence. Common strategies are:

- **Cold start:** $\Delta\mathbf{v}^{(0)} = \mathbf{0}$. Safe but slow, since the initial residual is $\mathbf{r}(\mathbf{0}) = -h\,\mathbf{f}(\mathbf{x}_t,\mathbf{v}_t)$.

- **Explicit Euler predictor:** $\Delta\mathbf{v}^{(0)} = h\,\mathbf{M}^{-1}\mathbf{f}(\mathbf{x}_t, \mathbf{v}_t)$. One forward Euler step, which provides a first-order accurate estimate of the velocity increment at negligible cost since $\mathbf{M}$ is diagonal.
- **Warm start:** $\Delta\mathbf{v}^{(0)} = \Delta\mathbf{v}^{-}$, reusing the converged velocity increment from the previous timestep. This exploits temporal coherence and is effective when the simulation evolves smoothly.

Algorithm 2 Newton-Raphson method applied to discretized equations of motion for deformable bodies.

function SIMULATIONSTEPNEWTONRAPHSON($\mathbf{x}^{-}, \mathbf{v}^{-}, \Delta\mathbf{v}^{-}, N$)
  INITIALIZE $\Delta\mathbf{v}(\mathbf{x}^{-}, \mathbf{v}^{-}, \Delta\mathbf{v}^{-})$
  $\mathbf{v} \leftarrow \mathbf{v}^{-} + \Delta\mathbf{v}$
  $\mathbf{x} \leftarrow \mathbf{x}^{-} + h\mathbf{v}$
  for $k = 0, 1, \ldots, N$ **do**
    $\mathbf{r} \leftarrow \mathbf{M}\Delta\mathbf{v} - h\,\mathbf{f}(\mathbf{x}, \mathbf{v})$
    if $\|\mathbf{r}\| < \epsilon$ **then**
      return $[\mathbf{x}, \mathbf{v}]$
    $\mathbf{A} \leftarrow \mathbf{M} - h\,\frac{\partial \mathbf{f}}{\partial \mathbf{v}}\big|_{[\mathbf{x},\mathbf{v}]} - h^2\,\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\big|_{[\mathbf{x},\mathbf{v}]}$
    SOLVE $\mathbf{A}\,\delta\mathbf{v} = -\mathbf{r}$
    $\Delta\mathbf{v} \leftarrow \Delta\mathbf{v} + \delta\mathbf{v}$
    $\mathbf{v} \leftarrow \mathbf{v}^{-} + \Delta\mathbf{v}$
    $\mathbf{x} \leftarrow \mathbf{x}^{-} + h\mathbf{v}$
  return $[\mathbf{x}, \mathbf{v}]$
end function

2.4 Equivalent Formulations: Variational vs. Force Balance

$$\mathbf{r}(\mathbf{x}_{t+1}) = \frac{1}{h}\mathbf{M}\left(\mathbf{x}_{t+1} - \tilde{\mathbf{x}}\right) - h\,\mathbf{f}\left(\mathbf{x}_{t+1}\right) = \mathbf{0} \tag{10}$$

$$\mathbf{x}_{t+1} = \arg\min_{\mathbf{x}} g(\mathbf{x}) = \frac{1}{2h^2}(\mathbf{x} - \tilde{\mathbf{x}})^{\top}\mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x}) \tag{11}$$

3 Gradient Computation via Adjoint Method

We derive the gradients using the adjoint method as formulated in Derivation 3. Consider a simulation of s steps with state $\mathbf{q}_i$ and control parameters $\mathbf{u}$. Given an objective function $\phi(\mathbf{Q}, \mathbf{u})$, we seek the total derivative $\frac{d\phi}{d\mathbf{u}}$ to optimize $\mathbf{u}$ via gradient descent.

The adjoint method reduces this computation to a single backward pass, regardless of the dimension of $\mathbf{u}$, by introducing adjoint states $\mathbf{r}_i$ that propagate gradient information backwards in time. The total gradient decomposes as:

$$\frac{d\phi}{d\mathbf{u}} = \sum_{t=1}^{s} \mathbf{r}_t^\top \frac{\partial \mathbf{f}_{t-1}}{\partial \mathbf{u}} + \frac{\partial \phi}{\partial \mathbf{u}} \tag{12}$$

where the adjoint states satisfy the backward recursion:

$$\mathbf{r}_t = \begin{cases} \left(\frac{\partial \phi}{\partial \mathbf{q}_t}\right)^\top & t = s \\ \left(\frac{\partial \mathbf{f}_t}{\partial \mathbf{q}_t}\right)^\top \mathbf{r}_{t+1} + \left(\frac{\partial \phi}{\partial \mathbf{q}_t}\right)^\top & t < s \end{cases} \tag{13}$$

The term $\left(\frac{\partial \phi}{\partial \mathbf{q}_t}\right)^\top$ vanishes at all timesteps where ϕ does not explicitly depend on the state $\mathbf{q}_t$.

Explicit Adjoint Method Derivation We derive the adjoint method for differentiable simulators. The key objects are:

- $\mathbf{q}_t \in \mathbb{R}^{n_q}$: system state at timestep t. For a 3D cloth of n particles, $n_q = 3n + 3n$ (positions and velocities).
- $\mathbf{f}_t \in \mathbb{R}^{n_q} \times \mathbb{R}^m \to \mathbb{R}^{n_q}$: the time-stepping function, defined by:

$$\mathbf{q}_{t+1} = \mathbf{f}_t(\mathbf{q}_t, \mathbf{u}).$$

- $\mathbf{u} \in \mathbb{R}^m$: vector of m control parameters, assumed to be constant throughout the simulation.
- $\mathbf{Q} = [\mathbf{q}_1^\top, \mathbf{q}_2^\top, \ldots, \mathbf{q}_s^\top]^\top \in \mathbb{R}^{n_q \cdot s}$: concatenation of all computed states, where s is the number of simulated steps.
- $\mathbf{F}(\mathbf{Q}, \mathbf{u}) = [\mathbf{f}_0(\mathbf{q}_0, \mathbf{u})^\top, \ldots, \mathbf{f}_{s-1}(\mathbf{q}_{s-1}, \mathbf{u})^\top]^\top \in \mathbb{R}^{n_q \cdot s}$: concatenation of all time-stepping functions.
- $\phi(\mathbf{Q}, \mathbf{u}) \in \mathbb{R}^{n_q \cdot s} \times \mathbb{R}^m \to \mathbb{R}$: the objective function.

These definitions allow us to write the simulation constraint compactly as:

$$\mathbf{Q} = \mathbf{F}(\mathbf{Q}, \mathbf{u}) \tag{14}$$

We want to compute how the objective $\phi(\mathbf{Q}, \mathbf{u})$ changes with respect to the control $\mathbf{u}$. Since ϕ depends on $\mathbf{u}$ both explicitly ($\mathbf{u}$ appears directly in ϕ) and implicitly ($\mathbf{u}$ affects $\mathbf{Q}$, which in turn affects ϕ) we must account for both contributions. Applying the total derivative via the chain rule:

$$\frac{d\phi}{d\mathbf{u}} = \frac{\partial \phi}{\partial \mathbf{Q}} \frac{d\mathbf{Q}}{d\mathbf{u}} + \frac{\partial \phi}{\partial \mathbf{u}} \tag{15}$$

The term $\frac{d\mathbf{Q}}{d\mathbf{u}} \in \mathbb{R}^{(n_q \cdot s) \times m}$ is a large matrix encoding the sensitivity of every state at every timestep to every control parameter. Computing it directly is intractable for large simulations.

We differentiate the constraint equation 14 over $\mathbf{u}$ and obtain the equation:

$$\frac{d\mathbf{Q}}{d\mathbf{u}} = \frac{\partial \mathbf{F}}{\partial \mathbf{Q}} \frac{d\mathbf{Q}}{d\mathbf{u}} + \frac{\partial \mathbf{F}}{\partial \mathbf{u}}$$

$$\left(\mathbf{I} - \frac{\partial \mathbf{F}}{\partial \mathbf{Q}}\right) \frac{d\mathbf{Q}}{d\mathbf{u}} = \frac{\partial \mathbf{F}}{\partial \mathbf{u}}$$

So if we compose the interesting part of the full derivative equation 15 and this derived constraint we obtain the problem:

$$\frac{\partial \phi}{\partial \mathbf{Q}} \frac{d\mathbf{Q}}{d\mathbf{u}} \qquad \text{S.T.} \qquad \left(\mathbf{I} - \frac{\partial \mathbf{F}}{\partial \mathbf{Q}}\right) \frac{d\mathbf{Q}}{d\mathbf{u}} = \frac{\partial \mathbf{F}}{\partial \mathbf{u}}$$

This problem can be simplified using the principal of duality, obtaining the equivalent (but easier) problem:

$$\mathbf{R}^\top \frac{\partial \mathbf{F}}{\partial \mathbf{u}} \qquad \text{S.T} \qquad \left(\mathbf{I} - \frac{\partial \mathbf{F}}{\partial \mathbf{Q}}\right)^\top \mathbf{R} = \frac{\partial \phi}{\partial \mathbf{Q}}^\top \tag{16}$$

Where $\mathbf{R} \in \mathbb{R}^{n_q \cdot s}$ is the adjoint states long vector. So if we find a way to calculate $\mathbf{R}$ we can solve the original problem. Because:

$$\mathbf{R}^\top \frac{\partial \mathbf{F}}{\partial \mathbf{u}} = \frac{\partial \phi}{\partial \mathbf{Q}} \frac{d\mathbf{Q}}{d\mathbf{u}}$$

we have that the derivative we were interesting in becomes:

$$\frac{d\phi}{d\mathbf{u}} = \mathbf{R}^\top \frac{\partial \mathbf{F}}{\partial \mathbf{u}} + \frac{\partial \phi}{\partial \mathbf{u}} \tag{17}$$

Now we need to find an easy way to calculate the adjoint vector $\mathbf{R}$. We

start from the constraint of the adjoint problem 16:

$$\left(\mathbf{I} - \frac{\partial \mathbf{F}}{\partial \mathbf{Q}}\right)^{\top} \mathbf{R} = \frac{\partial \phi}{\partial \mathbf{Q}}^{\top}$$

$$\mathbf{R} = \left(\frac{\partial \mathbf{F}}{\partial \mathbf{Q}}\right)^{\top} \mathbf{R} + \frac{\partial \phi}{\partial \mathbf{Q}}^{\top}$$

We then notice the sparse structure of $\frac{\partial \mathbf{F}}{\partial \mathbf{Q}}$:

$$\frac{\partial \mathbf{F}}{\partial \mathbf{Q}} = \begin{bmatrix} \frac{\partial \mathbf{f}_0}{\partial \mathbf{q}_1} & \frac{\partial \mathbf{f}_0}{\partial \mathbf{q}_2} & \cdots & \frac{\partial \mathbf{f}_0}{\partial \mathbf{q}_s} \\ \frac{\partial \mathbf{f}_1}{\partial \mathbf{q}_1} & \frac{\partial \mathbf{f}_1}{\partial \mathbf{q}_2} & \cdots & \frac{\partial \mathbf{f}_1}{\partial \mathbf{q}_s} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \mathbf{f}_{s-1}}{\partial \mathbf{q}_1} & \frac{\partial \mathbf{f}_{s-1}}{\partial \mathbf{q}_2} & \cdots & \frac{\partial \mathbf{f}_{s-1}}{\partial \mathbf{q}_s} \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & \cdots & 0 & 0 & 0 \\ \frac{\partial \mathbf{f}_1}{\partial \mathbf{q}_1} & 0 & \cdots & 0 & 0 & 0 \\ 0 & \frac{\partial \mathbf{f}_2}{\partial \mathbf{q}_2} & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & \frac{\partial \mathbf{f}_{s-1}}{\partial \mathbf{q}_{s-1}} & 0 \end{bmatrix}$$

where the lower off diagonal contains the derivative of the time stepping function with respect to the input state, and all other evaluates to 0. So

in explicit matrix form the problem of finding $\mathbf{R}$ becomes:

$$\begin{bmatrix} \mathbf{r}_1 \\ \mathbf{r}_2 \\ \vdots \\ \mathbf{r}_{s-1} \\ \mathbf{r}_s \end{bmatrix} = \begin{bmatrix} 0 & \frac{\partial \mathbf{f}_1}{\partial \mathbf{q}_1}^\top & 0 & \cdots & 0 \\ 0 & 0 & \frac{\partial \mathbf{f}_2}{\partial \mathbf{q}_2}^\top & \cdots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \cdots & \frac{\partial \mathbf{f}_{s-1}}{\partial \mathbf{q}_{s-1}}^\top \\ 0 & 0 & 0 & \cdots & 0 \end{bmatrix} \begin{bmatrix} \mathbf{r}_1 \\ \mathbf{r}_2 \\ \vdots \\ \mathbf{r}_{s-1} \\ \mathbf{r}_s \end{bmatrix} + \begin{bmatrix} \frac{\partial \phi}{\partial \mathbf{q}_1} \\ \frac{\partial \phi}{\partial \mathbf{q}_2} \\ \vdots \\ \frac{\partial \phi}{\partial \mathbf{q}_{s-1}} \\ \frac{\partial \phi}{\partial \mathbf{q}_s} \end{bmatrix}$$

$$= \begin{bmatrix} \left(\frac{\partial \mathbf{f}_1}{\partial \mathbf{q}_1}\right)^\top \mathbf{r}_2 + \frac{\partial \phi}{\partial \mathbf{q}_1} \\ \left(\frac{\partial \mathbf{f}_2}{\partial \mathbf{q}_2}\right)^\top \mathbf{r}_3 + \frac{\partial \phi}{\partial \mathbf{q}_2} \\ \vdots \\ \left(\frac{\partial \mathbf{f}_{s-1}}{\partial \mathbf{q}_{s-1}}\right)^\top \mathbf{r}_s + \frac{\partial \phi}{\partial \mathbf{q}_{s-1}} \\ \frac{\partial \phi}{\partial \mathbf{q}_s} \end{bmatrix}$$

From which we generalize:

$$\begin{aligned} \mathbf{r}_t &= \left(\frac{\partial \mathbf{f}_t}{\partial \mathbf{q}_t}\right)^\top \mathbf{r}_{t+1} + \frac{\partial \phi}{\partial \mathbf{q}_t} \qquad \text{if: } t < s \\ \mathbf{r}_s &= \frac{\partial \phi}{\partial \mathbf{q}_s} \end{aligned} \tag{18}$$

The remainder of this section derives $(\frac{\partial g}{\partial q_t})^\top$ explicitly for the implicit Euler integrator defined in equation **??**. The partial derivatives of A and b with respect to x_t and v_t are:

$$\begin{cases} \frac{\partial b}{\partial x} = h\left[\frac{\partial f}{\partial x} + h\frac{\partial^2 f}{\partial x^2}v\right] \approx h\frac{\partial f}{\partial x} \\ \frac{\partial b}{\partial v} = h\left[\frac{\partial f}{\partial v} + h\frac{\partial f}{\partial x} + h\frac{\partial^2 f}{\partial x \partial v}\right] \approx h\frac{\partial f}{\partial v} + h^2\frac{\partial f}{\partial x} \\ \frac{\partial A}{\partial x} = -h\frac{\partial^2 f}{\partial v \partial x} - h^2\frac{\partial^2 f}{\partial x^2} \approx 0 \\ \frac{\partial A}{\partial v} = -h\frac{\partial^2 f}{\partial v^2} - h^2\frac{\partial^2 f}{\partial x \partial v} \approx 0 \end{cases} \tag{19}$$

Terms in red are dropped for two reasons. The $\frac{\partial A}{\partial x}$ and $\frac{\partial A}{\partial v}$ terms contribute

$O(h^2)$ corrections when multiplied by $\Delta v = O(h)$ in the implicit differentiation formula, making them negligible for small h. The remaining red terms involve second derivatives of f, which are expensive to compute and rarely available in closed form.

Applying implicit differentiation to $A\,\Delta v = b$, dropping the $\frac{\partial A}{\partial x}$ and $\frac{\partial A}{\partial v}$ terms and substituting the approximations for $\frac{\partial b}{\partial x}$ and $\frac{\partial b}{\partial v}$:

$$\begin{cases} A\dfrac{\partial\,\Delta v}{\partial\,x} = \dfrac{\partial\,b}{\partial\,x} - \dfrac{\partial\,A}{\partial\,x}\Delta v \approx h\dfrac{\partial\,f}{\partial\,x} \\ A\dfrac{\partial\,\Delta v}{\partial\,v} = \dfrac{\partial\,b}{\partial\,v} - \dfrac{\partial\,A}{\partial\,v}\Delta v \approx h\dfrac{\partial\,f}{\partial\,v} + h^2\dfrac{\partial\,f}{\partial\,x} \end{cases} \tag{20}$$

As shown later, neither system needs to be solved explicitly.

The adjoint state update requires $\frac{\partial g}{\partial q}$, the Jacobian of the simulation step with respect to the state:

$$\frac{\partial\,g}{\partial\,q} = \begin{bmatrix} \dfrac{\partial\,x_{t+1}}{\partial\,x_t} & \dfrac{\partial\,x_{t+1}}{\partial\,v_t} \\ \dfrac{\partial\,v_{t+1}}{\partial\,x_t} & \dfrac{\partial\,v_{t+1}}{\partial\,v_t} \end{bmatrix} \tag{21}$$

Differentiating the update equations $v_{t+1} = v_t + \Delta v$ and $x_{t+1} = x_t + h\,v_{t+1}$ with respect to x_t and v_t:

$$\begin{cases} \dfrac{\partial\,x_{t+1}}{\partial\,x_t} = I + h\dfrac{\partial\,\Delta v}{\partial\,x_t} \\ \dfrac{\partial\,x_{t+1}}{\partial\,v_t} = h\left(I + \dfrac{\partial\,\Delta v}{\partial\,v_t}\right) \\ \dfrac{\partial\,v_{t+1}}{\partial\,x_t} = \dfrac{\partial\,\Delta v}{\partial\,x_t} \\ \dfrac{\partial\,v_{t+1}}{\partial\,v_t} = I + \dfrac{\partial\,\Delta v}{\partial\,v_t} \end{cases} \tag{22}$$

Expanding equation **??** and noting that the transpose swaps the off-diagonal blocks, we can write:

$$r_t = \begin{bmatrix} r_t^x \\ r_t^v \end{bmatrix} = \begin{bmatrix} \dfrac{\partial\,x_{t+1}}{\partial\,x_t}^\top & \dfrac{\partial\,v_{t+1}}{\partial\,x_t}^\top \\ \dfrac{\partial\,x_{t+1}}{\partial\,v_t}^\top & \dfrac{\partial\,v_{t+1}}{\partial\,v_t}^\top \end{bmatrix} \begin{bmatrix} r_{t+1}^x \\ r_{t+1}^v \end{bmatrix} = \begin{bmatrix} r_{t+1}^x + \left(\dfrac{\partial\,\Delta v}{\partial\,x_t}\right)^\top \lambda \\ \left(I + \dfrac{\partial\,\Delta v}{\partial\,v_t}\right)^\top \lambda \end{bmatrix} \tag{23}$$

where $\lambda = h\,r_{t+1}^x + r_{t+1}^v$ collects the repeated factor. Substituting the expressions for $\frac{\partial \Delta v}{\partial x_t}$ and $\frac{\partial \Delta v}{\partial v_t}$ from equation 20 and letting $\mu_t = A^{-T}\lambda_t$, obtained via a single

solve of $A^\top \mu_t = \lambda_t$, yields the final adjoint update:

$$\boxed{r_t = \begin{bmatrix} r_t^x \\ r_t^v \end{bmatrix} = \begin{bmatrix} r_{t+1}^x + h\left(\frac{\partial f}{\partial x_t}\right)^\top \mu_t \\ \lambda_t + \left(h\frac{\partial f}{\partial v_t} + h^2\frac{\partial f}{\partial x_t}\right)^\top \mu_t \end{bmatrix} + \begin{bmatrix} \left(\frac{\partial \mathcal{L}}{\partial x_t}\right)^\top \\ \left(\frac{\partial \mathcal{L}}{\partial v_t}\right)^\top \end{bmatrix}} \tag{24}$$

The backward pass requires one solve with $A^\top$ per timestep plus two matrix-vector products with $\frac{\partial f}{\partial x_t}$ and $\frac{\partial f}{\partial v_t}$. Since $A^\top = M - h\left(\frac{\partial f}{\partial v_t}\right)^\top - h^2\left(\frac{\partial f}{\partial x_t}\right)^\top$, storing the sparse force Jacobians $\frac{\partial f}{\partial x_t}$ and $\frac{\partial f}{\partial v_t}$ at each forward timestep is sufficient for the entire backward pass.

4 Baraff-Witkin Cloth Simulation Model [6]

Internal cloth forces are derived from a vector condition $\mathbf{C}(x) : \mathbb{R}^{3n} \to \mathbb{R}^c$ which encodes a desired geometric state and should be zero at rest. The associated elastic energy and its induced force are:

$$\Psi_{\mathbf{C}}(x) = \frac{k_{\mathbf{C}}}{2}\,\mathbf{C}(x)^\top \mathbf{C}(x) \qquad f_{\mathbf{C}} = -\frac{\partial\,\Psi_{\mathbf{C}}}{\partial\,x} = -k_{\mathbf{C}}\frac{\partial\,\mathbf{C}}{\partial\,x}\,\mathbf{C}(x) \tag{25}$$

The force Jacobian is a $\mathbb{R}^{3n\times 3n}$ sparse block matrix. Each 3×3 block ij is:

$$\frac{\partial\,f_{\mathbf{C}_j}}{\partial\,x_i} = -\frac{\partial^2\,\Psi_{\mathbf{C}}}{\partial\,x_i \partial\,x_j} = -k_{\mathbf{C}}\left[\frac{\partial\,\mathbf{C}}{\partial\,x_i}\frac{\partial\,\mathbf{C}}{\partial\,x_j}^\top + \frac{\partial^2\,\mathbf{C}}{\partial\,x_i \partial\,x_j}\mathbf{C}(x)\right] \tag{26}$$

Blocks ij are nonzero only for particles i, j that $\mathbf{C}$ depends on, keeping the matrix sparse.

The damping force is derived from the rate of change of $\mathbf{C}$:

$$\dot{\mathbf{C}}(x,v) = \left(\frac{\partial\,\mathbf{C}}{\partial\,x}\right)^\top v \qquad f_{\dot{\mathbf{C}}} = -k_{\dot{\mathbf{C}}}\frac{\partial\,\mathbf{C}}{\partial\,x}\,\dot{\mathbf{C}}(x,v) \tag{27}$$

The force Jacobian blocks ij with respect to positions and velocities are:

$$\frac{\partial\,f_{\dot{\mathbf{C}}_j}}{\partial\,x_i} = -k_{\dot{\mathbf{C}}}\left[\frac{\partial\,\mathbf{C}}{\partial\,x_i}\frac{\partial\,\dot{\mathbf{C}}}{\partial\,x_j}^\top + \frac{\partial^2\,\mathbf{C}}{\partial\,x_i \partial\,x_j}\dot{\mathbf{C}}(x,v)\right] \approx 0 \tag{28}$$

$$\frac{\partial\,f_{\dot{\mathbf{C}}_j}}{\partial\,v_i} = -k_{\dot{\mathbf{C}}}\frac{\partial\,\mathbf{C}}{\partial\,x_i}\frac{\partial\,\mathbf{C}}{\partial\,x_j}^\top \tag{29}$$

The term in red breaks symmetry of the Jacobian and is dropped ([6]). The term orange is also sometimes dropped [7], leaving $\frac{\partial f_{\dot{\mathbf{C}}}}{\partial x} \approx 0$ and a symmetric $\frac{\partial f_{\dot{\mathbf{C}}}}{\partial v}$.

4.1 Preliminaries

In the following we will mix the classic Baraff-Witkin notation with the one used in [7] and [8]. A cloth triangle is defined by its rest-state vertices $\bar{\mathbf{x}}_0, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2 \in \mathbb{R}^2$ in material space and its world-space vertices $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^3$ during simulation (such that $\mathbf{x} = [\,\mathbf{x}_0 \mid \mathbf{x}_1 \mid \mathbf{x}_2\,] \in \mathbb{R}^{3\times 3}$). We define the material and world edge matrices:

$$\mathbf{D}_M = [\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_0 \mid \bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_0] \in \mathbb{R}^{2\times 2} \qquad \mathbf{D}_S = [\mathbf{x}_1 - \mathbf{x}_0 \mid \mathbf{x}_2 - \mathbf{x}_0] \in \mathbb{R}^{3\times 2} \tag{30}$$

$$\mathbf{D}_M^{-1} = \begin{bmatrix} \tilde{d}_{00} & \tilde{d}_{01} \\ \tilde{d}_{10} & \tilde{d}_{11} \end{bmatrix} \in \mathbb{R}^{2\times 2} \tag{31}$$

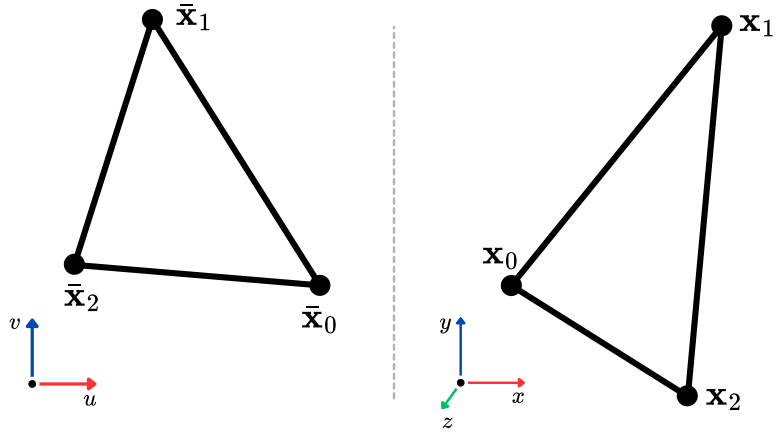

Figure 1: Mapping of a cloth element from 2D material space coordinates (u, v) to 3D world space coordinates (x, y, z).

The deformation gradient $\mathbf{F} \in \mathbb{R}^{3\times 2}$ maps material vectors to world-space vectors and is computed as:

$$\mathbf{F} = \mathbf{D}_S \mathbf{D}_M^{-1} = [\mathbf{F}_0 \mid \mathbf{F}_1] \in \mathbb{R}^{3\times 2} \tag{32}$$

$\mathbf{D}_M$ depends only on the rest state and is precomputed. The condition functions are written in terms of the columns of $\mathbf{F}$.
To compute the derivatives of the condition functions we need $\frac{\partial \mathbf{F}}{\partial \mathbf{x}}$. Since $\mathbf{D}_M^{-1}$ is constant:

$$\frac{\partial \mathbf{F}}{\partial \mathbf{x}} = \left[\frac{\partial \mathbf{F}_0}{\partial \mathbf{x}} \,\middle|\, \frac{\partial \mathbf{F}_1}{\partial \mathbf{x}}\right] = \frac{\partial \mathbf{D}_S}{\partial \mathbf{x}} \mathbf{D}_M^{-1} \in \mathbb{R}^{3\times 2\times 3\times 3}$$

The derivatives $\frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{ij}}$ are sparse 3×2 matrices with ± 1 entries:

$$\frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{00}} = \begin{bmatrix} -1 & -1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \quad \frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{01}} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \quad \frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{02}} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{10}} = \begin{bmatrix} 0 & 0 \\ -1 & -1 \\ 0 & 0 \end{bmatrix} \quad \frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{11}} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} \quad \frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{12}} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

$$\frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{20}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ -1 & -1 \end{bmatrix} \quad \frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{21}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} \quad \frac{\partial \mathbf{D}_S}{\partial \mathbf{x}_{22}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}$$

where x_{ij} denotes the j-th component of vertex x_i. Multiplying by $\mathbf{D}_M^{-1}$, each

block $\frac{\partial \mathbf{F}_k}{\partial \mathbf{x}_i} \in \mathbb{R}^{3\times 3}$ reduces to a scalar multiple of the identity:

$$\frac{\partial \mathbf{F}_0}{\partial \mathbf{x}_0} = -(\tilde{d}_{00} + \tilde{d}_{10})\,\mathbf{I}_{3\times 3} \qquad \frac{\partial \mathbf{F}_1}{\partial \mathbf{x}_0} = -(\tilde{d}_{01} + \tilde{d}_{11})\,\mathbf{I}_{3\times 3}$$

$$\frac{\partial \mathbf{F}_0}{\partial \mathbf{x}_1} = \tilde{d}_{00}\,\mathbf{I}_{3\times 3} \qquad \frac{\partial \mathbf{F}_1}{\partial \mathbf{x}_1} = \tilde{d}_{01}\,\mathbf{I}_{3\times 3}$$

$$\frac{\partial \mathbf{F}_0}{\partial \mathbf{x}_2} = \tilde{d}_{10}\,\mathbf{I}_{3\times 3} \qquad \frac{\partial \mathbf{F}_1}{\partial \mathbf{x}_2} = \tilde{d}_{11}\,\mathbf{I}_{3\times 3}$$

Since all blocks are scalar multiples of $\mathbf{I}_{3\times 3}$, we compress the coefficients into a single precomputed 3×2 matrix:

$$\left.\frac{\partial \mathbf{F}}{\partial \mathbf{x}}\right|_{\text{comp}} = \begin{bmatrix} -(\tilde{d}_{00} + \tilde{d}_{10}) & -(\tilde{d}_{01} + \tilde{d}_{11}) \\ \tilde{d}_{00} & \tilde{d}_{01} \\ \tilde{d}_{10} & \tilde{d}_{11} \end{bmatrix} \equiv \left[\hat{\mathbf{d}}_0 \mid \hat{\mathbf{d}}_1\right] \in \mathbb{R}^{3\times 2} \tag{33}$$

so that $\frac{\partial \mathbf{F}_k}{\partial \mathbf{x}_i} = \hat{\mathbf{d}}_{ik}\,\mathbf{I}_{3\times 3}$, where $\hat{\mathbf{d}}_{ik}$ is the (k, i) entry of this matrix.

4.2 Stretch

Stretch along the u axis is measured by how much $\|\mathbf{F}_0\|$ deviates from 1 (its rest value). The condition function, energy, and their derivatives are:

$$\mathbf{C}_u(\mathbf{x}) = \|\mathbf{F}_0\| - 1 \in \mathbb{R} \tag{34}$$

$$\Psi_{\text{stretch}_u}(\mathbf{x}) = \frac{a\,k_u}{2}\,\mathbf{C}_u(\mathbf{x})^2 \tag{35}$$

where a is the triangle's rest area in material space. The first and second derivatives with respect to vertex positions are:

$$\frac{\partial \mathbf{C}_u}{\partial \mathbf{x}_i} = \hat{\mathbf{d}}_{0i}\,\hat{\mathbf{F}}_0 \in \mathbb{R}^3 \tag{36}$$

$$\frac{\partial^2 \mathbf{C}_u}{\partial \mathbf{x}_i \partial \mathbf{x}_j} = \frac{\hat{\mathbf{d}}_{0i}\,\hat{\mathbf{d}}_{0j}}{\|\mathbf{F}_0\|}\left(\mathbf{I} - \hat{\mathbf{F}}_0\hat{\mathbf{F}}_0^\top\right) \in \mathbb{R}^{3\times 3} \tag{37}$$

where $\hat{\mathbf{F}}_0 = \mathbf{F}_0/\|\mathbf{F}_0\|$. The term $(\mathbf{I} - \hat{\mathbf{F}}_0\hat{\mathbf{F}}_0^\top)$ is the projection onto the plane perpendicular to $\mathbf{F}_0$. Forces, force Jacobians, and damping follow directly from equations 25, 26, 27, 28 with stiffness constants k_u and $\dot{k}_u$.

The v axis is identical, substituting $\mathbf{F}_1$, $\hat{\mathbf{d}}_1$, and stiffness constants k_v, $\dot{k}_v$, allowing anisotropic material behavior.

4.3 Shear

Shear is measured by the inner product of the two deformation gradient columns, which is zero at rest. The condition function, energy, and derivative are:

$$\mathbf{C}_s(\mathbf{x}) = \hat{\mathbf{F}}_0 \cdot \hat{\mathbf{F}}_1 \in \mathbb{R} \tag{38}$$

$$\Psi_{\text{shear}}(\mathbf{x}) = \frac{a\,k_s}{2}\,\mathbf{C}_s(\mathbf{x})^2 \tag{39}$$

$$\frac{\partial\,\mathbf{C}_s}{\partial\,\mathbf{x}_i} = \frac{1}{\|\mathbf{F}_0\|}\frac{\partial\,\mathbf{F}_0}{\partial\,\mathbf{x}_i}\text{proj}^{\perp}(\hat{\mathbf{F}}_0,\,\hat{\mathbf{F}}_1) + \frac{1}{\|\mathbf{F}_1\|}\frac{\partial\,\mathbf{F}_1}{\partial\,\mathbf{x}_i}\text{proj}^{\perp}(\hat{\mathbf{F}}_1,\,\hat{\mathbf{F}}_0) \in \mathbb{R}^3 \tag{40}$$

where $\text{proj}^{\perp}(\mathbf{a}, \hat{\mathbf{b}}) = \mathbf{a} - (\mathbf{a} \cdot \hat{\mathbf{b}})\hat{\mathbf{b}}$ is the component of $\mathbf{a}$ perpendicular to $\hat{\mathbf{b}}$. The second derivative $\dfrac{\partial^2\,\mathbf{C}_s}{\partial\,\mathbf{x}_i\partial\,\mathbf{x}_j}$ is not derived since the corresponding term in equation 26 is dropped for shear. Forces, Jacobians, and damping follow from equations 25–28 with stiffness constants k_s and $\dot{k}_s$.

4.4 Dihedral Bending

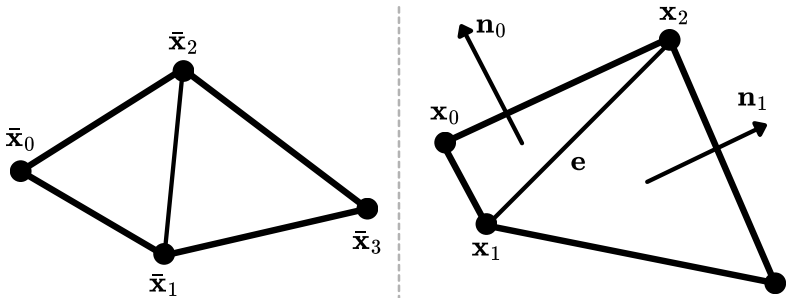

Figure 2: Geometric configuration for dihedral bending involving four vertices and two adjacent triangles.

Bending is defined over two adjacent triangles sharing a common edge, involving 4 unique vertices. We define:

$$\mathbf{x}_{i\to j} = \mathbf{x}_j - \mathbf{x}_i \qquad \mathbf{n}_0 = \mathbf{x}_{0\to 1} \times \mathbf{x}_{0\to 2} \qquad \mathbf{n}_1 = \mathbf{x}_{3\to 2} \times \mathbf{x}_{3\to 1}$$

$$\mathbf{e} = \mathbf{x}_{1\to 2}$$

where $\mathbf{n}_0$, $\mathbf{n}_1$ are the triangle normals and $\mathbf{e}$ is the shared edge. The dihedral angle θ between the two triangles is recovered via:

$$\sin\theta = (\hat{\mathbf{n}}_0 \times \hat{\mathbf{n}}_1) \cdot \hat{\mathbf{e}} \qquad \cos\theta = \hat{\mathbf{n}}_0 \cdot \hat{\mathbf{n}}_1 \qquad \theta(\mathbf{x}) = \arctan\left(\frac{\sin\theta}{\cos\theta}\right) \tag{41}$$

The condition, with θ_r being the rest angle, and the associated energy are:

$$\mathbf{C}_b(\mathbf{x}) = \theta(\mathbf{x}) - \theta_r \in \mathbb{R} \qquad \Psi_{\text{bend}}(\mathbf{x}) = \frac{k_b}{2}\,\mathbf{C}_b(\mathbf{x})^2 \tag{42}$$

Unlike stretch and shear, the energy is not scaled by triangle area.
The condition derivative reduces to:

$$\frac{\partial\,\mathbf{C}_b}{\partial\,\mathbf{x}_i} = \frac{\partial\,\theta}{\partial\,\mathbf{x}_i} = \cos\theta\,\frac{\partial\,\sin\theta}{\partial\,\mathbf{x}_i} - \sin\theta\,\frac{\partial\,\cos\theta}{\partial\,\mathbf{x}_i} \in \mathbb{R}^3 \tag{43}$$

where:

$$\frac{\partial\,\sin\theta}{\partial\,\mathbf{x}_i} = \left[\frac{1}{\|\mathbf{n}_1\|}\frac{\partial\,\mathbf{n}_1}{\partial\,\mathbf{x}_i}\mathbf{n}_0^\times - \frac{1}{\|\mathbf{n}_0\|}\frac{\partial\,\mathbf{n}_0}{\partial\,\mathbf{x}_i}\mathbf{n}_1^\times\right]\hat{\mathbf{e}} \tag{44}$$

$$\frac{\partial\,\cos\theta}{\partial\,\mathbf{x}_i} = -\frac{1}{\|\mathbf{n}_0\|}\frac{\partial\,\mathbf{n}_0}{\partial\,\mathbf{x}_i}\text{proj}^\perp(\hat{\mathbf{n}}_1, \hat{\mathbf{n}}_0) - \frac{1}{\|\mathbf{n}_1\|}\frac{\partial\,\mathbf{n}_1}{\partial\,\mathbf{x}_i}\text{proj}^\perp(\hat{\mathbf{n}}_0, \hat{\mathbf{n}}_1) \tag{45}$$

The normal Jacobians $\frac{\partial \mathbf{n}_k}{\partial \mathbf{x}_i} \in \mathbb{R}^{3\times 3}$ are skew-symmetric matrices:

$$\begin{aligned}
&\frac{\partial\,\mathbf{n}_0}{\partial\,\mathbf{x}_0} = \mathbf{e}^\times \quad \frac{\partial\,\mathbf{n}_0}{\partial\,\mathbf{x}_1} = \mathbf{x}_{2\to 0}^\times \quad \frac{\partial\,\mathbf{n}_0}{\partial\,\mathbf{x}_2} = \mathbf{x}_{0\to 1}^\times \quad \frac{\partial\,\mathbf{n}_0}{\partial\,\mathbf{x}_3} = \mathbf{0} \\
&\frac{\partial\,\mathbf{n}_1}{\partial\,\mathbf{x}_0} = \mathbf{0} \quad \frac{\partial\,\mathbf{n}_1}{\partial\,\mathbf{x}_1} = -\mathbf{e}^\times \quad \frac{\partial\,\mathbf{n}_1}{\partial\,\mathbf{x}_2} = \mathbf{x}_{3\to 2}^\times \quad \frac{\partial\,\mathbf{n}_1}{\partial\,\mathbf{x}_3} = \mathbf{x}_{1\to 3}^\times
\end{aligned} \tag{46}$$

where $v^\times$ denotes the skew-symmetric cross-product matrix of v:

$$v^\times = \begin{bmatrix} 0 & -v_2 & v_1 \\ v_2 & 0 & -v_0 \\ -v_1 & v_0 & 0 \end{bmatrix}$$

Second derivatives $\dfrac{\partial^2\,\theta}{\partial\,\mathbf{x}_i \partial\,\mathbf{x}_j}$ are not derived as the corresponding term in equation 26 is dropped.

5 Descent Methods [9]

We start from the variational form of implicit Euler time integration for an elastic deformable body:

$$g(\mathbf{x}) = \frac{1}{2h^2}(\mathbf{x}-\tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x}-\tilde{\mathbf{x}}) + \sum_i U_i(\mathbf{x})$$
$$\mathbf{x}^+ = \arg\min_{\mathbf{x}} g(\mathbf{x}) \tag{47}$$

where $\tilde{\mathbf{x}} = \mathbf{x}^- + h\mathbf{v}^- + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}$ is the inertial predictor. A descent method solves this minimization iteratively: starting from an initial guess $\mathbf{x}^{(0)}$, each iterate is updated as

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)}\Delta\mathbf{x}^{(k)}, \qquad \Delta\mathbf{x}^{(k)} \cdot \nabla g(\mathbf{x}^{(k)}) < 0 \tag{48}$$

The descent condition on the right ensures g decreases locally along $\Delta\mathbf{x}^{(k)}$ for sufficiently small $\alpha^{(k)}$. Different descent methods correspond to different choices of $\Delta\mathbf{x}^{(k)}$ and $\alpha^{(k)}$, and optionally a momentum-based acceleration step. The general structure is:

Algorithm 3 General Descent Method Time Step

```
function DescentTimestep(x⁻, v⁻, h)
    Initialize x^(0)
    for k = 0, 1, 2, ... do
        Δx^(k) ← ComputeDescentDirection()
        α^(k) ← ComputeStepLength()
        x̄^(k+1) ← x^(k) + α^(k) Δx^(k)
        x^(k+1) ← Acceleration(x̄^(k+1), x̄^(k), x^(k), x^(k−1))
    return x^(k)
end function
```

5.1 Existing Methods as Descent Methods

Many existing simulation methods can be cast as descent methods within the framework of Algorithm 5, differing only in their choice of descent direction $\Delta\mathbf{x}^{(k)}$.

Gradient Descent. Step in the direction of steepest decrease. Cheap per iteration but slow to converge on ill-conditioned problems.

$$\Delta\mathbf{x}^{(k)} = -\mathbf{J}^{(k)} \qquad \text{with} \quad \mathbf{J}^{(k)} = \nabla g(\mathbf{x}^{(k)}) \tag{49}$$

Newton. Locally approximates g as a quadratic and jumps to its minimum. Quadratic convergence, but each iteration requires solving a $3n \times 3n$ linear system.

$$\Delta\mathbf{x}^{(k)} = -[\mathbf{H}^{(k)}]^{-1}\mathbf{J}^{(k)} \qquad \text{with} \quad \mathbf{H}^{(k)} = \nabla^2 g(\mathbf{x}^{(k)}) \tag{50}$$

Quasi-Newton. Replaces $[\mathbf{H}^{(k)}]^{-1}$ with a cheaper SPD approximation $\mathbf{P}^{(k)}$ that avoids or simplifies the linear solve, trading convergence rate for per-iteration cost.

$$\Delta\mathbf{x}^{(k)} = -\mathbf{P}^{(k)}\mathbf{J}^{(k)} \qquad \text{with} \quad \mathbf{P}^{(k)} \approx [\mathbf{H}^{(k)}]^{-1} \tag{51}$$

Nonlinear Conjugate Gradient. Adds a momentum term to gradient descent, weighted by the Fletcher–Reeves coefficient $z^{(k)}/z^{(k-1)}$, requires dot products at every iteration.

$$\Delta\mathbf{x}^{(k)} = -\mathbf{J}^{(k)} + \frac{z^{(k)}}{z^{(k-1)}}\Delta\mathbf{x}^{(k-1)} \qquad \text{with} \quad z^{(k)} = \mathbf{J}^{(k)} \cdot \mathbf{J}^{(k)} \tag{52}$$

Projective Dynamics [2]. A Quasi-Newton method applied to the augmented PD energy g_{PD}, where $\mathbf{L}$ is a constant Hessian approximation evaluated at the rest configuration. Since $\mathbf{L}$ never changes, it can be Cholesky-factorized once at initialization, reducing each iteration to a back-substitution.

$$\Delta\mathbf{x}^{(k)} = -\mathbf{L}^{-1}\nabla g_{\mathrm{PD}}^{(k)} \qquad \text{with} \quad \mathbf{L} \approx \mathbf{H} \tag{53}$$

Baraff-Witkin [6]. One iteration of Newton's method ($k = 1$) with no further refinement. Fast but inexact: the implicit Euler equation is only approximately satisfied per timestep.

$$\Delta\mathbf{x} = -\mathbf{H}^{-1}\mathbf{J} \qquad \text{with} \quad k = 1 \tag{54}$$

5.2 Jacobi-Preconditioned Gradient Descent with Chebyshev Acceleration

The method proposed by [9] is designed to be GPU-friendly: every operation is element-wise and reduction-free.

Descent direction. A Quasi-Newton step with the Jacobi preconditioner, i.e. the inverse diagonal of the Hessian:

$$\Delta\mathbf{x}^{(k)} = -\mathrm{DIAG}(\mathbf{H}^{(k)})^{-1}\mathbf{J}^{(k)} \tag{55}$$

The preconditioner is trivial to invert (element-wise reciprocal), avoiding the linear solve required by full Newton or Quasi-Newton methods. This approximation is effective for elastic problems because the Hessian is diagonally dominant: $\mathbf{H} = \mathbf{M} + h^2\mathbf{J}^\top\mathbf{K}\mathbf{J}$, where $\mathbf{M}$ is already exactly diagonal and the elastic term aggregates per-element contributions whose diagonal entries dominate the off-diagonals.

Chebyshev acceleration. On top of each descent step, a momentum-based correction blends the current iterate with the one two steps prior:

$$\mathbf{x}^{(k+1)} = \omega^{(k+1)}\big(\bar{\mathbf{x}}^{(k+1)} - \mathbf{x}^{(k-1)}\big) + \mathbf{x}^{(k-1)} \tag{56}$$

where $\bar{\mathbf{x}}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)}\Delta\mathbf{x}^{(k)}$ is the raw descent update and the scalar coefficient $\omega^{(k+1)}$ evolves as:

$$\omega^{(k+1)} = \frac{4}{4 - \rho^2\,\omega^{(k)}}, \qquad \omega^{(1)} = 1 \tag{57}$$

The parameter $\rho \in (0, 1)$ controls how aggressive the acceleration is: values close to 1 speed up convergence the most but can cause the iteration to diverge.

Initialization. $\mathbf{x}^{(0)}$ is predicted from the previous two timesteps assuming constant acceleration: $\mathbf{x}^{(0)} = \mathbf{x}^{-} + h\mathbf{v}^{-} + \eta h(\mathbf{v}^{-} - \mathbf{v}^{--})$ with damping $\eta \approx 0.2$.

6 Primal/Dual Descent Methods [4]

Following the Article [4], we derive a variational formulation of implicit Euler time integration for deformable bodies. Starting from the equations of motion, we cast the time step as a minimization problem (the primal objective), then derive its Lagrangian dual and show how Newton descent on the dual yields an efficient per-constraint update rule that forms the basis of XPBD.

6.1 Equations of Motion

For a deformable body, the discrete equations of motion are defined by the implicit system:

$$\mathbf{M}(\mathbf{v}_{t+1} - \tilde{\mathbf{v}}) - h\,\mathbf{f}(\mathbf{x}_{t+1}, \mathbf{v}_{t+1}) = \mathbf{0} \tag{58}$$

where the *unconstrained velocity* $\tilde{\mathbf{v}}$ represents the state of the system under external forces alone:

$$\tilde{\mathbf{v}} = \mathbf{v}_t + h\mathbf{M}^{-1}\mathbf{f}_{\text{ext}} \tag{59}$$

Starting from Newton's second law $\mathbf{M}\ddot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \dot{\mathbf{x}})$, we introduce $\mathbf{v} = \dot{\mathbf{x}}$ to rewrite the continuous equations of motion as a first-order system:

$$\dot{\mathbf{x}} = \mathbf{v}, \qquad \mathbf{M}\dot{\mathbf{v}} = \mathbf{f}(\mathbf{x}, \mathbf{v})$$

Forces are split into external (non-stiff) and internal (stiff) contributions:

$$\mathbf{f}(\mathbf{x}, \mathbf{v}) = \mathbf{f}_{\text{ext}} + \mathbf{f}_{\text{int}}(\mathbf{x}, \mathbf{v})$$

Applying implicit Euler:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + h\,\mathbf{v}_{t+1}$$
$$\mathbf{M}\,\mathbf{v}_{t+1} = \mathbf{M}\,\mathbf{v}_t + h\,\mathbf{f}_{\text{ext}} + h\,\mathbf{f}_{\text{int}}(\mathbf{x}_{t+1}, \mathbf{v}_{t+1})$$

Collecting all known quantities on the right-hand side and defining the *unconstrained velocity*:

$$\tilde{\mathbf{v}} \equiv \mathbf{v}_t + h\,\mathbf{M}^{-1}\mathbf{f}_{\text{ext}} \qquad \Longleftrightarrow \qquad \mathbf{M}\,\tilde{\mathbf{v}} = \mathbf{M}\,\mathbf{v}_t + h\,\mathbf{f}_{\text{ext}}$$

substituting into the velocity equation and writing $\mathbf{f} \equiv \mathbf{f}_{\text{int}}$ gives:

$$\mathbf{M}\,\mathbf{v}_{t+1} - h\,\mathbf{f}(\mathbf{x}_{t+1}, \mathbf{v}_{t+1}) = \mathbf{M}\,\tilde{\mathbf{v}}$$

$$\mathbf{M}(\mathbf{v}_{t+1} - \tilde{\mathbf{v}}) - h\,\mathbf{f}(\mathbf{x}_{t+1}, \mathbf{v}_{t+1}) = \mathbf{0}$$

6.2 The Primal Objective

The primal optimization problem that encodes the implicit Euler time integration scheme is defined by the objective function $g(\mathbf{v})$:

$$g(\mathbf{v}) = \frac{1}{2}(\mathbf{v} - \tilde{\mathbf{v}})^\top \mathbf{M}(\mathbf{v} - \tilde{\mathbf{v}}) + \sum_i U_i(\mathbf{x}_{t+1}(\mathbf{v})) \tag{60}$$

where the dependence of the potential energy on the velocity is defined through the position update:

$$\mathbf{x}_{t+1}(\mathbf{v}) = \mathbf{x}_t + h\mathbf{v} \tag{61}$$

The velocity for the next time step is then found by minimizing this objective:

$$\mathbf{v}_{t+1} = \arg\min_{\mathbf{v}} g(\mathbf{v}) \tag{62}$$

6.3 Quadratic Potentials

Restricting our model to quadratic potentials, a single elastic energy term takes the form:

$$U_i = \frac{1}{2}\, k_i\, \mathbf{C}_i(\mathbf{x})^2 \in \mathbb{R} \qquad U = \frac{1}{2}\, \mathbf{C}(\mathbf{x})^\top\, \mathbf{K}\, \mathbf{C}(\mathbf{x}) \in \mathbb{R} \tag{63}$$

where $\mathbf{K} = \textsc{diag}([k_1, k_2, \ldots, k_m]) \in \mathbb{R}^{m\times m}$ is the stiffness coefficients matrix and $\mathbf{C}(\mathbf{x}) = [\mathbf{C}_1(\mathbf{x}), \mathbf{C}_2(\mathbf{x}), \ldots, \mathbf{C}_m(\mathbf{x})] \in \mathbb{R}^m$ is the constraint function evaluation column vector. Defining the constraints Jacobian as:

$$\mathbf{J}_i = \frac{\partial\, \mathbf{C}_i}{\partial\, \mathbf{x}} \in \mathbb{R}^{1\times 3n} \qquad \mathbf{J} = \frac{\partial\, \mathbf{C}}{\partial\, \mathbf{x}} \in \mathbb{R}^{m\times 3n} \tag{64}$$

The generalized elastic force arising from a single potential U_i is given by the negative gradient:

$$\mathbf{f}_i = -\left(\frac{\partial\, U_i}{\partial\, \mathbf{x}}\right)^\top = -k_i\, \mathbf{J}_i^\top \mathbf{C}_i(\mathbf{x}) \in \mathbb{R}^{3n} \tag{65}$$

and the full force vector arising from all the potentials U:

$$\mathbf{f} = -\left(\frac{\partial\, U}{\partial\, \mathbf{x}}\right)^\top = -\mathbf{J}^\top \mathbf{K}\, \mathbf{C}(\mathbf{x}) \in \mathbb{R}^{3n} \tag{66}$$

To solve the minimization problem trough Newton's method we need first and second derivative of g with respect to $\mathbf{v}$:

$$\begin{aligned} \frac{\partial\, g}{\partial\, \mathbf{v}} &= \left[\, \mathbf{M}(\mathbf{v} - \tilde{\mathbf{v}}) + h\, \mathbf{J}^\top \mathbf{K}\, \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v})) \,\right] && \in \mathbb{R}^{3n} \\ \frac{\partial^2 g}{\partial\, \mathbf{v}^2} &= \left[\, \mathbf{M} + h^2\, \mathbf{J}^\top \mathbf{K}\, \mathbf{J} \,\right] && \in \mathbb{R}^{3n\times 3n} \end{aligned} \tag{67}$$

The objective g can be split into two terms:

$$g(\mathbf{v}) = g_{\text{inertia}}(\mathbf{v}) + g_{\text{elastic}}(\mathbf{v})$$

The Jacobian and Hessian decompose accordingly:

$$\frac{\partial\, g}{\partial\, \mathbf{v}} = \frac{\partial\, g_{\text{inertia}}}{\partial\, \mathbf{v}} + \frac{\partial\, g_{\text{elastic}}}{\partial\, \mathbf{v}}, \qquad \frac{\partial^2\, g}{\partial\, \mathbf{v}^2} = \frac{\partial^2\, g_{\text{inertia}}}{\partial\, \mathbf{v}^2} + \frac{\partial^2\, g_{\text{elastic}}}{\partial\, \mathbf{v}^2}$$

Inertia term. The inertia term is a quadratic form in $\mathbf{v}$, so its Jacobian and Hessian are simply:

$$\frac{\partial\, g_{\text{inertia}}}{\partial\, \mathbf{v}} = \frac{\partial \left[\frac{1}{2}(\mathbf{v}-\tilde{\mathbf{v}})^\top \mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}})\right]}{\partial\, \mathbf{v}} = \mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}})$$

$$\frac{\partial^2\, g_{\text{inertia}}}{\partial\, \mathbf{v}^2} = \frac{\partial\ \mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}})}{\partial\, \mathbf{v}} = \mathbf{M}$$

Elastic term. Applying the chain rule through $\mathbf{x}_{t+1} = \mathbf{x}_t + h\mathbf{v}$, the first derivative of U with respect to $\mathbf{v}$ is:

$$\frac{\partial\, U(\mathbf{x}_{t+1}(\mathbf{v}))}{\partial\, \mathbf{v}} = \frac{\partial\, \mathbf{x}_{t+1}}{\partial\, \mathbf{v}} \frac{\partial\, U}{\partial\, \mathbf{x}_{t+1}} = h\, \frac{\partial\, U}{\partial\, \mathbf{x}}(\mathbf{x}_{t+1}) = h\, \mathbf{J}^\top \mathbf{K}\, \mathbf{C}(\mathbf{x}_{t+1})$$

So first derivative of the elastic term is symply:

$$\frac{\partial\, g_{\text{elastic}}}{\partial\, \mathbf{v}} = \frac{\partial\, U}{\partial\, \mathbf{v}} = h\, \mathbf{J}^\top \mathbf{K}\, \mathbf{C}(\mathbf{x}_{t+1})$$

Differentiating once more with respect to $\mathbf{v}$, applying the chain rule a second time:

$$\frac{\partial^2\, g_{\text{elastic}}}{\partial\, \mathbf{v}^2} = \frac{\partial^2\, U(\mathbf{x}_{t+1})}{\partial\, \mathbf{v}^2} = h\, \frac{\partial \left[\frac{\partial\, U}{\partial\, \mathbf{x}}(\mathbf{x}_{t+1})\right]}{\partial\, \mathbf{v}} = h^2\, \frac{\partial^2\, U}{\partial\, \mathbf{x}^2}$$

The second derivative of U with respect to positions expands as:

$$\frac{\partial^2\, U}{\partial\, \mathbf{x}^2} = \mathbf{J}^\top \mathbf{K}\, \mathbf{J} + \sum_i k_i\, \frac{\partial^2\, \mathbf{C}_i}{\partial\, \mathbf{x}^2} \mathbf{C}_i(\mathbf{x})$$

The red term is usually dropped. Combining both contributions, the full Hessian of g with respect to $\mathbf{v}$ is:

$$\frac{\partial^2 g}{\partial\, \mathbf{v}^2} = \mathbf{M} + h^2\, \mathbf{J}^\top \mathbf{K}\, \mathbf{J}$$

6.4 Newton Descent

The exact Newton update for $\mathbf{v}$ at iteration k is:

$$\mathbf{v}^{k+1} = \mathbf{v}^k - \alpha\, \mathbf{H}^{-1}\, \frac{\partial\, g}{\partial\, \mathbf{v}} \tag{68}$$

where $\mathbf{H} = \frac{\partial^2 g}{\partial \mathbf{v}^2} = \mathbf{M} + h^2 \mathbf{J}^\top \mathbf{K} \mathbf{J}$ is recomputed at each iteration. This converges quadratically, but requires solving an $3n \times 3n$ linear system at each step.

Quasi-Newton. Since $\mathbf{H}^{-1}$ is expensive to compute, one replaces it with a symmetric positive definite approximation $\mathbf{P} \approx \mathbf{H}^{-1}$:

$$\mathbf{v}^{k+1} = \mathbf{v}^k - \alpha\, \mathbf{P}\, \frac{\partial\, g}{\partial\, \mathbf{v}} \tag{69}$$

Convergence usually degrades from quadratic to linear, but each iteration costs less.

Diagonal preconditioner. The simplest choice is the inverse diagonal of $\mathbf{H}$:

$$\mathbf{P}_{dd} = \frac{1}{\mathbf{M}_{dd} + h^2\, \mathbf{J}_d^\top \mathbf{K}\, \mathbf{J}_d} \tag{70}$$

where d indexes the diagonal entries and $\mathbf{J}_d \in \mathbb{R}^m$ is the d-th column of $\mathbf{J}$, collecting the derivatives of all constraints with respect to DOF d.

Algorithm 4 Primal Descent Time Step

function PRIMALSTEP($\mathbf{x}_t$, $\mathbf{v}_t$, $\mathbf{M}$, $\mathbf{K}$, h)
  $\mathbf{v} \leftarrow \tilde{\mathbf{v}}$ $\in \mathbb{R}^{3n}$
  $\mathbf{x} \leftarrow \mathbf{x}_t + h\mathbf{v}$ $\in \mathbb{R}^{3n}$
  for k iterations **do**
    $\mathbf{f} \leftarrow \mathbf{0}$ $\in \mathbb{R}^{3n}$
    $\mathbf{p} \leftarrow \mathbf{0}$ $\in \mathbb{R}^{3n}$
    for i forces **do**
      $\mathbf{f} \leftarrow \mathbf{f} + h\, \mathbf{J}_i^\top\, k_i\, \mathbf{C}_i(\mathbf{x})$
      $\mathbf{p} \leftarrow \mathbf{p} + \text{DIAG}(h^2\, \mathbf{J}_i^\top\, k_i\, \mathbf{J}_i)$
    $\mathbf{d} \leftarrow \mathbf{M}(\mathbf{v} - \tilde{\mathbf{v}}) + \mathbf{f}$ $\in \mathbb{R}^{3n}$
    $\mathbf{P} \leftarrow (\mathbf{M} + \text{DIAG}(\mathbf{p}))^{-1}$ $\in \mathbb{R}^{3n \times 3n}$
    $\mathbf{v} \leftarrow \mathbf{v} - \mathbf{P}\mathbf{d}$
    $\mathbf{x} \leftarrow \mathbf{x}_t + h\mathbf{v}$
end function

6.5 Collision and Friction

6.5.1 Collision

Collision constraint. For a candidate contact pair, we define the signed-gap constraint:

$$\mathbf{C}_i(\mathbf{x}) = \mathbf{n}^\top \left[\mathbf{a}_i(\mathbf{x}) - \mathbf{b}_i(\mathbf{x})\right] - d \in \mathbb{R} \tag{71}$$

where $\mathbf{a}_i(\mathbf{x}), \mathbf{b}_i(\mathbf{x}) \in \mathbb{R}^3$ are the closest points on the two colliding primitives (point-triangle or edge-edge), $\mathbf{n} \in \mathbb{R}^3$ is the contact normal, and d is the collision thickness. Non-penetration corresponds to $\mathbf{C}_i(\mathbf{x}) \geq 0$.

Penalty potential. We enforce non-penetration through a one-sided polynomial potential that activates only under interpenetration:

$$U_i(\mathbf{x}) = \frac{k_i}{p} \min\big(0,\, \mathbf{C}_i(\mathbf{x})\big)^p \tag{72}$$

where $k_i > 0$ is the contact stiffness and $p \geq 2$ is a fixed exponent. The case $p = 2$ recovers the standard quadratic penalty consistent with the elastic potentials of the previous section; higher p yields smoother derivatives at the activation boundary. As $k_i \to \infty$ the potential approaches hard contact.

Force and Jacobian approximation. The contact force is the negative gradient of U_i:

$$\mathbf{f}_i = -k_i\, \mathbf{J}_i^\top \min\big(0, \mathbf{C}_i(\mathbf{x})\big)^{p-1} \in \mathbb{R}^{3n} \tag{73}$$

with $\mathbf{J}_i = \frac{\partial \mathbf{C}_i}{\partial \mathbf{x}} \in \mathbb{R}^{1\times 3n}$ the constraint Jacobian. Dropping the geometric-stiffness term the force Jacobian needed for the preconditioner reduces to:

$$\frac{\partial\, \mathbf{f}_i}{\partial\, \mathbf{x}} \approx \begin{cases} -k_i\,(p-1)\, \mathbf{J}_i^\top \mathbf{J}_i\, \min\big(0, \mathbf{C}_i(\mathbf{x})\big)^{p-2} & \mathbf{C}_i(\mathbf{x}) < 0 \\ \mathbf{0} & \mathbf{C}_i(\mathbf{x}) \geq 0 \end{cases} \in \mathbb{R}^{3n\times 3n} \tag{74}$$

This approximation preserves positive semi-definiteness, ensuring the descent direction remains valid, and is justified by re-evaluation of $\mathbf{J}_i$ and $\mathbf{C}_i$ at each iteration.

6.5.2 Friction

Let $\mathbf{D} = [\mathbf{t}_1 \mid \mathbf{t}_2] \in \mathbb{R}^{3\times 2}$ be the tangent-plane basis at the contact, with $\mathbf{t}_1, \mathbf{t}_2$ unit vectors orthogonal to the normal $\mathbf{n}$. Given the relative velocity $\bar{\mathbf{v}} \in \mathbb{R}^3$ of the two contacting primitives, the slip velocity is $\bar{\mathbf{v}}_s = \mathbf{D}^\top \bar{\mathbf{v}} \in \mathbb{R}^2$.

Penalty potential. We define a velocity-dependent friction potential with stick and slip regimes separated by the Coulomb threshold $\mu|\mathbf{f}_n|$:

$$U_f(\mathbf{v}) = \begin{cases} \frac{1}{2}\, k_f\, |\bar{\mathbf{v}}_s|^2 & k_f|\bar{\mathbf{v}}_s| < \mu|\mathbf{f}_n| \quad \text{(stick)} \\ \mu|\mathbf{f}_n|\, |\bar{\mathbf{v}}_s| - \gamma & \text{otherwise} \qquad\quad \text{(slip)} \end{cases} \tag{75}$$

where $\mathbf{f}_n$ is the normal contact force (treated as a lagged parameter) and $\gamma = \mu^2|\mathbf{f}_n|^2/(2k_f)$ ensures C^0 continuity at the transition.

Force. The friction force is the negative gradient of U_f:

$$\mathbf{f}_f = -\min\left(k_f,\; \mu\frac{|\mathbf{f}_n|}{|\bar{\mathbf{v}}_s|}\right) \mathbf{D}\mathbf{D}^\top \bar{\mathbf{v}} \in \mathbb{R}^3 \tag{76}$$

Preconditioner. The Hessian factorizes as:

$$\frac{\partial \mathbf{f}_f}{\partial \mathbf{v}} = -\mathbf{D}\,\mathbf{\Lambda}\,\mathbf{D}^\top \tag{77}$$

with:

$$\mathbf{\Lambda} = \begin{cases} k_f\,\mathbf{I}_{2\times 2} \\ \mu\dfrac{|\mathbf{f}_n|}{|\bar{\mathbf{v}}_s|}\left(\mathbf{I}_{2\times 2} - \dfrac{\bar{\mathbf{v}}_s\bar{\mathbf{v}}_s^\top}{|\bar{\mathbf{v}}_s|^2}\right) \end{cases} \quad \approx \quad \begin{cases} k_f & k_f|\bar{\mathbf{v}}_s| < \mu|\mathbf{f}_n| \\ \mu\dfrac{|\mathbf{f}_n|}{|\bar{\mathbf{v}}_s|} & \text{otherwise} \end{cases} \tag{78}$$

6.6 Dual Objective

The Lagrangian for the dual formulation is:

$$\mathcal{L}(\mathbf{v},\boldsymbol{\lambda}) = \frac{1}{2}(\mathbf{v}-\tilde{\mathbf{v}})^\top\mathbf{M}\,(\mathbf{v}-\tilde{\mathbf{v}}) - \boldsymbol{\lambda}^\top\mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v})) - \frac{1}{2}\boldsymbol{\lambda}^\top\mathbf{K}^{-1}\boldsymbol{\lambda} \tag{79}$$

where $\boldsymbol{\lambda} = -\mathbf{K}\,\mathbf{C}(\mathbf{x}_{t+1})$ are the Lagrange multipliers associated with the elastic constraints.

Derivation of the Lagrangian. The primal minimization is rewritten as a constrained problem by introducing the auxiliary variable $\mathbf{c} \in \mathbb{R}^m$:

$$\min_{\mathbf{v},\,\mathbf{c}} \ \frac{1}{2}(\mathbf{v}-\tilde{\mathbf{v}})^\top\mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}}) + \frac{1}{2}\mathbf{c}^\top\mathbf{K}\,\mathbf{c} \qquad \text{s.t.} \qquad \mathbf{c} = \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v}))$$

Applying the method of Lagrange multipliers to the equality constraint yields:

$$\mathcal{L}(\mathbf{v},\mathbf{c},\boldsymbol{\lambda}) = \frac{1}{2}(\mathbf{v}-\tilde{\mathbf{v}})^\top\mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}}) + \frac{1}{2}\mathbf{c}^\top\mathbf{K}\,\mathbf{c} + \boldsymbol{\lambda}^\top(\mathbf{c} - \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v})))$$

The stationarity conditions are:

$$\begin{aligned} \frac{\partial\,\mathcal{L}}{\partial\,\mathbf{v}} = \mathbf{0} \qquad &\longrightarrow \qquad \mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}}) = h\,\mathbf{J}^\top\boldsymbol{\lambda} \\ \frac{\partial\,\mathcal{L}}{\partial\,\mathbf{c}} = \mathbf{0} \qquad &\longrightarrow \qquad \mathbf{c} = -\mathbf{K}^{-1}\boldsymbol{\lambda} \\ \frac{\partial\,\mathcal{L}}{\partial\,\boldsymbol{\lambda}} = \mathbf{0} \qquad &\longrightarrow \qquad \mathbf{c} = \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v})) \end{aligned}$$

Substituting $\mathbf{c} = -\mathbf{K}^{-1}\boldsymbol{\lambda}$ from the second condition eliminates $\mathbf{c}$, reducing

$\mathcal{L}$ to a function of $\mathbf{v}$ and $\boldsymbol{\lambda}$ only:

$$\begin{aligned}\mathcal{L}(\mathbf{v},\boldsymbol{\lambda}) &= \frac{1}{2}(\mathbf{v}-\tilde{\mathbf{v}})^\top\mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}}) + \frac{1}{2}(-\mathbf{K}^{-1}\boldsymbol{\lambda})^\top\mathbf{K}\,(-\mathbf{K}^{-1}\boldsymbol{\lambda}) \\ &\quad + \boldsymbol{\lambda}^\top(-\mathbf{K}^{-1}\boldsymbol{\lambda} - \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v}))) \\ &= \frac{1}{2}(\mathbf{v}-\tilde{\mathbf{v}})^\top\mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}}) + \frac{1}{2}\boldsymbol{\lambda}^\top\mathbf{K}^{-1}\boldsymbol{\lambda} \\ &\quad - \boldsymbol{\lambda}^\top\mathbf{K}^{-1}\boldsymbol{\lambda} - \boldsymbol{\lambda}^\top\mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v})) \\ &= \frac{1}{2}(\mathbf{v}-\tilde{\mathbf{v}})^\top\mathbf{M}(\mathbf{v}-\tilde{\mathbf{v}}) - \boldsymbol{\lambda}^\top\mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v})) - \frac{1}{2}\boldsymbol{\lambda}^\top\mathbf{K}^{-1}\boldsymbol{\lambda}\end{aligned}$$

which is equation (79).

The corresponding Lagrange dual function for 79 is:

$$\mathbf{h}(\lambda) = \min_{\mathbf{v}} \mathcal{L}(\mathbf{v},\lambda) = \mathcal{L}(\mathbf{v}^*,\lambda) \tag{80}$$

We should now try to find an analytical formula to calculate the optimal $\mathbf{v}^*$ in terms of λ. This can not be done for this case so we use this approximation:

$$\mathbf{v}^* = \tilde{\mathbf{v}} + h\,\mathbf{M}^{-1}\mathbf{J}^\top\lambda \tag{81}$$

We derive this from the stationary condition $\dfrac{\partial\mathcal{L}}{\partial\mathbf{v}} = 0$:

$$\begin{aligned}\mathbf{M}(\mathbf{v}^*-\tilde{\mathbf{v}}) &= h\,\mathbf{J}^\top\lambda \\ \mathbf{v}^*-\tilde{\mathbf{v}} &= h\,\mathbf{M}^{-1}\mathbf{J}^\top\lambda \\ \mathbf{v}^* &= \tilde{\mathbf{v}} + h\,\mathbf{M}^{-1}\mathbf{J}^\top\lambda\end{aligned}$$

which is equation 81. This is just a linear approximation because the Jacobian $\mathbf{J}$ does depends on the velocity $\mathbf{v}$.

So substituting $\mathbf{v}^*$ back in the Lagrangian 79 we obtain:

$$\mathbf{h}(\lambda) \approx \mathcal{L}(\mathbf{v}^*,\lambda) = \frac{h^2}{2}\lambda^\top\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top\lambda - \lambda^\top\mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v}^*)) - \frac{1}{2}\lambda^\top\mathbf{K}^{-1}\lambda \tag{82}$$

and the dual maximization problem is:

$$\lambda_{t+1} = \arg\max_{\lambda} \mathbf{h}(\lambda) \tag{83}$$

So now we take the first and second derivative of $\mathbf{h}$ with respect to λ:

$$\begin{aligned}\frac{\partial \mathbf{h}}{\partial \lambda} &= -\Big[\, \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v}^*)) + \mathbf{K}^{-1}\lambda \,\Big] \\ \frac{\partial^2 \mathbf{h}}{\partial \lambda^2} &= -\Big[\, h^2\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top + \mathbf{K}^{-1} \,\Big]\end{aligned} \tag{84}$$

We split $\mathbf{h}$ into three terms:

$$\mathbf{h}(\lambda) = \underbrace{\left[\frac{h^2}{2}\lambda^\top \mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top\lambda\right]}_{\mathbf{h}_1} - \underbrace{\left[\lambda^\top \mathbf{C}(\mathbf{x}(\mathbf{v}^*(\lambda)))\right]}_{\mathbf{h}_2} - \underbrace{\left[\frac{1}{2}\lambda^\top \mathbf{K}^{-1}\lambda\right]}_{\mathbf{h}_3}$$

and differentiate each term with respect to λ. The first and third terms are standard quadratic forms. The second requires the chain rule through $\lambda \to \mathbf{v}^* \to \mathbf{x}_{t+1} \to \mathbf{C}$:

$$\begin{aligned}\frac{\partial \mathbf{h}_1}{\partial \lambda} &= h^2\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top\lambda \\ \frac{\partial \mathbf{h}_2}{\partial \lambda} &= \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v}^*(\lambda))) + \frac{\partial \mathbf{C}}{\partial \mathbf{x}_{t+1}}\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{v}^*}\frac{\partial \mathbf{v}^*}{\partial \lambda}\lambda \\ &= \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v}^*(\lambda))) + h^2\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top\lambda \\ \frac{\partial \mathbf{h}_3}{\partial \lambda} &= \mathbf{K}^{-1}\lambda\end{aligned}$$

Assembling the full gradient, the explicit $h^2\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top\lambda$ terms from $\mathbf{h}_1$ and $\mathbf{h}_2$ cancel:

$$\begin{aligned}\frac{\partial \mathbf{h}}{\partial \lambda} &= h^2\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top\lambda - \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v}^*(\lambda))) - h^2\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top\lambda - \mathbf{K}^{-1}\lambda \\ &= -\Big[\, \mathbf{C}(\mathbf{x}_{t+1}(\mathbf{v}^*)) + \mathbf{K}^{-1}\lambda \,\Big]\end{aligned}$$

Differentiating once more, only $\mathbf{h}_2$ and $\mathbf{h}_3$ contribute since $\mathbf{h}_1$ cancelled:

$$\frac{\partial^2 \mathbf{h}}{\partial \lambda^2} = -\frac{\partial \mathbf{C}}{\partial \mathbf{x}_{t+1}}\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{v}^*}\frac{\partial \mathbf{v}^*}{\partial \lambda} - \mathbf{K}^{-1} = -\Big[\, h^2\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top + \mathbf{K}^{-1} \,\Big]$$

The Hessian is negative definite, since $\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top$ is positive semi-definite (as $\mathbf{M}^{-1}$ is SPD) and $\mathbf{K}^{-1}$ is strictly positive definite. Their sum is therefore SPD, confirming $\mathbf{h}$ is strictly concave and the maximum is unique.

7 XPBD [1]

XPBD reformulates implicit Euler time integration as a position-based optimization problem. Following the primal/dual descent framework of Section 6, we derive the constraint multiplier updates and position corrections that define the algorithm, and show how they arise naturally from a Newton descent on the dual objective.

First, we rewrite the objective function in a position-based form:

$$g(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + h^2\, U(\mathbf{x}) \tag{85}$$

where $\tilde{\mathbf{x}}$ is the *unconstrained position*, calculated as:

$$\tilde{\mathbf{x}} = \mathbf{x}_t + h\,\mathbf{v}_t = 2\mathbf{x}_t - \mathbf{x}_{t-1} \tag{86}$$

This comes from using this equation of motion in position-based form:

$$\begin{aligned}
\mathbf{M}\left(\frac{\mathbf{v}_{t+1} - \mathbf{v}_t}{h}\right) &= \mathbf{f}(\mathbf{x}_{t+1}) \\
\mathbf{M}\left(\frac{\mathbf{v}_{t+1} - \mathbf{v}_t}{h}\right) &= -\frac{\partial U}{\partial \mathbf{x}_{t+1}} \\
\mathbf{M}\left(\frac{\mathbf{x}_{t+1} - \mathbf{x}_t}{h^2} - \frac{\mathbf{x}_t - \mathbf{x}_{t-1}}{h^2}\right) &= -\frac{\partial U}{\partial \mathbf{x}_{t+1}} \\
\frac{1}{h^2}\mathbf{M}\left(\mathbf{x}_{t+1} - 2\mathbf{x}_t + \mathbf{x}_{t-1}\right) &= -\frac{\partial U}{\partial \mathbf{x}_{t+1}} \\
\mathbf{M}\left(\mathbf{x}_{t+1} - \tilde{\mathbf{x}}\right) &= -h^2 \frac{\partial U}{\partial \mathbf{x}_{t+1}}
\end{aligned}$$

Restricting the system to quadratic energy potentials, we have:

$$g_{\text{XPBD}}(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + \frac{h^2}{2}\mathbf{C}(\mathbf{x})^\top \mathbf{K}\mathbf{C}(\mathbf{x}) \tag{87}$$

Following the same derivation as in 6.6, the Lagrangian of the dual position-based problem is:

$$\mathcal{L}_{\text{XPBD}}(\mathbf{x}, \lambda) = \frac{1}{2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) - \lambda^\top \mathbf{C}(\mathbf{x}) - \frac{1}{2h^2}\lambda^\top \mathbf{K}^{-1}\lambda \tag{88}$$

where, now, $\lambda = -h^2\mathbf{K}\mathbf{C}(\mathbf{x})$.

From first optimality condition $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = 0$ we obtain an approximated optimal value for $\mathbf{x}^*$ given λ:

$$\mathbf{x}^* = \tilde{\mathbf{x}} + \mathbf{M}^{-1}\mathbf{J}^\top \lambda \tag{89}$$

that we substitute back into the Lagrangian obtaining the approximated dual objective function:

$$\mathbf{h}_{\text{XPBD}}(\lambda) \approx \mathcal{L}(\mathbf{x}^*, \lambda) = \frac{1}{2}\lambda^\top \mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top \lambda - \lambda^\top \mathbf{C}(\mathbf{x}^*) - \frac{1}{2h^2}\lambda^\top \mathbf{K}^{-1}\lambda \tag{90}$$

So now we take the first and second derivative of $\mathbf{h}$ with respect to λ:

$$\begin{aligned} \frac{\partial\, \mathbf{h}_{\text{XPBD}}}{\partial\, \lambda} &= -\left[\, \mathbf{C}(\mathbf{x}^*) + \frac{1}{h^2}\mathbf{K}^{-1}\lambda \,\right] = -\left[\, \mathbf{C}(\mathbf{x}^*) + \tilde{\mathbf{A}}\, \lambda \,\right] \\ \frac{\partial^2\, \mathbf{h}_{\text{XPBD}}}{\partial\, \lambda^2} &= -\left[\, \mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top + \frac{1}{h^2}\mathbf{K}^{-1} \,\right] = -\left[\, \mathbf{J}\mathbf{M}^{-1}\mathbf{J}^\top + \tilde{\mathbf{A}} \,\right] \end{aligned} \tag{91}$$

where $\mathbf{A} = \mathbf{K}^{-1}$ is the compliance matrix (inverse of stiffness) and $\tilde{\mathbf{A}} = \frac{1}{h^2}\mathbf{A}$ is the compliance divided by the square of the timestep (the term called $\tilde{\boldsymbol{\alpha}}$ in the article).
Following the Newton descent of Section 6.4 with the diagonal preconditioner of Equation 70, the dual Newton step per constraint i gives the multiplier update:

$$\Delta\lambda_i = \frac{-\mathbf{C}_i(\mathbf{x}^*) - \tilde{\mathbf{A}}_{ii}\, \lambda_i}{\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\mathbf{A}}_{ii}} \tag{92}$$

where $i = 1, \ldots, m$ indexes the constraints, $\mathbf{J}_i \in \mathbb{R}^{1\times 3n}$ is the i-th row of $\mathbf{J}$, and $\tilde{\mathbf{A}}_{ii} = \tilde{\boldsymbol{\alpha}}_i = k_i^{-1}/h^2$ is the i-th diagonal entry of $\tilde{\mathbf{A}}$. This is exactly Eq. (18) of [1]. From this and Equation 89 it follows the position correction formula:

$$\Delta\mathbf{x}_i = \mathbf{M}^{-1}\mathbf{J}^\top \Delta\lambda_i \tag{93}$$

Algorithm 5 XPBD Time Step

function XPBDStep($\mathbf{x}_t$, $\mathbf{v}_t$, $\mathbf{M}$, $\tilde{\mathbf{A}}$, h)
  $\mathbf{x} \leftarrow \tilde{\mathbf{x}}$ $\in \mathbb{R}^{3n}$
  $\boldsymbol{\lambda} \leftarrow \mathbf{0}$ $\in \mathbb{R}^{m}$
  for k iterations **do**
    $\Delta\boldsymbol{\lambda} \leftarrow \mathbf{0}$ $\in \mathbb{R}^{m}$
    for i constraints **do**
      $\Delta\lambda_i \leftarrow \dfrac{-\mathbf{C}_i(\mathbf{x}) - \tilde{\boldsymbol{\alpha}}_i\, \lambda_i}{\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i}$
    $\Delta\mathbf{x} \leftarrow \mathbf{M}^{-1}\mathbf{J}^\top \Delta\boldsymbol{\lambda}$
    $\lambda \leftarrow \lambda + \Delta\boldsymbol{\lambda}$
    $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}$
  $\mathbf{x}_{t+1} \leftarrow \mathbf{x}$
  $\mathbf{v}_{t+1} \leftarrow (\mathbf{x}_{t+1} - \mathbf{x}_t)\,/\,h$
end function

7.1 Constraints

A constraint $\mathbb{C}_i$ is defined by the following elements:

- $n_c \in \mathbb{N}$: number of particles it acts on;
- $[j_1, j_2, \ldots, j_{n_c}] \in \mathbb{N}^{n_c}$: indices of those particles;
- $C_i : \mathbb{R}^{3n_c} \to \mathbb{R}$: scalar constraint function;
- $\mathbf{J}_i : \mathbb{R}^{3n_c} \to \mathbb{R}^{1\times 3n_c}$: Jacobian of C_i;
- $\mathbf{H}_i : \mathbb{R}^{3n_c} \to \mathbb{R}^{3n_c\times 3n_c}$: Hessian of C_i (only used when differentiating backward the algorithm).

In matrix form:

$$\mathbf{J}_i = \begin{bmatrix} \dfrac{\partial C_i}{\partial \mathbf{x}_{j_1}} & \dfrac{\partial C_i}{\partial \mathbf{x}_{j_2}} & \cdots & \dfrac{\partial C_i}{\partial \mathbf{x}_{j_{n_c}}} \end{bmatrix}$$

$$\mathbf{H}_i = \begin{bmatrix} \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_1}^2} & \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_1}\partial \mathbf{x}_{j_2}} & \cdots & \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_1}\partial \mathbf{x}_{j_{n_c}}} \\ \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_2}\partial \mathbf{x}_{j_1}} & \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_2}^2} & \cdots & \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_2}\partial \mathbf{x}_{j_{n_c}}} \\ \vdots & \vdots & \ddots & \vdots \\ \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_{n_c}}\partial \mathbf{x}_{j_1}} & \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_{n_c}}\partial \mathbf{x}_{j_2}} & \cdots & \dfrac{\partial^2 C_i}{\partial \mathbf{x}_{j_{n_c}}^2} \end{bmatrix}$$

7.1.1 Distance Constraint

A distance constraint enforces a rest length l_0 between two particles $\mathbf{x}_{j_1}, \mathbf{x}_{j_2} \in \mathbb{R}^3$, so $n_c = 2$. Defining $\mathbf{d} = \mathbf{x}_{j_2} - \mathbf{x}_{j_1} \in \mathbb{R}^3$ and $l = \|\mathbf{d}\|$, the constraint elements are:

$$C_i(\mathbf{x}_{j_1}, \mathbf{x}_{j_2}) = l - l_0 = \|\mathbf{x}_{j_2} - \mathbf{x}_{j_1}\| - l_0$$

$$\mathbf{J}_i = \begin{bmatrix} \dfrac{\partial C_i}{\partial \mathbf{x}_{j_1}} & \dfrac{\partial C_i}{\partial \mathbf{x}_{j_2}} \end{bmatrix} = \begin{bmatrix} -\hat{\mathbf{d}}^\top & \hat{\mathbf{d}}^\top \end{bmatrix} \in \mathbb{R}^{1\times 6}$$

where $\hat{\mathbf{d}} = \mathbf{d}/l$ is the unit vector along the edge. The Hessian blocks are:

$$\frac{\partial^2 C_i}{\partial \mathbf{x}_{j_1}^2} = \frac{\partial^2 C_i}{\partial \mathbf{x}_{j_2}^2} = \frac{1}{l}\left(\mathbf{I} - \hat{\mathbf{d}}\hat{\mathbf{d}}^\top\right) \in \mathbb{R}^{3\times 3}$$

$$\frac{\partial^2 C_i}{\partial \mathbf{x}_{j_1}\partial \mathbf{x}_{j_2}} = \frac{\partial^2 C_i}{\partial \mathbf{x}_{j_2}\partial \mathbf{x}_{j_1}} = -\frac{1}{l}\left(\mathbf{I} - \hat{\mathbf{d}}\hat{\mathbf{d}}^\top\right) \in \mathbb{R}^{3\times 3}$$

so the full Hessian is:

$$\mathbf{H}_i = \frac{1}{l}\left(\mathbf{I} - \hat{\mathbf{d}}\hat{\mathbf{d}}^\top\right)\begin{bmatrix} \mathbf{I} & -\mathbf{I} \\ -\mathbf{I} & \mathbf{I} \end{bmatrix} \in \mathbb{R}^{6\times 6}$$

$\mathbf{H}_i$ is symmetric and positive semi-definite.

7.2 Differentiable XPBD through Explicit Adjoint Method

7.2.1 1-iteration Case

First we look at the simple 1-iteration version of the XPBD algorithm:

Algorithm 6 XPBD Time Step with 1 Iteration

function $\mathbf{f}_{\text{XPBD}}(\mathbf{q}_t = [\,\mathbf{x}_t,\, \mathbf{v}_t\,],\, \tilde{\mathbf{A}} = [\,\tilde{\boldsymbol{\alpha}}_1,\, \tilde{\boldsymbol{\alpha}}_2, \ldots,\, \tilde{\boldsymbol{\alpha}}_m\,])$
  $\mathbf{x} \leftarrow \tilde{\mathbf{x}}$
  for i constraints **do**
    $\Delta\lambda_i \leftarrow \dfrac{-\mathbf{C}_i(\mathbf{x})}{\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i}$
    $\Delta\mathbf{x}_i \leftarrow \mathbf{M}^{-1}\mathbf{J}_i\,\Delta\lambda_i$
  $\Delta\mathbf{x} \leftarrow \Delta\mathbf{x}_1 + \Delta\mathbf{x}_2 + \cdots + \Delta\mathbf{x}_m$
  $\mathbf{x}_{t+1} \leftarrow \tilde{\mathbf{x}} + \Delta\mathbf{x}$
  $\mathbf{v}_{t+1} \leftarrow (\mathbf{x}_{t+1} - \mathbf{x}_t)\,/\,h$
end function

Abstractly, the explicit time-stepping function is:

$$\mathbf{f}_{\text{XPBD}}\left(\begin{bmatrix}\mathbf{x}_t\\ \mathbf{v}_t\end{bmatrix}, \tilde{\mathbf{A}}\right) = \begin{bmatrix}\mathbf{x}_{t+1}\\ \mathbf{v}_{t+1}\end{bmatrix}$$

For the calculation of adjoint states, we need $\dfrac{\partial\,\mathbf{f}_{\text{XPBD}}}{\partial\,\mathbf{q}_t}$:

$$\frac{\partial\,\mathbf{f}_{\text{XPBD}}}{\partial\,\mathbf{q}_t} = \begin{bmatrix}\dfrac{\partial\,\mathbf{x}_{t+1}}{\partial\,\mathbf{x}_t} & \dfrac{\partial\,\mathbf{x}_{t+1}}{\partial\,\mathbf{v}_t}\\ \dfrac{\partial\,\mathbf{v}_{t+1}}{\partial\,\mathbf{x}_t} & \dfrac{\partial\,\mathbf{v}_{t+1}}{\partial\,\mathbf{v}_t}\end{bmatrix}$$

We know:

$$\begin{aligned}\frac{\partial\,\tilde{\mathbf{x}}}{\partial\,\mathbf{x}_t} &= \frac{\partial\,\mathbf{x}_t + h(\mathbf{v}_t + h\mathbf{M}^{-1}\mathbf{f}_{\text{ext}})}{\partial\,\mathbf{x}_t} = \mathbf{I}\\ \frac{\partial\,\tilde{\mathbf{x}}}{\partial\,\mathbf{v}_t} &= h\end{aligned} \tag{94}$$

Then we can start the derivation:

$$\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} = \frac{\partial \tilde{\mathbf{x}}}{\partial \mathbf{x}_t} + \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t} = \mathbf{I} + \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t}$$

$$\frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t} = \frac{\partial \Delta\mathbf{x}_1}{\partial \mathbf{x}_t} + \frac{\partial \Delta\mathbf{x}_2}{\partial \mathbf{x}_t} + \cdots \frac{\partial \Delta\mathbf{x}_m}{\partial \mathbf{x}_t}$$

$$\frac{\partial \Delta\mathbf{x}_i}{\partial \mathbf{x}_t} = \frac{\partial \mathbf{M}^{-1}\mathbf{J}_i\Delta\lambda_i}{\partial \mathbf{x}_t} = \mathbf{M}^{-1}\left(\frac{\partial \mathbf{J}_i}{\partial \mathbf{x}_t}\Delta\lambda_i + \mathbf{J}_i\frac{\partial \Delta\lambda_i}{\partial \mathbf{x}_t}\right)$$

knowing $\boxed{\Delta\lambda_i = -\mathbf{C}_i(\tilde{\mathbf{x}})(\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i)^{-1} = \mathbf{b}_i\,\mathbf{D}_i^{-1}}$

$$\frac{\partial \Delta\lambda_i}{\partial \mathbf{x}_t} = \frac{\partial \mathbf{b}_i}{\partial \mathbf{x}_t}\mathbf{D}_i^{-1} + \mathbf{b}_i\frac{\partial \mathbf{D}_i^{-1}}{\partial \mathbf{x}_t}$$

$$\frac{\partial \mathbf{b}_i}{\partial \mathbf{x}_t} = -\mathbf{J}_i$$

$$\frac{\partial \mathbf{D}_i^{-1}}{\partial \mathbf{x}_t} = -\mathbf{D}_i^{-2}\frac{\partial \mathbf{D}_i}{\partial \mathbf{x}_t} = -2\mathbf{D}_i^{-2}\mathbf{H}_i\mathbf{M}^{-1}\mathbf{J}_i$$

where $\mathbf{H}_i = \dfrac{\partial \mathbf{J}_i}{\partial \mathbf{x}_t}$ is the Hessian of the constraint function $\mathbf{C}_i$. So at the end we have:

$$\frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t} = \mathbf{M}^{-1}\left(\sum_i \left[\mathbf{H}_i\Delta\lambda_i - \frac{1}{\mathbf{D}_i}\mathbf{J}_i^\top\mathbf{J}_i + \frac{2\mathbf{C}_i}{\mathbf{D}_i^2}\mathbf{J}_i^\top\mathbf{J}_i\mathbf{M}^{-1}\mathbf{H}_i\right]\right) \tag{95}$$

and:

$$\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} = \mathbf{I} + \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t}$$

$$\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{v}_t} = \frac{\partial \tilde{\mathbf{x}}}{\partial \mathbf{v}_t} + \frac{\partial \Delta\mathbf{x}(\tilde{\mathbf{x}})}{\partial \mathbf{v}_t} = h + h\frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t} = h\left(\mathbf{I} + \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t}\right) = h\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t}$$

$$\frac{\partial \mathbf{v}_{t+1}}{\partial \mathbf{x}_t} = \frac{1}{h}\left(\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} - \mathbf{I}\right)$$

$$\frac{\partial \mathbf{v}_{t+1}}{\partial \mathbf{v}_t} = \frac{1}{h}\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{v}_t} = \frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t}$$

And finally:

$$\begin{aligned}\frac{\partial \mathbf{f}_{\text{XPBD}_t}}{\partial \mathbf{q}_t} &= \begin{bmatrix} \dfrac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} & h\dfrac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} \\ \dfrac{1}{h}\left(\dfrac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} - \mathbf{I}\right) & \dfrac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I} + \dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t} & h\left(\mathbf{I} + \dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right) \\ \dfrac{1}{h}\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t} & \mathbf{I} + \dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t} \end{bmatrix}\end{aligned} \tag{96}$$

Then, using the adjoint state formula:

$$\begin{aligned}\begin{bmatrix} \hat{\mathbf{x}}_t \\ \hat{\mathbf{v}}_t \end{bmatrix} &= \begin{bmatrix} \mathbf{I} + \left(\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right)^\top & \dfrac{1}{h}\left(\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right)^\top \\ h\mathbf{I} + h\left(\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right)^\top & \mathbf{I} + \left(\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right)^\top \end{bmatrix} \begin{bmatrix} \hat{\mathbf{x}}_{t+1} \\ \hat{\mathbf{v}}_{t+1} \end{bmatrix} + \begin{bmatrix} \dfrac{\partial \phi}{\partial \mathbf{x}_t} \\ \dfrac{\partial \phi}{\partial \mathbf{v}_t} \end{bmatrix} \\ &= \begin{bmatrix} \hat{\mathbf{x}}_{t+1} + \left(\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right)^\top \hat{\mathbf{x}}_{t+1} + \dfrac{1}{h}\left(\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right)^\top \hat{\mathbf{v}}_{t+1} + \dfrac{\partial \phi}{\partial \mathbf{x}_t} \\ h\,\hat{\mathbf{x}}_{t+1} + h\left(\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right)^\top \hat{\mathbf{x}}_{t+1} + \hat{\mathbf{v}}_{t+1} + \left(\dfrac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}\right)^\top \hat{\mathbf{v}}_{t+1} + \dfrac{\partial \phi}{\partial \mathbf{v}_t} \end{bmatrix}\end{aligned} \tag{97}$$

$$\begin{bmatrix} \hat{\mathbf{x}}_t \\ \hat{\mathbf{v}}_t \end{bmatrix} = \begin{bmatrix} \dfrac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} & h\dfrac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} \\ \dfrac{1}{h}\left(\dfrac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} - \mathbf{I}\right) & \dfrac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} \end{bmatrix}^\top \begin{bmatrix} \hat{\mathbf{x}}_{t+1} \\ \hat{\mathbf{v}}_{t+1} \end{bmatrix} + \begin{bmatrix} \dfrac{\partial \phi}{\partial \mathbf{x}_t} \\ \dfrac{\partial \phi}{\partial \mathbf{v}_t} \end{bmatrix} \tag{98}$$

To be able to differentiate over the compliance matrix/vector $\tilde{\mathbf{A}}$, through the adjoint method, we need to be able to calculate the partial derivative $\frac{\partial \mathbf{f}_{\text{XPBD}}(\mathbf{q}_t, \tilde{\mathbf{A}})}{\partial \tilde{\mathbf{A}}}$, where $\tilde{\mathbf{A}} \in \mathbb{R}^m$ are the control variables $\mathbf{u}$ and $\mathbf{q}_t = [\,\mathbf{x}_t, \mathbf{v}_t\,] \in \mathbb{R}^{(3n+3n)}$ is the simulation state progressing over time.
We find that for each constraint i we can calculate:

$$\frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_i} = \mathbf{M}^{-1}\mathbf{J}_i^\top \left[\mathbf{C}_i(\mathbf{x})(\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i)^{-2}\right] \tag{99}$$

$$\frac{\partial \mathbf{v}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_i} = \frac{1}{h}\frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_i} \tag{100}$$

and finally:

$$\begin{aligned}\frac{\partial \mathbf{f}_{\text{XPBD}}}{\partial \tilde{\mathbf{A}}} &= \begin{bmatrix} \frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_1} & \cdots & \frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_m} \\ \frac{\partial \mathbf{v}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_1} & \cdots & \frac{\partial \mathbf{v}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_m} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I} \\ \frac{1}{h}\mathbf{I} \end{bmatrix} \begin{bmatrix} \frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\alpha}_1} & \cdots & \frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\alpha}_m} \end{bmatrix} \qquad \in \mathbb{R}^{(3n+3n)\times m}\end{aligned} \tag{101}$$

$$\frac{\partial \mathbf{f}_t}{\partial \tilde{\mathbf{A}}} = \begin{bmatrix} \frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\mathbf{A}}} & \frac{\partial \mathbf{v}_{t+1}}{\partial \tilde{\mathbf{A}}} \end{bmatrix} \qquad \in \mathbb{R}^{(3n+3n)\times m}$$

$$\frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\mathbf{A}}} = \begin{bmatrix} \frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_1} & \frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_2} & \cdots & \frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_m} \end{bmatrix} \qquad \in \mathbb{R}^{3n\times m}$$

$$\frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}_i} = \frac{\partial \Delta \mathbf{x}}{\partial \tilde{\boldsymbol{\alpha}}_i} = \frac{\partial \Delta \mathbf{x}_i}{\partial \tilde{\boldsymbol{\alpha}}_i} \qquad \in \mathbb{R}^{3n}$$

$$\frac{\partial \Delta \mathbf{x}_i}{\partial \tilde{\boldsymbol{\alpha}}_i} = \mathbf{M}^{-1}\mathbf{J}_i^\top \frac{\partial \Delta \lambda_i}{\partial \tilde{\boldsymbol{\alpha}}_i} \qquad \in \mathbb{R}^{3n}$$

$$\frac{\partial \Delta \lambda_i}{\partial \tilde{\boldsymbol{\alpha}}_i} = \mathbf{C}_i(\mathbf{x})(\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i)^{-2} \qquad \in \mathbb{R}$$

because:

$$\Delta\lambda_i(\tilde{\boldsymbol{\alpha}}_i) = -\mathbf{C}_i(\mathbf{x})(\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i)^{-1}$$

Algorithms 7 and 8 present the explicit differentiable XPBD for the 1-iteration case. Algorithm 7 covers the general constraint formulation, while Algorithm 8 specializes it to distance constraints with closed-form expressions. Lines in blue are required only when differentiating with respect to the compliance matrix/vector $\tilde{\mathbf{A}}$. In Algorithm 8, $w_1, w_2 \in \mathbb{R}$ are the scalar inverse masses of the two particles.

Algorithm 7 Differentiated XPBD Time Step with 1 Iteration

function $\mathbf{f}_{\text{XPBD}}(\mathbf{x}_t,\ \mathbf{v}_t,\ \mathbb{C},\ \tilde{\mathbf{A}},\ \mathbf{M},\ h)$
  $\mathbf{x} \leftarrow \mathbf{x}_t + h(\mathbf{v}_t + h\mathbf{M}^{-1}\mathbf{f}_{\text{ext}})$
  $\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t} \leftarrow \mathbf{0}$
  $\frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\mathbf{A}}} \leftarrow \mathbf{0}$
  for $\mathbb{C}_i$ in $\mathbb{C}$ **do**
    COMPUTE $\mathbf{C}_i(\mathbf{x}),\ \mathbf{J}_i(\mathbf{x}),\ \mathbf{H}_i(\mathbf{x})$
    $\mathbf{D}_i \leftarrow \mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i$
    $\Delta\lambda_i \leftarrow \frac{-\mathbf{C_i}}{\mathbf{D}_i}$
    $\Delta\mathbf{x}_i \leftarrow \mathbf{M}^{-1}\mathbf{J}_i\Delta\lambda_i$
    $\frac{\partial \Delta \mathbf{x}_i}{\partial \mathbf{x}_t} \leftarrow \mathbf{H}_i\Delta\lambda_i - \frac{\mathbf{J}_i^\top\mathbf{J}_i}{\mathbf{D}_i} + \frac{2\mathbf{C}_i}{\mathbf{D}_i^2}\mathbf{J}_i^\top\mathbf{J}_i\mathbf{M}^{-1}\mathbf{H}_i$
    $\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t} \leftarrow \frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t} + \frac{\partial \Delta \mathbf{x}_i}{\partial \mathbf{x}_t}$
    $\left[\frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\mathbf{A}}}\right]_{:,i} \leftarrow \mathbf{M}^{-1}\mathbf{J}_i^\top\frac{\mathbf{C}_i}{\mathbf{D}_i^2}$
  $\mathbf{x}_{t+1} \leftarrow \mathbf{x} + \sum_{\mathbb{C}_i}\Delta\mathbf{x}_i$
  $\mathbf{v}_{t+1} \leftarrow (\mathbf{x}_{t+1} - \mathbf{x}_t)\,/\,h$
  $\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} \leftarrow \mathbf{I} + \mathbf{M}^{-1}\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t}$
end function

Algorithm 8 Differentiated Distance Constraint Solve for 1-iteration XPBD Time Step

function DISTANCECONSTRAINT$(\mathbf{x}_1, \mathbf{x}_2, w_1, w_2, \tilde{\boldsymbol{\alpha}})$
  $\mathbf{C} \leftarrow \|\mathbf{x}_2 - \mathbf{x}_1\| - l_0$
  $\mathbf{n} \leftarrow \dfrac{\mathbf{x}_2 - \mathbf{x}_1}{\|\mathbf{x}_2 - \mathbf{x}_1\|}$
  $\mathbf{H} \leftarrow \dfrac{1}{\|\mathbf{x}_2 - \mathbf{x}_1\|}(\mathbf{I} - \mathbf{n}\mathbf{n}^\top)\begin{bmatrix}\mathbf{I} & -\mathbf{I}\\ -\mathbf{I} & \mathbf{I}\end{bmatrix}$
  $\mathbf{D} \leftarrow w_1 + w_2 + \tilde{\boldsymbol{\alpha}}$
  $\mathbf{JJ} \leftarrow \begin{bmatrix}\mathbf{I}\\ -\mathbf{I}\end{bmatrix}\mathbf{n}\mathbf{n}^\top\begin{bmatrix}\mathbf{I} & -\mathbf{I}\end{bmatrix}$
  $\Delta\lambda \leftarrow \dfrac{-\mathbf{C}}{\mathbf{D}}$
  $\Delta\mathbf{x} \leftarrow [-w_1\mathbf{n}\ \ w_2\mathbf{n}]\Delta\lambda$
  $\mathbf{W} = \text{LUMPED}_{6\times 6}(w_1, w_2)$
  $\frac{\partial \Delta \mathbf{x}}{\partial \mathbf{x}_t} \leftarrow \mathbf{H}\Delta\lambda - \frac{\mathbf{JJ}}{\mathbf{D}} + \frac{2\mathbf{C}}{\mathbf{D}^2}\mathbf{JJ}\ \mathbf{W}\ \mathbf{H}$
  $\frac{\partial \mathbf{x}_{t+1}}{\partial \tilde{\boldsymbol{\alpha}}} \leftarrow [-w_1\mathbf{n}\ \ w_2\mathbf{n}]\frac{\mathbf{C}}{\mathbf{D}^2}$
end function

7.2.2 k-iterations Case

Now let's look at the general k-iterations case.

Algorithm 9 XPBD Time Step with N Iteration

function $\mathbf{f}_{\text{XPBD}}(\mathbf{q}_t = [\,\mathbf{x}_t,\, \mathbf{v}_t\,],\, \tilde{\mathbf{A}} = [\,\tilde{\boldsymbol{\alpha}}_1,\, \tilde{\boldsymbol{\alpha}}_2, \ldots,\, \tilde{\boldsymbol{\alpha}}_m\,])$
  $\mathbf{x}_0 \leftarrow \mathbf{x}_t + h(\mathbf{v}_t + h\,\mathbf{M}^{-1}\mathbf{f}_{\text{ext}})$
  $\lambda_0 \leftarrow \mathbf{0}$
  for $k \in [\,1, \ldots, N\,]$ **do**
    $\Delta\lambda^k \leftarrow \Delta\lambda(\mathbf{x}^{k-1}, \lambda^{k-1}, \tilde{\mathbf{A}})$
    $\Delta\mathbf{x}^k \leftarrow \Delta\mathbf{x}(\mathbf{x}^{k-1}, \Delta\lambda^k)$
    $\lambda^k \leftarrow \lambda^{k-1} + \Delta\lambda^k$
    $\mathbf{x}^k \leftarrow \mathbf{x}^{k-1} + \Delta\mathbf{x}^k$
  $\mathbf{x}_{t+1} \leftarrow \mathbf{x}^N$
  $\mathbf{v}_{t+1} \leftarrow (\mathbf{x}_{t+1} - \mathbf{x}_t)\,/\,h$
end function

function $\Delta\lambda(\mathbf{x}^{k-1}, \lambda^{k-1}, \tilde{\mathbf{A}})$
  for i constraints **do**
    $\Delta\lambda_i \leftarrow \dfrac{-\mathbf{C}_i(\mathbf{x}^{k-1}) - \tilde{\boldsymbol{\alpha}}_i\lambda_i^{k-1}}{\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i}$
  $\Delta\lambda = [\,\Delta\lambda_1, \Delta\lambda_2, \ldots, \Delta\lambda_m\,]$
end function

function $\Delta\mathbf{x}(\mathbf{x}^{k-1}, \Delta\lambda^k)$
  for i constraints **do**
    $\Delta\mathbf{x}_i \leftarrow \mathbf{M}^{-1}\mathbf{J}_i\Delta\lambda_i^k$
  $\Delta\mathbf{x} = \Delta\mathbf{x}_1 + \Delta\mathbf{x}_2 + \cdots + \Delta\mathbf{x}_m$
end function

To compute the gradient of the time-step function we will need to compute, for each iteration k, these derivatives with respect to $\mathbf{x}_t$:

$$
\begin{aligned}
\frac{\partial\,\Delta\lambda^k}{\partial\,\mathbf{x}_t} &= \frac{\partial\,\Delta\lambda^k}{\partial\,\mathbf{x}^{k-1}}\frac{\partial\,\mathbf{x}^{k-1}}{\partial\,\mathbf{x}_t} + \frac{\partial\,\Delta\lambda^k}{\partial\,\lambda^{k-1}}\frac{\partial\,\lambda^{k-1}}{\partial\,\mathbf{x}_t} && \in \mathbb{R}^{m\times 3n} \\
\frac{\partial\,\Delta\mathbf{x}^k}{\partial\,\mathbf{x}_t} &= \frac{\partial\,\Delta\mathbf{x}^k}{\partial\,\mathbf{x}^{k-1}}\frac{\partial\,\mathbf{x}^{k-1}}{\partial\,\mathbf{x}_t} + \frac{\partial\,\Delta\mathbf{x}}{\partial\,\Delta\lambda^k}\frac{\partial\,\Delta\lambda^k}{\partial\,\mathbf{x}_t} && \in \mathbb{R}^{3n\times 3n} \\
\frac{\partial\,\lambda^k}{\partial\,\mathbf{x}_t} &= \frac{\partial\,\lambda^{k-1}}{\partial\,\mathbf{x}_t} + \frac{\partial\,\Delta\lambda^k}{\partial\,\mathbf{x}_t} && \in \mathbb{R}^{m\times 3n} \\
\frac{\partial\,\mathbf{x}^k}{\partial\,\mathbf{x}_t} &= \frac{\partial\,\mathbf{x}^{k-1}}{\partial\,\mathbf{x}_t} + \frac{\partial\,\Delta\mathbf{x}^k}{\partial\,\mathbf{x}_t} && \in \mathbb{R}^{3n\times 3n}
\end{aligned}
\tag{102}
$$

with base cases:

$$\begin{aligned} \frac{\partial \lambda^0}{\partial \mathbf{x}_t} &= \mathbf{0} \quad \in \mathbb{R}^{m\times 3n} & \frac{\partial \mathbf{x}^0}{\partial \mathbf{x}_t} &= \mathbf{I} \quad \in \mathbb{R}^{3n\times 3n} \\ \frac{\partial \lambda^0}{\partial \mathbf{v}_t} &= \mathbf{0} \quad \in \mathbb{R}^{m\times 3n} & \frac{\partial \mathbf{x}^0}{\partial \mathbf{v}_t} &= h\,\mathbf{I} \quad \in \mathbb{R}^{3n\times 3n} \end{aligned} \tag{103}$$

The blue terms are supposed to be available from previous iteration and the green ones will already have been computed in the same iteration. That leaves us with the need to compute the following 4 analytical derivatives:

$$\begin{aligned} \frac{\partial \Delta\lambda^k}{\partial \lambda^{k-1}} &= -\tilde{\mathbf{A}} && \in \mathbb{R}^{m\times m} \\ \frac{\partial \Delta\mathbf{x}}{\partial \Delta\lambda^k} &= \mathbf{M}^{-1}\underbrace{[\,\mathbf{J}_1, \mathbf{J}_2, \ldots, \mathbf{J}_m\,]^\top}_{\mathbf{J}} && \in \mathbb{R}^{3n\times m} \\ \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}^{k-1}} &= \sum_i \frac{\partial \Delta\mathbf{x}_i}{\partial \mathbf{x}^{k-1}} = \sum_i \mathbf{M}^{-1}\mathbf{H}_i \Delta\lambda_i^k && \in \mathbb{R}^{3n\times 3n} \\ \frac{\partial \Delta\lambda}{\partial \mathbf{x}^{k-1}} &= \left[\frac{\partial \Delta\lambda_1}{\partial \mathbf{x}^{k-1}} \frac{\partial \Delta\lambda_2}{\partial \mathbf{x}^{k-1}} \cdots \frac{\partial \Delta\lambda_m}{\partial \mathbf{x}^{k-1}}\right] && \in \mathbb{R}^{m\times 3n} \end{aligned} \tag{104}$$

where:

$$\frac{\partial \Delta\lambda_i}{\partial \mathbf{x}^{k-1}} = \frac{-\mathbf{J}_i}{\mathbf{D}_i^{k-1}} + \mathbf{b}_i^{k-1}\left(\frac{2\,\mathbf{H}_i\mathbf{M}^{-1}\mathbf{J}_i}{(\mathbf{D}_i^{k-1})^2}\right) \tag{105}$$

with:

$$\mathbf{D}_i^{k-1} = \mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i \qquad \mathbf{b}_i^{k-1} = \mathbf{C}_i(\mathbf{x}^{k-1}) + \tilde{\boldsymbol{\alpha}}_i\lambda_i^{k-1}$$

Finally, the full derivative of the time-stepping function over the input state is:

$$\frac{\partial \mathbf{f}_{\mathrm{XPBD}_t}}{\partial \mathbf{q}_t} = \begin{bmatrix} \dfrac{\partial \mathbf{x}^N}{\partial \mathbf{x}_t} & h\dfrac{\partial \mathbf{x}^N}{\partial \mathbf{x}_t} \\ \dfrac{1}{h}\left(\dfrac{\partial \mathbf{x}^N}{\partial \mathbf{x}_t} - \mathbf{I}\right) & \dfrac{\partial \mathbf{x}^N}{\partial \mathbf{x}_t} \end{bmatrix} \tag{106}$$

Now, this is enough to calculate the adjoint states. To calculate the gradient of the objective function with respect to $\tilde{\mathbf{A}}$ we need to compute $\dfrac{\partial \mathbf{f}_{\mathrm{XPBD}}}{\partial \tilde{\mathbf{A}}}$ for a

single time-stepping pass, so for each iteration k we will need:

$$\begin{aligned}
\frac{\partial \Delta\lambda^k}{\partial \tilde{\mathbf{A}}} &= \frac{\partial \Delta\lambda}{\partial \mathbf{x}^{k-1}}\frac{\partial \mathbf{x}^{k-1}}{\partial \tilde{\mathbf{A}}} + \frac{\partial \Delta\lambda}{\partial \lambda^{k-1}}\frac{\partial \lambda^{k-1}}{\partial \tilde{\mathbf{A}}} + \frac{\partial \Delta\lambda}{\partial \tilde{\mathbf{A}}} && \in \mathbb{R}^{m\times m} \\
\frac{\partial \Delta\mathbf{x}^k}{\partial \tilde{\mathbf{A}}} &= \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}^{k-1}}\frac{\partial \mathbf{x}^{k-1}}{\partial \tilde{\mathbf{A}}} + \frac{\partial \Delta\mathbf{x}}{\partial \Delta\lambda^{k}}\frac{\partial \Delta\lambda^{k}}{\partial \tilde{\mathbf{A}}} && \in \mathbb{R}^{3n\times m} \\
\frac{\partial \lambda^k}{\partial \tilde{\mathbf{A}}} &= \frac{\partial \lambda^{k-1}}{\partial \tilde{\mathbf{A}}} + \frac{\partial \Delta\lambda^k}{\partial \tilde{\mathbf{A}}} && \in \mathbb{R}^{m\times m} \\
\frac{\partial \mathbf{x}^k}{\partial \tilde{\mathbf{A}}} &= \frac{\partial \mathbf{x}^{k-1}}{\partial \tilde{\mathbf{A}}} + \frac{\partial \Delta\mathbf{x}^k}{\partial \tilde{\mathbf{A}}} && \in \mathbb{R}^{3n\times m}
\end{aligned} \tag{107}$$

with base cases:

$$\frac{\partial \lambda^0}{\partial \tilde{\mathbf{A}}} = \mathbf{0} \quad \in \mathbb{R}^{m\times n} \qquad \frac{\partial \mathbf{x}^0}{\partial \tilde{\mathbf{A}}} = \mathbf{0} \quad \in \mathbb{R}^{3n\times m} \tag{108}$$

Again, blue terms are supposed to be available from previous iteration and green ones will already have been computed in the same iteration (the shown dimensions treat $\tilde{\mathbf{A}}$ as a vector, $\mathbb{R}^m$, and not as a diagonal matrix). We only need to calculate one more analytical formula:

$$\frac{\partial \Delta\lambda}{\partial \tilde{\mathbf{A}}} = \left[\frac{\partial \Delta\lambda_1}{\partial \tilde{\mathbf{A}}}, \ldots, \frac{\partial \Delta\lambda_m}{\partial \tilde{\mathbf{A}}}\right] = \operatorname{diag}\left(\frac{\partial \Delta\lambda_1}{\partial \tilde{\boldsymbol{\alpha}}_1}, \ldots, \frac{\partial \Delta\lambda_m}{\partial \tilde{\boldsymbol{\alpha}}_m}\right) \qquad \in \mathbb{R}^{m\times m}$$

where:

$$\frac{\partial \Delta\lambda_i}{\partial \tilde{\boldsymbol{\alpha}}_i} = \frac{\mathbf{C}_i(\mathbf{x}^{k-1}) - \lambda_i^{k-1}(\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top)}{(\mathbf{J}_i\mathbf{M}^{-1}\mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i)^2} \tag{109}$$

Algorithm 10 Differentiated XPBD Time Step with k Iterations

function $\mathbf{f}_{\text{XPBD}}(\mathbf{x}_t, \mathbf{v}_t, \mathbf{M}, \tilde{\mathbf{A}}, \mathbb{C}, N)$
  $\mathbf{x}^- \leftarrow \mathbf{x}_t + h(\mathbf{v}_t + h\mathbf{M}^{-1}\mathbf{f}_{\text{ext}})$
  $\lambda^- \leftarrow \mathbf{0}$
  $\frac{\partial \lambda^-}{\partial \mathbf{x}_t} \leftarrow \mathbf{0}$
  $\frac{\partial \mathbf{x}^-}{\partial \mathbf{x}_t} \leftarrow \mathbf{I}$
  for $k \in [\,1, \ldots, N\,]$ **do**
    $\frac{\partial \Delta\lambda}{\partial \mathbf{x}^-} \leftarrow \mathbf{0}$
    $\frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}^-} \leftarrow \mathbf{0}$
    $\frac{\partial \Delta\mathbf{x}}{\partial \Delta\lambda} \leftarrow \mathbf{0}$
    for each constraint $\mathbb{C}_i \in \mathbb{C}$ **do**
      $\mathbf{C} \leftarrow \mathbf{C}_i(\mathbf{x}^-)$
      $\mathbf{J} \leftarrow \mathbf{J}_i(\mathbf{x}^-)$
      $\mathbf{H} \leftarrow \mathbf{H}_i(\mathbf{x}^-)$
      $\Delta\lambda \leftarrow \Delta\lambda(\mathbf{C}, \mathbf{J}, \lambda_i^-, \mathbf{M}, \tilde{\boldsymbol{\alpha}}_i)$ ▷ Eq. 92
      $\Delta\mathbf{x} \leftarrow \Delta\mathbf{x}(\mathbf{J}, \Delta\lambda, \mathbf{M})$ ▷ Eq. 93
      $\frac{\partial \Delta\lambda_i}{\partial \mathbf{x}^-} \leftarrow \frac{\partial \Delta\lambda}{\partial \mathbf{x}^-}(\mathbf{C}, \mathbf{J}, \mathbf{H}, \lambda_i^-, \mathbf{M}, \tilde{\boldsymbol{\alpha}}_i)$ ▷ Eq. 105
      $\frac{\partial \Delta\mathbf{x}_i}{\partial \mathbf{x}^-} \leftarrow \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}^-}(\mathbf{H}, \Delta\lambda, \mathbf{M})$ ▷ Eq. 104
      $\frac{\partial \Delta\mathbf{x}_i}{\partial \Delta\lambda_i} \leftarrow \frac{\partial \Delta\mathbf{x}}{\partial \Delta\lambda}(\mathbf{J}, \mathbf{M})$ ▷ Eq. 104
      $\frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}^-} \leftarrow \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}^-} + \frac{\partial \Delta\mathbf{x}_i}{\partial \mathbf{x}^-}$
      $\text{ROW}\left(\frac{\partial \Delta\lambda}{\partial \mathbf{x}^-}, i\right) \leftarrow \frac{\partial \Delta\lambda_i}{\partial \mathbf{x}^-}$
      $\text{COL}\left(\frac{\partial \Delta\mathbf{x}}{\partial \Delta\lambda}, i\right) \leftarrow \frac{\partial \Delta\mathbf{x}_i}{\partial \Delta\lambda_i}$
    for each constraint $\mathbb{C}_i \in \mathbb{C}$ **do**
      $\lambda^- \leftarrow \lambda^- + \Delta\lambda_i$
      $\mathbf{x}^- \leftarrow \mathbf{x}^- + \Delta\mathbf{x}_i$
    $\frac{\partial \Delta\lambda}{\partial \mathbf{x}_t} \leftarrow \frac{\partial \Delta\lambda}{\partial \mathbf{x}^-}\frac{\partial \mathbf{x}^-}{\partial \mathbf{x}_t} - \tilde{\mathbf{A}}\frac{\partial \lambda^-}{\partial \mathbf{x}_t}$ ▷ Eq. 102
    $\frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t} \leftarrow \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}^-}\frac{\partial \mathbf{x}^-}{\partial \mathbf{x}_t} + \frac{\partial \Delta\mathbf{x}}{\partial \Delta\lambda}\frac{\partial \Delta\lambda}{\partial \mathbf{x}_t}$ ▷ Eq. 102
    $\frac{\partial \lambda^-}{\partial \mathbf{x}_t} \leftarrow \frac{\partial \lambda^-}{\partial \mathbf{x}_t} + \frac{\partial \Delta\lambda}{\partial \mathbf{x}_t}$ ▷ Eq. 102
    $\frac{\partial \mathbf{x}^-}{\partial \mathbf{x}_t} \leftarrow \frac{\partial \mathbf{x}^-}{\partial \mathbf{x}_t} + \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}_t}$ ▷ Eq. 102
  $\mathbf{x}_{t+1} \leftarrow \mathbf{x}^-$
  $\mathbf{v}_{t+1} \leftarrow \frac{1}{h}(\mathbf{x}_{t+1} - \mathbf{x}_t)$
  $\frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{x}_t} \leftarrow \frac{\partial \mathbf{x}^-}{\partial \mathbf{x}_t}$
end function

7.2.3 k-iterations Case (second method)

Algorithm 11 Another view of the multi-iteration XPBD time step solve.

function $\mathbf{f}_{\text{XPBD}}(\mathbf{x}^-, \mathbf{v}^-)$
  $\mathbf{x}^{(0)} = \mathbf{x}^- + h\,(\mathbf{v}^- + h\,\mathbf{M}^{-1}\mathbf{f}_{\text{ext}})$
  $\boldsymbol{\lambda}^{(0)} = \mathbf{0}$
  for $k \in [1, 2, \ldots, N]$ **do**
    $\Delta\boldsymbol{\lambda}^{(k)} = \mathbf{g}\,(\,\mathbf{x}^{(k-1)},\, \boldsymbol{\lambda}^{(k-1)}\,)$
    $\Delta\mathbf{x}^{(k)} = \mathbf{h}\,(\,\mathbf{x}^{(k-1)},\, \Delta\boldsymbol{\lambda}\,)$
    $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \Delta\mathbf{x}^{(k)}$
    $\boldsymbol{\lambda}^{(k)} = \boldsymbol{\lambda}^{(k-1)} + \Delta\boldsymbol{\lambda}^{(k)}$
  $\mathbf{x}^+ = \mathbf{x}^{(N)}$
  $\mathbf{v}^+ = \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$
end function

$$\mathbb{J}^{(k)} = \begin{bmatrix} \dfrac{\partial\,\mathbf{x}^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} & \dfrac{\partial\,\mathbf{x}^{(k)}}{\partial\,\boldsymbol{\lambda}^{(k-1)}} \\ \dfrac{\partial\,\boldsymbol{\lambda}^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} & \dfrac{\partial\,\boldsymbol{\lambda}^{(k)}}{\partial\,\boldsymbol{\lambda}^{(k-1)}} \end{bmatrix} \tag{110}$$

$$\mathbb{J} = \mathbb{J}^{(N)}\,\mathbb{J}^{(N-1)} \cdots \mathbb{J}^{(2)}\,\mathbb{J}^{(1)} \tag{111}$$

From $\mathbb{J}$ we extract $\frac{\partial \mathbf{x}^{(N)}}{\partial \mathbf{x}^{(0)}}$, which is what we need.

$$\frac{\partial\,\mathbf{x}^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} = \mathbf{I} + \nabla_{\mathbf{h}}(\mathbf{x}^{(k-1)}) + \nabla_{\mathbf{h}}(\Delta\boldsymbol{\lambda}^{(k)})\,\nabla_{\mathbf{g}}(\mathbf{x}^{(k-1)}) \qquad \in \mathbb{R}^{3n\times 3n} \tag{112}$$

$$\frac{\partial\,\mathbf{x}^{(k)}}{\partial\,\boldsymbol{\lambda}^{(k-1)}} = \nabla_{\mathbf{h}}(\Delta\boldsymbol{\lambda}^{(k)})\,\nabla_{\mathbf{g}}(\boldsymbol{\lambda}^{(k-1)}) \qquad \in \mathbb{R}^{3n\times m} \tag{113}$$

$$\frac{\partial\,\boldsymbol{\lambda}^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} = \nabla_{\mathbf{g}}(\mathbf{x}^{(k-1)}) \qquad \in \mathbb{R}^{m\times 3n} \tag{114}$$

$$\frac{\partial\,\boldsymbol{\lambda}^{(k)}}{\partial\,\boldsymbol{\lambda}^{(k-1)}} = \mathbf{I} + \nabla_{\mathbf{g}}(\boldsymbol{\lambda}^{(k-1)}) \qquad \in \mathbb{R}^{m\times m} \tag{115}$$

The block $\nabla_{\mathbf{g}}(\boldsymbol{\lambda}^{(k-1)})$ is diagonal: $\Delta\lambda_i^{(k)}$ depends on $\lambda_i^{(k-1)}$ only through the

compliance term of its own constraint update, with no inter-constraint coupling.

$$\nabla_{\mathbf{g}}(\boldsymbol{\lambda}^{(k-1)}) = \begin{bmatrix} \frac{\partial \Delta\lambda_1^{(k)}}{\partial \lambda_1^{(k-1)}} & 0 & \cdots & 0 \\ 0 & \frac{\partial \Delta\lambda_2^{(k)}}{\partial \lambda_2^{(k-1)}} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{\partial \Delta\lambda_m^{(k)}}{\partial \lambda_m^{(k-1)}} \end{bmatrix} \quad \in \mathbb{R}^{m\times m} \tag{116}$$

Each diagonal entry follows directly from the XPBD update for constraint i:

$$\Delta\lambda_i^{(k)} = \frac{-\mathbf{C_i}(\mathbf{x^{(k-1)}}) - \tilde{\boldsymbol{\alpha}}_i \lambda_i^{(k-1)}}{\underbrace{\mathbf{J}_i \mathbf{M}^{-1} \mathbf{J}_i^\top + \tilde{\boldsymbol{\alpha}}_i}_{\mathbf{D}_i}}$$

Only the numerator depends on $\lambda_i^{(k-1)}$, giving

$$\frac{\partial\, \Delta\lambda_i^{(k)}}{\partial\, \lambda_i^{(k-1)}} = -\frac{\tilde{\boldsymbol{\alpha}}_i}{\mathbf{D_i}} \quad \in \mathbb{R} \tag{117}$$

Collecting these scalars, the block factors as a product of diagonal terms:

$$\nabla_{\mathbf{g}}(\boldsymbol{\lambda}^{(k-1)}) = -\bar{\mathbf{D}}\, \tilde{\mathbf{A}} \tag{118}$$

with $\bar{\mathbf{D}} = \mathrm{diag}(\mathbf{D}_1^{-1}, \mathbf{D}_2^{-1}, \ldots, \mathbf{D}_m^{-1}) \in \mathbb{R}^{m\times m}$ being the matrix/vector of inverse denominators, and $\tilde{\mathbf{A}} \in \mathbb{R}^{m\times m}$ the diagonal matrix of compliances. The whole factor is constant across iterations and time steps whenever the denominators $\mathbf{D}_i$ do not depend on positions, i.e. when $\mathbf{J}_i = \nabla \mathbf{C}_i$ is position-independent and $\mathbf{M}$, $\tilde{\boldsymbol{\alpha}}$, and h are fixed; under those conditions it can be precomputed once.

The block $\nabla_{\mathbf{h}}(\Delta\boldsymbol{\lambda}^{(k)})$ is sparse: column i is nonzero only on the DOFs of the particles touched by constraint i.

$$\nabla_{\mathbf{h}}(\Delta\boldsymbol{\lambda}^{(k)}) = \left[\frac{\partial\, \Delta\mathbf{x}_1^{(k)}}{\partial\, \Delta\lambda_1^{(k)}} \;\middle|\; \frac{\partial\, \Delta\mathbf{x}_2^{(k)}}{\partial\, \Delta\lambda_2^{(k)}} \;\middle|\; \cdots \;\middle|\; \frac{\partial\, \Delta\mathbf{x}_m^{(k)}}{\partial\, \Delta\lambda_m^{(k)}} \right] \quad \in \mathbb{R}^{3n\times m} \tag{119}$$

The position update contributed by constraint i is

$$\Delta\mathbf{x}_i^{(k)} = \mathbf{M}^{-1}\mathbf{J}_i^\top\, \Delta\lambda_i^{(k)}$$

so each column of the block is

$$\frac{\partial\, \Delta\mathbf{x}_i^{(k)}}{\partial\, \Delta\lambda_i^{(k)}} = \mathbf{M}^{-1}\mathbf{J}_i^\top \tag{120}$$

Stacking the columns recovers the full constraint Jacobian:

$$\nabla_{\mathbf{h}}(\Delta\boldsymbol{\lambda}^{(k)}) = \mathbf{M}^{-1}\mathbf{J}^{\top} \tag{121}$$

where $\mathbf{J} \in \mathbb{R}^{m\times 3n}$ is the matrix whose rows are the per-constraint Jacobians $\mathbf{J}_i = \nabla\mathbf{C}_i$.

We now turn to the block $\nabla_{\mathbf{h}}(\mathbf{x}^{(k-1)})$. It is also sparse and decomposes as a sum of per-constraint contributions:

$$\nabla_{\mathbf{h}}(\mathbf{x}^{(k-1)}) = \frac{\partial\,\Delta\mathbf{x}_1^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} + \frac{\partial\,\Delta\mathbf{x}_2^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} + \cdots + \frac{\partial\,\Delta\mathbf{x}_m^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} \qquad \in \mathbb{R}^{3n\times 3n} \tag{122}$$

Each term comes from differentiating $\Delta\mathbf{x}_i^{(k)} = \mathbf{M}^{-1}\mathbf{J}_i^{\top}\,\Delta\lambda_i^{(k)}$ in $\mathbf{x}^{(k-1)}$, picking up the constraint Hessian $\mathbf{H}_i = \nabla^2\mathbf{C}_i$:

$$\frac{\partial\,\Delta\mathbf{x}_i^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} = \mathbf{M}^{-1}\mathbf{H}_i\,\Delta\lambda_i^{(k)} \qquad \in \mathbb{R}^{3n\times 3n} \tag{123}$$

Factoring out $\mathbf{M}^{-1}$, with the shorthand

$$\mathbf{H}(\boldsymbol{\mu}) \;:=\; \sum_{i=1}^{m} \mathbf{H}_i\,\mu_i \qquad \in \mathbb{R}^{3n\times 3n}$$

the block becomes

$$\nabla_{\mathbf{h}}(\mathbf{x}^{(k-1)}) = \mathbf{M}^{-1}\,\mathbf{H}(\Delta\boldsymbol{\lambda}^{(k)}) \tag{124}$$

Each per-constraint term is nonzero only on the DOFs of the particles touched by constraint i, so in practice the partials $\frac{\partial\Delta\mathbf{x}_i^{(k)}}{\partial\mathbf{x}^{(k-1)}}$ are small local blocks that get scattered into the full sparse matrix.

For the last block, $\nabla_{\mathbf{g}}(\mathbf{x}^{(k-1)})$, we restrict ourselves to distance constraints, whose denominator $\mathbf{D}$ is position-independent and admits further simplification.

$$\nabla_{\mathbf{g}}(\mathbf{x}^{(k-1)}) = \begin{bmatrix} \dfrac{\partial\,\Delta\lambda_1^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} \\ \dfrac{\partial\,\Delta\lambda_2^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} \\ \vdots \\ \dfrac{\partial\,\Delta\lambda_m^{(k)}}{\partial\,\mathbf{x}^{(k-1)}} \end{bmatrix} \qquad \in \mathbb{R}^{m\times 3n} \tag{125}$$

For a distance constraint the update reduces to

$$\Delta\lambda_i^{(k)} = \frac{-\mathbf{C_i}(\mathbf{x^{k-1}}) - \tilde{\boldsymbol{\alpha}}_i\lambda_i^{(k-1)}}{\underbrace{w_{i_1} + w_{i_2} + \tilde{\boldsymbol{\alpha}}_i}_{\mathbf{D}_i}},$$

with $\mathbf{D}_i$ depending only on the inverse masses of the two endpoints and on the compliance. The denominator is therefore constant in $\mathbf{x}^{(k-1)}$, and only $\mathbf{C}_i$ contributes to the derivative:

$$\frac{\partial \Delta\lambda_i^{(k)}}{\partial \mathbf{x}^{(k-1)}} = -\mathbf{D}_i^{-1}\mathbf{J}_i \qquad \in \mathbb{R}^{1\times 3n} \tag{126}$$

Stacking the rows gives the full block in factored form:

$$\nabla_{\mathbf{g}}(\mathbf{x}^{(k-1)}) = -\bar{\mathbf{D}}\,\mathbf{J}. \tag{127}$$

Finally we arrive at:

$$\frac{\partial \mathbf{x}^{(k)}}{\partial \mathbf{x}^{(k-1)}} = \mathbf{I} + \mathbf{W}\,[\,\mathbf{H}(\Delta\boldsymbol{\lambda}^{(k)}) - \mathbf{J}^\top\bar{\mathbf{D}}\mathbf{J}\,] \tag{128}$$

$$\frac{\partial \mathbf{x}^{(k)}}{\partial \boldsymbol{\lambda}^{(k-1)}} = -\mathbf{W}\mathbf{J}^\top\bar{\mathbf{D}}\,\tilde{\mathbf{A}} \tag{129}$$

$$\frac{\partial \boldsymbol{\lambda}^{(k)}}{\partial \mathbf{x}^{(k-1)}} = -\bar{\mathbf{D}}\mathbf{J} \tag{130}$$

$$\frac{\partial \boldsymbol{\lambda}^{(k)}}{\partial \boldsymbol{\lambda}^{(k-1)}} = \mathbf{I} - \bar{\mathbf{D}}\,\tilde{\mathbf{A}} \tag{131}$$

7.2.4 DiffXPBD [10]

DiffXPBD applies the implicit form of the adjoint method, in which the backward recursion satisfies

$$\hat{\mathbf{q}}^- = \left(\frac{\partial \mathbf{f}^-}{\partial \mathbf{q}^+}\right)^\top \hat{\mathbf{q}}^- + \left(\frac{\partial \mathbf{f}^+}{\partial \mathbf{q}^+}\right)^\top \hat{\mathbf{q}}^+ + \left(\frac{\partial \phi}{\partial \mathbf{q}^+}\right)^\top \tag{132}$$

Applied to the XPBD update for $\mathbf{q} = (\mathbf{x}, \mathbf{v})$, this yields

$$\hat{\mathbf{x}}^- = \hat{\mathbf{x}}^+ + \left(\frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}} + h^2\mathbf{M}^{-1}\frac{\partial \mathbf{f}_{\text{ext}}}{\partial \mathbf{x}}\right)\hat{\mathbf{x}}^- + \frac{\hat{\mathbf{v}}^- - \hat{\mathbf{v}}^+}{h} + \left(\frac{\partial \phi}{\partial \mathbf{x}}\right)^\top \tag{133}$$

$$\hat{\mathbf{v}}^- = \left(\frac{\partial \Delta\mathbf{x}}{\partial \mathbf{v}} + h^2\mathbf{M}^{-1}\frac{\partial \mathbf{f}_{\text{ext}}}{\partial \mathbf{v}}\right)\hat{\mathbf{x}}^- + h\,\hat{\mathbf{x}}^+ + \left(\frac{\partial \phi}{\partial \mathbf{v}}\right)^\top \tag{134}$$

Under the standard assumptions that $\mathbf{f}_{\text{ext}}$ is independent of $\mathbf{q}$ and $\Delta\mathbf{x}$ is independent of $\mathbf{v}$, eliminating $\hat{\mathbf{v}}^-$ gives the linear system

$$\left(\mathbf{I} - \frac{\partial \Delta\mathbf{x}}{\partial \mathbf{x}}\right)\hat{\mathbf{x}}^- = 2\,\hat{\mathbf{x}}^+ - \frac{\hat{\mathbf{v}}^+}{h} + \left(\frac{\partial \phi}{\partial \mathbf{x}}\right)^\top \tag{135}$$

which is solved at each backward step.

7.2.5 General Case with Collision Response

Algorithm 12 Single differentiable simulation step: explicit integration, XPBD constraint solve, and collision response.

1: **function** SIMULATIONSTEP($\mathbf{x}^-, \mathbf{v}^-$)
2: $\quad\tilde{\mathbf{x}} \leftarrow \mathbb{I}(\mathbf{x}^-, \mathbf{v}^-)$
3: $\quad\bar{\mathbf{x}} \leftarrow \mathbb{X}(\tilde{\mathbf{x}})$
4: $\quad\mathbf{x}^+ \leftarrow \mathbb{C}(\bar{\mathbf{x}}, \mathbf{x}^-)$
5: $\quad\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$
6: $\quad$**return** $[\mathbf{x}^+, \mathbf{v}^+]$
7: **end function**

Where $\mathbb{I}(\mathbf{x}^-, \mathbf{v}^-) = \mathbf{x}^- + h(\mathbf{v}^- + h\mathbf{M}^{-1}\mathbf{f}_{\text{ext}})$ is the explicit symplectic Euler predictor used by XPBD, $\mathbb{X}$ is the XPBD constraint solve enforcing the internal forces of the simulated body, and $\mathbb{C}$ is the collision response step (in blue the term needed for handling friction at position level), which corrects the positions to resolve any detected interpenetrations. Internal forces and collisions/friction are handled in two decoupled sequential passes.
In order to use the adjoint method, or any gradient propagation technique, we need to be able to calculate the derivatives of the result of the simulation step function with respect to input parameters. In particular: $\frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-}$, $\frac{\partial \mathbf{x}^+}{\partial \mathbf{v}^-}$, $\frac{\partial \mathbf{v}^+}{\partial \mathbf{x}^-}$ and $\frac{\partial \mathbf{v}^+}{\partial \mathbf{v}^-}$. We start with:

$$\frac{d\,\mathbf{x}^+}{d\,\mathbf{x}^-} = \frac{\partial\,\mathbb{C}}{\partial\,\bar{\mathbf{x}}}\frac{\partial\,\mathbb{X}}{\partial\,\tilde{\mathbf{x}}}\frac{\partial\,\mathbb{I}}{\partial\,\mathbf{x}^-} + \frac{\partial\,\mathbb{C}}{\partial\,\mathbf{x}^-} = \frac{\partial\,\mathbf{x}^+}{\partial\,\bar{\mathbf{x}}}\frac{\partial\,\bar{\mathbf{x}}}{\partial\,\tilde{\mathbf{x}}}\frac{\partial\,\tilde{\mathbf{x}}}{\partial\,\mathbf{x}^-} + \frac{\partial\,\mathbf{x}^+}{\partial\,\mathbf{x}^-} \tag{136}$$

where $\frac{\partial \mathbf{x}^+}{\partial \bar{\mathbf{x}}}$ is the derivative of the collision response step, $\frac{\partial \bar{\mathbf{x}}}{\partial \tilde{\mathbf{x}}}$ is the derivative of the XPBD step, which we derived in the previous sections, $\frac{\partial \tilde{\mathbf{x}}}{\partial \mathbf{x}^-}$ is simply $\mathbf{I}$ for the explicit integration method used here (we assume $\mathbf{f}_{\text{ext}}$ to be independent of $\mathbf{x}^-$ and $\mathbf{v}^-$), and $\frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-}$ is present when friction is handled by the collision response algorithm. So that for the 1-iteration case it can be simplified to be:

$$\frac{d\,\mathbf{x}^+}{d\,\mathbf{x}^-} = \frac{\partial\,\mathbb{C}}{\partial\,\bar{\mathbf{x}}}\left[\mathbf{I} + \frac{\partial\,\Delta\mathbf{x}}{\partial\,\tilde{\mathbf{x}}}\right] + \frac{\partial\,\mathbb{C}}{\partial\,\mathbf{x}^-}$$

The other derivatives are:

$$\frac{d\,\mathbf{v}^+}{d\,\mathbf{x}^-} = \frac{1}{h}\left(\frac{\partial\,\mathbf{x}^+}{\partial\,\mathbf{x}^-} - \mathbf{I}\right) \tag{137}$$

$$\frac{d\,\mathbf{x}^+}{d\,\mathbf{v}^-} = \underbrace{\frac{\partial\,\mathbf{x}^+}{\partial\,\bar{\mathbf{x}}}\frac{\partial\,\bar{\mathbf{x}}}{\partial\,\tilde{\mathbf{x}}}}_{\frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-}}\underbrace{\frac{\partial\,\tilde{\mathbf{x}}}{\partial\,\mathbf{v}^-}}_{h} = h\frac{\partial\,\mathbf{x}^+}{\partial\,\mathbf{x}^-} \tag{138}$$

$$\frac{d\,\mathbf{v}^+}{d\,\mathbf{v}^-} = \frac{1}{h}\left(\frac{\partial\,\mathbf{x}^+}{\partial\,\mathbf{v}^-}\right) = \frac{\partial\,\mathbf{x}^+}{\partial\,\mathbf{x}^-} \tag{139}$$

8 Collisions

8.1 Vertex Projection

The simplest approach to collision response operates independently on each vertex, at position level: for every particle position $\bar{\mathbf{x}}_i$ that penetrates a collider, we project it onto the nearest point on the collider surface (we use $\bar{\mathbf{x}}$ because we suppose this pass is applied after the internal forces solve). No coupling between vertices is considered and no global solve is required. This method is cheap, trivially parallelizable, and sufficient for many practical scenarios involving static rigid colliders.
Formally, given a collider defined by a signed distance function $\Phi(\mathbf{x}) \geq 0$ outside and $\Phi(\mathbf{x}) < 0$ inside, the projection of a penetrating vertex is (we will now drop the i subscript, remembering this is done for each particle in the system):

$$\bar{\mathbf{x}}^+ = \begin{cases} \bar{\mathbf{x}} & \text{if } \Phi(\bar{\mathbf{x}}) \geq 0 \\ \bar{\mathbf{x}} + |\Phi(\bar{\mathbf{x}})|\, \hat{\mathbf{n}}(\bar{\mathbf{x}}) & \text{if } \Phi(\bar{\mathbf{x}}) < 0 \end{cases} \tag{140}$$

where $\hat{\mathbf{n}}(\bar{\mathbf{x}})$ is the outward unit normal at the closest point on the collider surface to $\bar{\mathbf{x}}$, and $|\Phi(\bar{\mathbf{x}})|$ is the penetration depth. For analytic shapes both quantities are available in closed form; for general shapes they require a closest-point query.

Friction. For any active collision, a position-level friction correction can be applied after the normal projection. For example, let $\hat{\mathbf{n}}$ be the contact normal and decompose the total displacement during the timestep into normal and tangential components:

$$\Delta\mathbf{x}_{\text{norm}} = \left[(\bar{\mathbf{x}}^+ - \mathbf{x}^-) \cdot \hat{\mathbf{n}}\right] \hat{\mathbf{n}}, \qquad \Delta\mathbf{x}_{\tan} = (\bar{\mathbf{x}}^+ - \mathbf{x}^-) - \Delta\mathbf{x}_{\text{norm}}$$

The friction correction opposes tangential sliding, clamped by the Coulomb cone:

$$\mathbf{x}^+ = \bar{\mathbf{x}}^+ - \textsc{Clamp}(\Delta\mathbf{x}_{\tan},\ \mu\,\|\Delta\mathbf{x}_{\text{norm}}\|) \tag{141}$$

where μ is the friction coefficient and the vector clamp is defined as:

$$\textsc{Clamp}(\mathbf{v},\, c) = \frac{\mathbf{v}}{\|\mathbf{v}\|} \min(\|\mathbf{v}\|,\, c)$$

Another way of seeing it is:

$$\mathbf{x}^+ = \begin{cases} \bar{\mathbf{x}}^+ - \Delta\mathbf{x}_{\tan} & \text{if } \|\Delta\mathbf{x}_{\tan}\| \leq \mu\|\Delta\mathbf{x}_{\text{norm}}\| \\ \bar{\mathbf{x}}^+ - \mu\|\Delta\mathbf{x}_{\text{norm}}\|\ \hat{\mathbf{t}} & \text{otherwise} \end{cases} \tag{142}$$

where $\hat{\mathbf{t}} = \Delta\mathbf{x}_{\tan}/\|\Delta\mathbf{x}_{\tan}\|$ is the tangential motion unit direction. When $\|\Delta\mathbf{x}_{\tan}\| \leq \mu\|\Delta\mathbf{x}_{\text{norm}}\|$ the sliding is cancelled entirely (static regime); otherwise it is scaled down to the friction limit (kinetic regime).

Backward pass structure. The full derivative of the collision step with respect to the pre-collision positions $\bar{\mathbf{x}}$ factorizes as:

$$\frac{\partial \mathbb{C}}{\partial \bar{\mathbf{x}}} = \frac{\partial \mathbf{x}^+}{\partial \bar{\mathbf{x}}^+} \frac{\partial \bar{\mathbf{x}}^+}{\partial \bar{\mathbf{x}}} \tag{143}$$

The projection Jacobian $\frac{\partial \bar{\mathbf{x}}^+}{\partial \bar{\mathbf{x}}}$ depends on the collider geometry and is derived once per shape in the following sections. The friction Jacobian $\frac{\partial \mathbf{x}^+}{\partial \bar{\mathbf{x}}^+}$ is shape-agnostic, since it depends only on the contact normal $\hat{\mathbf{n}}$ and the displacement decomposition, and is therefore derived once here for all collider types. The backward pass requires gradients with respect to both inputs of the friction step; we derive $\frac{\partial \mathbf{x}^+}{\partial \bar{\mathbf{x}}^+}$ and $\frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-}$ in turn.

$$\frac{\partial \mathbf{x}^+}{\partial \bar{\mathbf{x}}^+} = \begin{cases} \hat{\mathbf{n}}\hat{\mathbf{n}}^\top & \text{if } \|\Delta \mathbf{x}_{\text{tan}}\| \le \mu \|\Delta \mathbf{x}_{\text{norm}}\| \\ \mathbf{I} - \mu \left[\hat{\mathbf{t}}\hat{\mathbf{n}}^\top + \dfrac{\|\Delta \mathbf{x}_{\text{norm}}\|}{\|\Delta \mathbf{x}_{\text{tan}}\|} \hat{\mathbf{b}}\hat{\mathbf{b}}^\top \right] & \text{otherwise} \end{cases} \tag{144}$$

where $\hat{\mathbf{b}} = \hat{\mathbf{n}} \times \hat{\mathbf{t}}$ is the unit vector perpendicular to both $\hat{\mathbf{n}}$ and $\hat{\mathbf{t}}$.

- if $\|\Delta \mathbf{x}_{\text{tan}}\| \le \mu \|\Delta \mathbf{x}_{\text{norm}}\|$ we have:

  $$\frac{\partial \mathbf{x}^+}{\partial \bar{\mathbf{x}}^+} = \mathbf{I} - \frac{\partial \Delta \mathbf{x}_{\text{tan}}}{\partial \bar{\mathbf{x}}^+} = \mathbf{I} - (\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top) = \hat{\mathbf{n}}\hat{\mathbf{n}}^\top$$

  since $\Delta \mathbf{x}_{\text{tan}} = (\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top)(\bar{\mathbf{x}}^+ - \mathbf{x}^-)$

- else:

  $$\frac{\partial \mathbf{x}^+}{\partial \bar{\mathbf{x}}^+} = \mathbf{I} - \mu \left[\hat{\mathbf{t}} \, \frac{\partial \|\Delta \mathbf{x}_{\text{norm}}\|}{\partial \bar{\mathbf{x}}^+} + \|\Delta \mathbf{x}_{\text{norm}}\| \frac{\partial \hat{\mathbf{t}}}{\partial \bar{\mathbf{x}}^+} \right]$$

  with:

  $$\frac{\partial \|\Delta \mathbf{x}_{\text{norm}}\|}{\partial \bar{\mathbf{x}}^+} = \frac{\Delta \mathbf{x}_{\text{norm}}^\top}{\|\Delta \mathbf{x}_{\text{norm}}\|} \frac{\partial \Delta \mathbf{x}_{\text{norm}}}{\partial \bar{\mathbf{x}}^+} = \hat{\mathbf{n}}^\top \hat{\mathbf{n}}\hat{\mathbf{n}}^\top = \hat{\mathbf{n}}^\top$$

  and:

  $$\begin{aligned} \frac{\partial \hat{\mathbf{t}}}{\partial \bar{\mathbf{x}}^+} &= \frac{1}{\|\Delta \mathbf{x}_{\text{tan}}\|} \left(\mathbf{I} - \frac{\Delta \mathbf{x}_{\text{tan}} \Delta \mathbf{x}_{\text{tan}}^\top}{\|\Delta \mathbf{x}_{\text{tan}}\|^2} \right) \frac{\partial \Delta \mathbf{x}_{\text{tan}}}{\partial \bar{\mathbf{x}}^+} \\ &= \frac{1}{\|\Delta \mathbf{x}_{\text{tan}}\|} (\mathbf{I} - \hat{\mathbf{t}}\hat{\mathbf{t}}^\top)(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top) \\ &= \frac{1}{\|\Delta \mathbf{x}_{\text{tan}}\|} (\mathbf{I} - \hat{\mathbf{t}}\hat{\mathbf{t}}^\top - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top) \end{aligned}$$

since $\mathbf{I} = \hat{\mathbf{n}}\hat{\mathbf{n}}^\top + \hat{\mathbf{t}}\hat{\mathbf{t}}^\top + \hat{\mathbf{b}}\hat{\mathbf{b}}^\top$ when $\hat{\mathbf{n}}, \hat{\mathbf{t}}, \hat{\mathbf{b}} \in \mathbb{R}^3$ are perpendicular to each others, we get the final form in (144).

Similarly we can get the partial derivative with respect to $\mathbf{x}^-$:

$$\frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} = \begin{cases} \mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top & \text{if } \|\Delta\mathbf{x}_{\text{tan}}\| \le \mu\|\Delta\mathbf{x}_{\text{norm}}\| \\ \mu\left[\hat{\mathbf{t}}\hat{\mathbf{n}}^\top + \dfrac{\|\Delta\mathbf{x}_{\text{norm}}\|}{\|\Delta\mathbf{x}_{\text{tan}}\|}\hat{\mathbf{b}}\hat{\mathbf{b}}^\top\right] & \text{otherwise} \end{cases} \tag{145}$$

8.1.1 Sphere

For a sphere with center $\mathbf{c} \in \mathbb{R}^3$ and radius r, the signed distance and outward normal are:

$$\Phi(\mathbf{x}) = \|\mathbf{x} - \mathbf{c}\| - r, \qquad \hat{\mathbf{n}}(\mathbf{x}) = \frac{\mathbf{x} - \mathbf{c}}{\|\mathbf{x} - \mathbf{c}\|}$$

The projection of a vertex $\mathbf{x}$ with $d = \|\mathbf{x} - \mathbf{c}\|$ is:

$$\bar{\mathbf{x}}^+ = \begin{cases} \bar{\mathbf{x}} & \text{if } d \ge r \\ \bar{\mathbf{x}} + (r - d)\,\hat{\mathbf{n}} & \text{if } d < r \end{cases} \tag{146}$$

For the backward pass, we need the Jacobian $\frac{\partial \bar{\mathbf{x}}^+}{\partial \mathbf{x}} \in \mathbb{R}^{3\times 3}$. Differentiating (146) gives:

$$\frac{\partial \bar{\mathbf{x}}^+}{\partial \bar{\mathbf{x}}} = \begin{cases} \mathbf{I} & \text{if } d \ge r \\ \dfrac{r}{d}\left(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top\right) & \text{if } d < r \end{cases} \tag{147}$$

Differentiating $\bar{\mathbf{x}}^+ = \mathbf{x} + (r - d)\,\hat{\mathbf{n}}$ with $\boldsymbol{\delta} = \mathbf{x} - \mathbf{c}$, $d = \|\boldsymbol{\delta}\|$, $\hat{\mathbf{n}} = \boldsymbol{\delta}/d$:

$$\frac{\partial d}{\partial \mathbf{x}} = \hat{\mathbf{n}}^\top, \qquad \frac{\partial \hat{\mathbf{n}}}{\partial \mathbf{x}} = \frac{1}{d}(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top)$$

Applying the product rule to $(r - d)\,\hat{\mathbf{n}}$:

$$\frac{\partial}{\partial \mathbf{x}}\left[(r - d)\,\hat{\mathbf{n}}\right] = -\hat{\mathbf{n}}\hat{\mathbf{n}}^\top + \frac{r - d}{d}(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top)$$

Adding the identity from $\frac{\partial \mathbf{x}}{\partial \mathbf{x}} = \mathbf{I}$ and collecting terms:

$$\frac{\partial \bar{\mathbf{x}}^+}{\partial \mathbf{x}} = \mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top + \frac{r - d}{d}(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top) = \frac{r}{d}(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top)$$

8.1.2 Capsule

A capsule is defined by a segment $(\mathbf{p}_1, \mathbf{p}_2)$ and radius r. Let $\mathbf{q}$ be the closest point on the segment to $\bar{\mathbf{x}}$:

$$\mathbf{q} = \mathbf{p}_1 + t\,(\mathbf{p}_2 - \mathbf{p}_1), \qquad t = \operatorname{clamp}\left(\frac{(\bar{\mathbf{x}} - \mathbf{p}_1)\cdot(\mathbf{p}_2 - \mathbf{p}_1)}{\|\mathbf{p}_2 - \mathbf{p}_1\|^2},\ 0,\ 1\right)$$

The signed distance and outward normal are then:

$$\Phi(\bar{\mathbf{x}}) = \|\bar{\mathbf{x}} - \mathbf{q}\| - r, \qquad \hat{\mathbf{n}}(\bar{\mathbf{x}}) = \frac{\bar{\mathbf{x}} - \mathbf{q}}{\|\bar{\mathbf{x}} - \mathbf{q}\|}$$

with $d = \|\bar{\mathbf{x}} - \mathbf{q}\|$. The projection is:

$$\bar{\mathbf{x}}^+ = \begin{cases} \bar{\mathbf{x}} & \text{if } d \geq r \\ \bar{\mathbf{x}} + (r - d)\,\hat{\mathbf{n}} & \text{if } d < r \end{cases} \tag{148}$$

which is identical in form to the sphere projection (146), with $\mathbf{q}$ playing the role of the center. The approximated projection Jacobian is also identical:

$$\frac{\partial\,\bar{\mathbf{x}}^+}{\partial\,\bar{\mathbf{x}}} = \begin{cases} \mathbf{I} & \text{if } d \geq r \\ \dfrac{r}{d}\left(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top\right) & \text{if } d < r \end{cases} \tag{149}$$

Note that $\mathbf{q}$ itself depends on $\bar{\mathbf{x}}$ through the projection parameter t, contributing an additional term:

$$\frac{\partial\,\mathbf{q}}{\partial\,\bar{\mathbf{x}}} = \frac{(\mathbf{p}_2 - \mathbf{p}_1)(\mathbf{p}_2 - \mathbf{p}_1)^\top}{\|\mathbf{p}_2 - \mathbf{p}_1\|^2}$$

to the full Jacobian. This term is neglected here. This approximation is geometrically sound: $\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top$ is always the correct projection onto the tangent plane of the capsule surface at the contact point, regardless of whether the contact lies on the cylindrical body or on one of the hemispherical caps. The neglected term affects only the rate at which $\hat{\mathbf{n}}$ rotates as $\bar{\mathbf{x}}$ moves along the cylinder, which is a second-order effect and negligible for small displacements.

8.1.3 Halfspace

A halfspace is defined by a point $\mathbf{p} \in \mathbb{R}^3$ on the boundary plane and a fixed outward unit normal $\hat{\mathbf{n}} \in \mathbb{R}^3$, giving the signed distance function:

$$\Phi(\mathbf{x}) = (\mathbf{x} - \mathbf{p}) \cdot \hat{\mathbf{n}}$$

The projection of a vertex $\mathbf{x}$ is:

$$\bar{\mathbf{x}}^+ = \begin{cases} \bar{\mathbf{x}} & \text{if } \Phi(\bar{\mathbf{x}}) \geq 0 \\ \bar{\mathbf{x}} - \Phi(\mathbf{x})\,\hat{\mathbf{n}} & \text{if } \Phi(\bar{\mathbf{x}}) < 0 \end{cases} \tag{150}$$

Since $\hat{\mathbf{n}}$ is constant, differentiating (150) gives:

$$\frac{\partial \bar{\mathbf{x}}^+}{\partial \bar{\mathbf{x}}} = \begin{cases} \mathbf{I} & \text{if } \Phi(\bar{\mathbf{x}}) \geq 0 \\ \mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top & \text{if } \Phi(\bar{\mathbf{x}}) < 0 \end{cases} \tag{151}$$

The active Jacobian $\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top$ is the orthogonal projector onto the plane: perturbations along $\hat{\mathbf{n}}$ are zeroed out, while tangential perturbations are preserved unchanged.

8.2 Primitive-Based Collision Handling

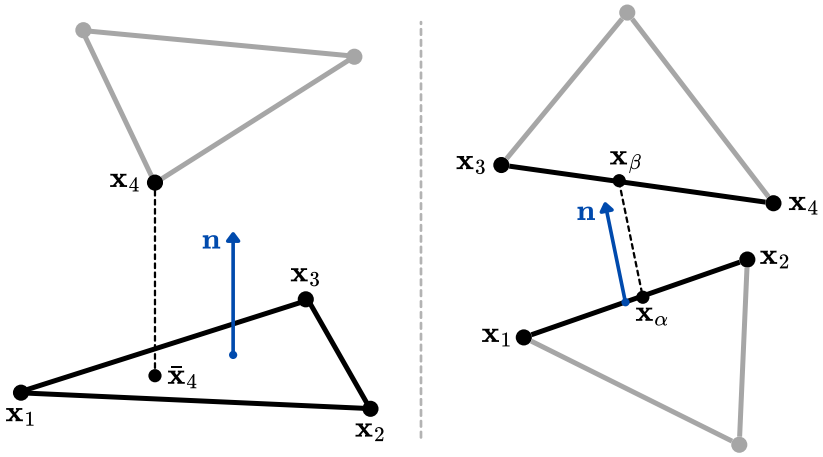

Figure 3: The two primitive collision cases for triangle meshes: vertex-face (left) and edge-edge (right).

For two triangle meshes, all collisions reduce to two primitive cases (Fig. 4). In both cases exactly four vertices are involved, and the signed distance between the two primitives takes the unified form:

$$\delta = \left(\sum_{i=1}^{4} w_i \mathbf{x}_i \right) \cdot \mathbf{n} \tag{152}$$

where $\mathbf{n}$ is the collision normal and the weights w_i depend on the case.

Vertex-Face. Vertices $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ form the face and $\mathbf{x}_4$ is the colliding vertex. Let $\bar{\mathbf{x}}_4 = \alpha_1 \mathbf{x}_1 + \alpha_2 \mathbf{x}_2 + \alpha_3 \mathbf{x}_3$ be the projection of $\mathbf{x}_4$ onto the face, with α_i its barycentric coordinates. The signed distance is:

$$\delta = (\mathbf{x}_4 - \bar{\mathbf{x}}_4) \cdot \mathbf{n} = (\mathbf{x}_4 - \alpha_1 \mathbf{x}_1 - \alpha_2 \mathbf{x}_3 - \alpha_3 \mathbf{x}_3) \cdot \mathbf{n}$$

giving weights:

$$w_1 = -\alpha_1, \qquad w_2 = -\alpha_2, \qquad w_3 = -\alpha_3, \qquad w_4 = 1$$

and $\mathbf{n}$ is the face normal.

Edge-Edge. Edges $(\mathbf{x}_1, \mathbf{x}_2)$ and $(\mathbf{x}_3, \mathbf{x}_4)$ have closest points $\mathbf{x}_\alpha = (1-\alpha)\mathbf{x}_1 + \alpha\mathbf{x}_2$ and $\mathbf{x}_\beta = (1-\beta)\mathbf{x}_3 + \beta\mathbf{x}_4$, with $\alpha, \beta \in [0, 1]$. The signed distance is:

$$\delta = (\mathbf{x}_\beta - \mathbf{x}_\alpha) \cdot \mathbf{n} = ((1-\beta)\mathbf{x}_3 + \beta\,\mathbf{x}_4 - (1-\alpha)\mathbf{x}_1 - \alpha\,\mathbf{x}_2) \cdot \mathbf{n}$$

giving weights:

$$w_1 = -(1-\alpha), \qquad w_2 = -\alpha, \qquad w_3 = (1-\beta), \qquad w_4 = \beta$$

and $\mathbf{n} = \frac{(\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_4 - \mathbf{x}_3)}{\|(\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_4 - \mathbf{x}_3)\|}$ is the common perpendicular to both edges.

8.3 Collision Detection via Continuous CCD

Assume a broad-phase algorithm (e.g. a Bounding Volume Hierarchy) has filtered out all primitive pairs that certainly do not collide during the timestep, leaving a small candidate subset. For each candidate pair, we use a Continuous Collision Detection (CCD) method to find the exact collision time within the timestep, under a linear trajectory assumption. This reduces to finding the roots of a cubic equation.

For a candidate collision pair, the positions of the involved vertices during the timestep are modeled as linear trajectories:

$$\mathbf{x}_k(t) = \mathbf{x}_k^t + t\,(\bar{\mathbf{x}}_k - \mathbf{x}_k^t) \qquad t \in [0, 1] \quad k \in [1, 2, 3, 4] \tag{153}$$

where $\mathbf{x}_k^\top$ is the position at the start of the timestep and $\bar{\mathbf{x}}_k$ is the candidate position after the internal forces solve. At $t = 0$ the vertex is at its original position; at $t = 1$ it reaches the post-dynamics position $\bar{\mathbf{x}}_k$.

A collision occurs only when the four involved vertices become coplanar, i.e.:

$$[(\mathbf{x}_2(t) - \mathbf{x}_1(t)) \times (\mathbf{x}_3(t) - \mathbf{x}_1(t))] \cdot (\mathbf{x}_4(t) - \mathbf{x}_1(t)) = 0$$

Substituting (153) for each vertex, since each $\mathbf{x}_k(t)$ is linear in t, the scalar triple product expands to a cubic polynomial:

$$a\,t^3 + b\,t^2 + c\,t + d = 0$$

Let $\mathcal{T}$ be the set of real roots of this equation. Roots outside $[0, 1]$ are discarded (the collision does not occur within the timestep), as are roots for which the primitives do not actually intersect at time t (coplanarity is necessary but not sufficient). If $\mathcal{T} = \emptyset$ the pair is collision-free; otherwise the collision time is:

$$t^* = \min(\mathcal{T})$$

The geometry of the involved primitives evaluated at t^* (specifically the contact normal $\mathbf{n}$ and the barycentric weights w_i) is then used to construct the collision constraint, as described in the next section.

Algorithm 13 Continuous Collision Detection: cubic root finding per candidate pair.

function ContinuousCollisionDetection($\mathbf{x}_t, \bar{\mathbf{x}}$)
  $\mathbb{P} \leftarrow$ AllPossibleContactPairs($\mathbf{x}_t, \bar{\mathbf{x}}$)
  $\mathbb{P} \leftarrow$ BroadPhaseFiltering($\mathbb{P}$)
  $\mathbb{C}_{\text{oll}} \leftarrow \emptyset$
  for $\mathbf{p} \in \mathbb{P}$ **in parallel do**
    $a, b, c, d \leftarrow$ ConstructCubicEquation($\mathbf{p}, \mathbf{x}_t, \bar{\mathbf{x}}$)
    $\mathcal{T} \leftarrow \{t \in$ Roots$(a, b, c, d) \mid t \in [0, 1] \wedge$ IsCollision$(t, \mathbf{p}, \mathbf{x}_t, \bar{\mathbf{x}})\}$
    if $\mathcal{T} \neq \emptyset$ **then**
      $\mathbb{C}_{\text{oll}} \leftarrow \mathbb{C}_{\text{oll}} \cup \{(\mathbf{p}, \min(\mathcal{T}))\}$
  return $\mathbb{C}_{\text{oll}}$
end function

function IsCollision($t, \mathbf{p}, \mathbf{x}_t, \bar{\mathbf{x}}$)
  $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4 \leftarrow$ InterpolatePoints($t, \mathbf{p}, \mathbf{x}_t, \bar{\mathbf{x}}$)
  if $\mathbf{p}$ is Vertex-Face **then**
    if PointIsInsideTriangle($\mathbf{x}_4, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$) **then**
      return True
  else if $\mathbf{p}$ is Edge-Edge **then**
    if SegmentsIntersect($\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4$) **then**
      return True
  return False
end function

8.4 Collision Response as a Constrained Quadratic Problem

Given the candidate positions $\bar{\mathbf{x}}$ produced by the dynamics step, collision response seeks the corrected positions $\mathbf{x}$ that satisfy all non-penetration constraints while deviating as little as possible from $\bar{\mathbf{x}}$. This is formulated as a constrained quadratic program:

$$\min_{\mathbf{x}} \frac{1}{2}(\mathbf{x} - \bar{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \bar{\mathbf{x}}) \qquad \text{s.t.} \qquad \mathbf{G}\mathbf{x} + \mathbf{h} \leq 0 \tag{154}$$

The objective is a mass-weighted displacement: heavier vertices are penalized more for moving, which is physically consistent. The constraint $\mathbf{G}\mathbf{x} + \mathbf{h} \leq 0$ en-

codes all m detected collisions, with one row per collision pair. The construction of $\mathbf{G} \in \mathbb{R}^{m\times 3n}$ and $\mathbf{h} \in \mathbb{R}^m$ from the detected collisions is detailed below.

For each primitive detected collision j, we require the signed distance δ_j to exceed the cloth thickness d:

$$\delta_j = \left(\sum_{i=1}^{4} w_i \mathbf{x}_i\right) \cdot \mathbf{n}_j \geq d$$

Since δ_j is linear in $\mathbf{x}$, this can be written as $\mathbf{g}_j^\top \mathbf{x} \geq d$, where $\mathbf{g}_j \in \mathbb{R}^{3n}$ is sparse, with nonzero entries only at the 12 coordinates of the 4 involved vertices:

$$\mathbf{g}_j = [\cdots \mathbf{0} \cdots \underbrace{w_{j1}\,\mathbf{n}_j}_{\in\mathbb{R}^3} \cdots \mathbf{0} \cdots \underbrace{w_{j2}\,\mathbf{n}_j}_{\in\mathbb{R}^3} \cdots \mathbf{0} \cdots]$$

Rewriting in the standard form $c \leq 0$ gives $-\mathbf{g}_j^\top \mathbf{x} + d \leq 0$. Stacking all m collision constraints yields:

$$\mathbf{G} = \begin{bmatrix} -\mathbf{g}_1^\top \\ -\mathbf{g}_2^\top \\ \vdots \\ -\mathbf{g}_m^\top \end{bmatrix} \in \mathbb{R}^{m\times 3n} \qquad \mathbf{h} = \begin{bmatrix} d \\ d \\ \vdots \\ d \end{bmatrix} \in \mathbb{R}^m$$

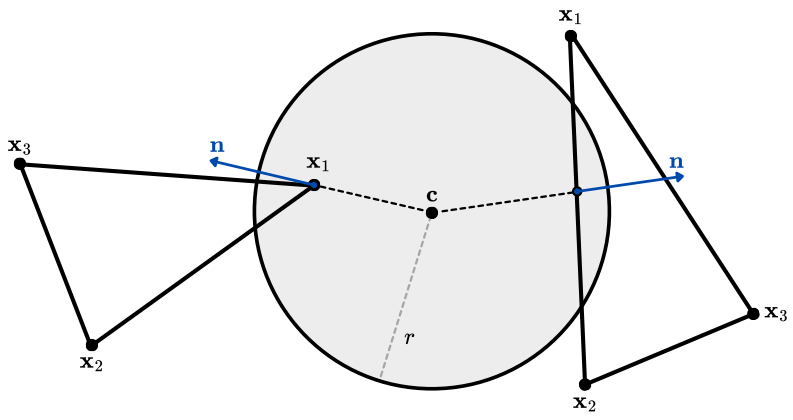

Figure 4:

Face-Sphere. Let $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ be the triangle vertices, $\mathbf{c} \in \mathbb{R}^3$ the sphere center, and r its radius. The contact point is the closest point on the triangle to $\mathbf{c}$:

$$\bar{\mathbf{x}} = \alpha_1 \mathbf{x}_1 + \alpha_2 \mathbf{x}_2 + \alpha_3 \mathbf{x}_3$$

where α_i are the barycentric coordinates of the projection of $\mathbf{c}$ onto the triangle, clamped to lie within it. The outward contact normal is $\mathbf{n} = \frac{\bar{\mathbf{x}} - \mathbf{c}}{\|\bar{\mathbf{x}} - \mathbf{c}\|}$ and the signed distance is:

$$\delta = (\bar{\mathbf{x}} - \mathbf{c}) \cdot \mathbf{n}$$

The constraint $\delta \geq r$ reads $\mathbf{g}^\top \mathbf{x} \geq d$ with:

$$\mathbf{g} = [\cdots \; \mathbf{0} \; \cdots \; \underbrace{\alpha_1 \mathbf{n}}_{\in \mathbb{R}^3} \; \cdots \; \underbrace{\alpha_2 \mathbf{n}}_{\in \mathbb{R}^3} \; \cdots \; \underbrace{\alpha_3 \mathbf{n}}_{\in \mathbb{R}^3} \; \cdots \; \mathbf{0} \; \cdots] \qquad d = r + \mathbf{c} \cdot \mathbf{n}$$

where the nonzero entries correspond to the 9 coordinates of $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$. The sphere contributes no degrees of freedom.

8.4.1 Solution via Non-Rigid Impact Zones

We now present an approximate solution to the collision response QP, adapting the non-rigid impact zone method of [11] to position space. The approximation consists of relaxing the inequality constraints to equalities, requiring each signed distance to be exactly at the thickness boundary rather than above it:

$$\min_{\mathbf{x}} \; \frac{1}{2}(\mathbf{x} - \bar{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \bar{\mathbf{x}}) \qquad \text{s.t.} \qquad \mathbf{G}\mathbf{x} + \mathbf{h} = 0 \tag{155}$$

This relaxation may produce negative Lagrange multipliers, introducing mild artificial sticking (since constraints may be pulled rather than pushed), but reduces the problem to a simple linear solve. The Lagrangian is:

$$\mathcal{L}(\mathbf{x}, \lambda) = \frac{1}{2}(\mathbf{x} - \bar{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \bar{\mathbf{x}}) + (\mathbf{G}\mathbf{x} + \mathbf{h})^\top \lambda$$

The stationarity condition $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = 0$ gives:

$$\mathbf{x}^* = \bar{\mathbf{x}} - \mathbf{M}^{-1} \mathbf{G}^\top \lambda \tag{156}$$

Substituting into the equality constraint $\mathbf{G}\mathbf{x}^* + \mathbf{h} = 0$ yields a linear system for the multipliers:

$$\underbrace{[\mathbf{G}\mathbf{M}^{-1}\mathbf{G}^\top]}_{\mathbf{A} \in \mathbb{R}^{m \times m}} \lambda = \underbrace{\mathbf{G}\bar{\mathbf{x}} + \mathbf{h}}_{\mathbf{b} \in \mathbb{R}^m} \tag{157}$$

Solving for λ and substituting back into (156) yields the corrected positions $\mathbf{x}^*$ satisfying all constraints.

Impact Zone Decomposition Since $\mathbf{A}_{ij} = \mathbf{g}_i^\top \mathbf{M}^{-1} \mathbf{g}_j = 0$ whenever constraints i and j share no vertices, $\mathbf{A}$ is block-diagonal when constraints are sorted by impact zone. Each impact zone k, involving m_k constraints and n_k vertices, yields one independent block system:

$$\mathbf{A}^k \lambda^k = \mathbf{b}^k$$

$$\mathbf{A}^k = \mathbf{G}^k (\mathbf{M}^k)^{-1} (\mathbf{G}^k)^\top \in \mathbb{R}^{m_k \times m_k} \qquad \mathbf{b}^k = \mathbf{G}^k \bar{\mathbf{x}}^k + \mathbf{h}^k \in \mathbb{R}^{m_k}$$

where all quantities are restricted to the n_k vertices of zone k. Since the blocks are independent, each system can be assembled and solved in parallel.

Impact Zone

Define the constraint graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $i \in \mathcal{V}$ corresponds to a collision constraint, and an edge $(i, j) \in \mathcal{E}$ exists if and only if constraints i and j share at least one vertex in their stencil. An **impact zone**, then, is a connected component of $\mathcal{G}$.

Algorithm 14 Non-Rigid Impact Zone collision response (single-iteration approximation).

function NonRigidImpactZoneCollisionResponse($\mathbf{x}_t, \bar{\mathbf{x}}, \mathbf{M}, d$)
  $\mathbb{C}_{\text{coll}} \leftarrow \text{CCD}(\mathbf{x}_t, \bar{\mathbf{x}})$
  $\mathbf{G}, \mathbf{h} \leftarrow$ ConstructConstraints($\mathbb{C}_{\text{coll}}, \mathbf{x}_t, \bar{\mathbf{x}}, d$)
  $\mathbf{x}^* \leftarrow \bar{\mathbf{x}}$
  $\mathbb{I}_z \leftarrow$ GroupByImpactZones($\mathbf{G}, \mathbf{h}$)
  for $Z \in \mathbb{I}_z$ **in parallel do**
    $\mathbf{x}^* \leftarrow$ ApplyProjection($Z, \bar{\mathbf{x}}, \mathbf{M}$) ▷ Eqs. 157, 156
  return $\mathbf{x}^*$
end function

9 Implicit Differentiation

Given that we already know $\frac{\partial \phi}{\partial \mathbf{y}}$, we want to calculate $\frac{\partial \phi}{\partial \mathbf{u}}$. We have a system of implicit constraints:

$$\mathbf{F}(\mathbf{y}, \mathbf{u}) = \mathbf{0}$$

that binds $\mathbf{y}$ and $\mathbf{u}$ together, where $\mathbf{y} \in \mathbb{R}^n$ is considered the output, $\mathbf{u} \in \mathbb{R}^m$ the input and $\mathbf{F} : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^l$.
First, we totally differentiate the constraint $\mathbf{F} = 0$:

$$\frac{\partial \mathbf{F}}{\partial \mathbf{y}} d\mathbf{y} + \frac{\partial \mathbf{F}}{\partial \mathbf{u}} d\mathbf{u} = 0 \tag{158}$$

solving for $d\mathbf{y}$:

$$d\mathbf{y} = -\left(\frac{\partial \mathbf{F}}{\partial \mathbf{y}}\right)^{-1} \frac{\partial \mathbf{F}}{\partial \mathbf{u}} d\mathbf{u} = -\mathbf{J}^{-1} \frac{\partial \mathbf{F}}{\partial \mathbf{u}} d\mathbf{u}$$

where $\mathbf{J} = \frac{\partial \mathbf{F}}{\partial \mathbf{y}} \in \mathbb{R}^{l \times n}$ is the Jacobian of the constraint with respect to the output. Then, the loss change with respect to $d\mathbf{u}$ is:

$$d\phi = \left(\frac{\partial \phi}{\partial \mathbf{y}}\right)^\top d\mathbf{y} = -\left(\frac{\partial \phi}{\partial \mathbf{y}}\right)^\top \mathbf{J}^{-1} \frac{\partial \mathbf{F}}{\partial \mathbf{u}} d\mathbf{u}$$

We define the adjoint vector $\boldsymbol{\mu}$ as:

$$\boldsymbol{\mu} = \mathbf{J}^{-T} \frac{\partial \phi}{\partial \mathbf{y}} \qquad \Longrightarrow \qquad \mathbf{J}^\top \boldsymbol{\mu} = \frac{\partial \phi}{\partial \mathbf{y}} \tag{159}$$

which can be found by solving the linear system on the right.
Then the gradient loss can be re-written as:

$$d\phi = -\boldsymbol{\mu}^\top \frac{\partial \mathbf{F}}{\partial \mathbf{u}} d\mathbf{u}$$

that let us calculate the gradient of the loss with respect to $\mathbf{u}$:

$$\frac{\partial \phi}{\partial \mathbf{u}} = -\left(\frac{\partial \mathbf{F}}{\partial \mathbf{u}}\right)^\top \boldsymbol{\mu} \tag{160}$$

9.1 Linear Solve

During the forward pass, the simulation requires solving the linear system

$$\mathbf{A}\mathbf{x} = \mathbf{b} \qquad \text{with } \mathbf{A} \in \mathbb{R}^{3n \times 3n},\ \mathbf{b} \in \mathbb{R}^{3n}$$

for the unknown $\mathbf{x}$. During the backward pass, given the upstream gradient $\partial\phi/\partial\mathbf{x}$, we seek $\partial\phi/\partial\mathbf{A}$ and $\partial\phi/\partial\mathbf{b}$. We first solve the linear system

$$\mathbf{A}^\top \boldsymbol{\mu} = \frac{\partial \phi}{\partial \mathbf{x}} \tag{161}$$

to find the adjoint vector $\boldsymbol{\mu} \in \mathbb{R}^{3n}$, from which the gradients follow as:

$$\frac{\partial \phi}{\partial \mathbf{b}} = \boldsymbol{\mu} \in \mathbb{R}^{3n} \qquad \frac{\partial \phi}{\partial \mathbf{A}} = -\boldsymbol{\mu}\,\mathbf{x}^\top \in \mathbb{R}^{3n \times 3n} \tag{162}$$

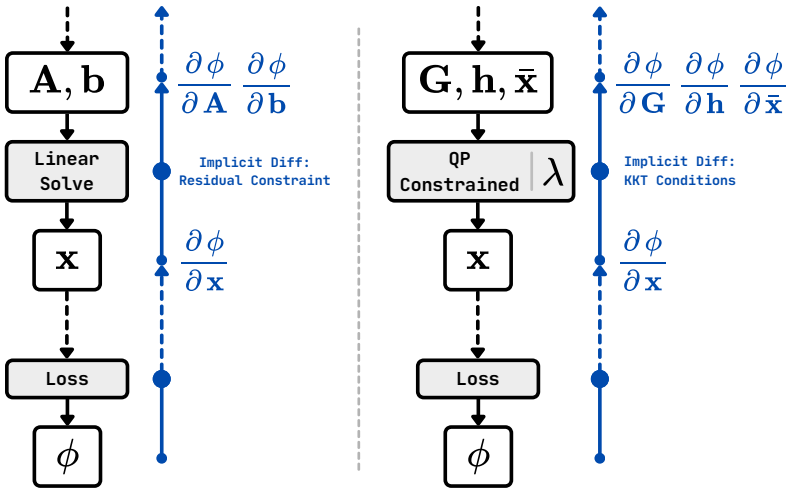

Figure 5: Forward and backward pipeline...

Following the method presented in Section 9, the linear system defines the implicit residual constraint:

$$\mathbf{F}(\mathbf{x}, \mathbf{A}, \mathbf{b}) = \mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{0}$$

with output $\mathbf{y} = \mathbf{x}$ and inputs $\mathbf{u} = (\mathbf{A}, \mathbf{b})$. The Jacobian with respect to the output is:

$$\mathbf{J} = \frac{\partial \mathbf{F}}{\partial \mathbf{x}} = \mathbf{A}$$

so the adjoint system $\mathbf{J}^\top \boldsymbol{\mu} = \frac{\partial \phi}{\partial \mathbf{x}}$ becomes:

$$\mathbf{A}^\top \boldsymbol{\mu} = \frac{\partial \phi}{\partial \mathbf{x}}.$$

The gradients then follow from $\frac{\partial \phi}{\partial \mathbf{u}} = -(\frac{\partial \mathbf{F}}{\partial \mathbf{u}})^\top \boldsymbol{\mu}$. Computing:

$$\frac{\partial \mathbf{F}}{\partial \mathbf{b}} = -\mathbf{I} \qquad \frac{\partial \mathbf{F}}{\partial \mathbf{A}} = \mathbf{x}^\top$$

gives:

$$\frac{\partial \phi}{\partial \mathbf{b}} = -(-\mathbf{I})^\top \boldsymbol{\mu} = \boldsymbol{\mu} \qquad \frac{\partial \phi}{\partial \mathbf{A}} = -\mathbf{x}\boldsymbol{\mu}^\top = -\boldsymbol{\mu}\,\mathbf{x}^\top,$$

which recovers (162).

9.2 Quadratic Program with only Linear Inequality Constraints

Here we follow derivations of the Paper [5]. During the forward pass, the simulation might solves the constrained QP

$$\min_{\mathbf{x}} \ \frac{1}{2}(\mathbf{x}-\bar{\mathbf{x}})^\top \mathbf{M}\,(\mathbf{x}-\bar{\mathbf{x}}) \qquad \text{s.t.} \qquad \mathbf{G}\mathbf{x}+\mathbf{h} \le 0,$$

During the backward pass, given $\frac{\partial \phi}{\partial \mathbf{x}}$, we seek $\frac{\partial \phi}{\partial \mathbf{G}}$, $\frac{\partial \phi}{\partial \mathbf{h}}$, and $\frac{\partial \phi}{\partial \bar{\mathbf{x}}}$.
Following the general implicit differentiation framework of Section 9, we use the KKT stationarity and complementary slackness conditions as the implicit constraints $\mathbf{F} = \mathbf{0}$. Identifying the Jacobian $\mathbf{J} = \frac{\partial \mathbf{F}}{\partial [\mathbf{x},\lambda]}$ and applying $\mathbf{J}^\top \boldsymbol{\mu} = \frac{\partial \phi}{\partial [\mathbf{x},\lambda]}$ yields the linear system:

$$\begin{bmatrix} \mathbf{M} & \mathbf{G}^\top \operatorname{DIAG}(\lambda) \\ \mathbf{G} & \operatorname{DIAG}(\mathbf{G}\mathbf{x}+\mathbf{h}) \end{bmatrix} \begin{bmatrix} \mathbf{d}_\mathbf{x} \\ \mathbf{d}_\lambda \end{bmatrix} = \begin{bmatrix} \left(\frac{\partial \phi}{\partial \mathbf{x}}\right)^\top \\ \mathbf{0} \end{bmatrix} \tag{163}$$

where the zero in the right-hand side reflects that λ does not appear in the loss directly. From the solution $(\mathbf{d}_\mathbf{x}, \mathbf{d}_\lambda)$, the gradients follow as

$$\frac{\partial \phi}{\partial \bar{\mathbf{x}}} = \mathbf{d}_\mathbf{x}^\top \mathbf{M} \qquad \in \mathbb{R}^{3n} \tag{164}$$

$$\frac{\partial \phi}{\partial \mathbf{G}} = -\operatorname{DIAG}(\lambda)\,\mathbf{d}_\lambda\,\mathbf{x}^\top - \lambda\,\mathbf{d}_\mathbf{x}^\top \qquad \in \mathbb{R}^{m\times 3n} \tag{165}$$

$$\frac{\partial \phi}{\partial \mathbf{h}} = -\mathbf{d}_\lambda^\top \operatorname{DIAG}(\lambda) \qquad \in \mathbb{R}^{m} \tag{166}$$

The linear system 163 is expensive to solve directly. Assuming all constraints are perfectly satisfied, i.e. $\mathbf{G}\mathbf{x}+\mathbf{h} = \mathbf{0}$, the bottom-right block of the system vanishes and the adjoint vectors can instead be obtained in closed form. We first compute the QR decomposition

$$\sqrt{\mathbf{M}}^{-1}\mathbf{G}^\top = \mathbf{Q}\mathbf{R} \tag{167}$$

from which the adjoint vectors follow directly as

$$\mathbf{d}_\mathbf{x} = \sqrt{\mathbf{M}}^{-1}(\mathbf{I}-\mathbf{Q}\mathbf{Q}^\top)\sqrt{\mathbf{M}}^{-1}\left(\frac{\partial \phi}{\partial \mathbf{x}}\right)^\top \tag{168}$$

$$\mathbf{d}_\lambda = \operatorname{DIAG}(\lambda)^{-1}\mathbf{R}^{-1}\mathbf{Q}^\top\sqrt{\mathbf{M}}^{-1}\left(\frac{\partial \phi}{\partial \mathbf{x}}\right)^\top \tag{169}$$

9.3 Generic Constrained Quadratic Program

Here we follow the derivations of [12]. Given the constrained QP:

$$\min_{\mathbf{x}} \ \frac{1}{2}\mathbf{x}^\top \mathbf{Q}\,\mathbf{x} + \mathbf{q}^\top \mathbf{x} \qquad \text{such that}$$
$$\mathbf{A}\,\mathbf{x} \ = \ \mathbf{b}$$
$$\mathbf{G}\,\mathbf{x} \ \leq \ \mathbf{h}$$

and given $\frac{\partial \phi}{\partial \mathbf{x}^\star}$, we seek $\frac{\partial \phi}{\partial \mathbf{Q}}, \frac{\partial \phi}{\partial \mathbf{q}}, \frac{\partial \phi}{\partial \mathbf{A}}, \frac{\partial \phi}{\partial \mathbf{b}}, \frac{\partial \phi}{\partial \mathbf{G}}, \frac{\partial \phi}{\partial \mathbf{h}}$.
The Lagrangian of the problem is:

$$\mathcal{L}(\mathbf{x}, \nu, \lambda) = \frac{1}{2}\mathbf{x}^\top \mathbf{Q}\,\mathbf{x} + \mathbf{q}^\top \mathbf{x} + \nu^\top(\mathbf{A}\mathbf{x} - \mathbf{b}) + \lambda^\top(\mathbf{G}\mathbf{x} - \mathbf{h})$$

The KKT stationary, primal feasibility, and complementary slackness conditions at the optimal $(\mathbf{x}^\star, \nu^\star, \lambda^\star)$ are:

$$\mathbf{F}(\mathbf{x}^\star, \nu^\star, \lambda^\star) = \begin{cases} \mathbf{Q}\,\mathbf{x}^\star + \mathbf{q} + \mathbf{A}^\top \nu^\star + \mathbf{G}^\top \lambda^\star = \mathbf{0} \\ \mathbf{A}\,\mathbf{x}^\star - \mathbf{b} = \mathbf{0} \\ \textsc{Diag}(\lambda^\star)(\mathbf{G}\mathbf{x}^\star - \mathbf{h}) = \mathbf{0} \end{cases}$$

We treat these as the implicit constraint $\mathbf{F} = \mathbf{0}$ of Section 9, with output $\mathbf{y} = (\mathbf{x}^\star, \nu^\star, \lambda^\star)$ and inputs $\mathbf{u} = (\mathbf{Q}, \mathbf{q}, \mathbf{A}, \mathbf{b}, \mathbf{G}, \mathbf{h})$. The adjoint system $\mathbf{J}^\top \boldsymbol{\mu} = \frac{\partial \phi}{\partial \mathbf{y}}$ becomes:

$$\begin{bmatrix} \mathbf{Q} & \mathbf{G}^\top \textsc{Diag}(\lambda^\star) & \mathbf{A}^\top \\ \mathbf{G} & \textsc{Diag}(\mathbf{G}\mathbf{x}^\star - \mathbf{h}) & \mathbf{0} \\ \mathbf{A} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{d}_\mathbf{x} \\ \mathbf{d}_\lambda \\ \mathbf{d}_\nu \end{bmatrix} = \begin{bmatrix} \left(\frac{\partial \phi}{\partial \mathbf{x}^\star}\right)^\top \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix} \tag{170}$$

where the zeros on the right reflect that the loss does not depend directly on $\lambda^\star$ or $\nu^\star$. From the solution $(\mathbf{d}_\mathbf{x}, \mathbf{d}_\lambda, \mathbf{d}_\nu)$, the parameter gradients follow as:

$$\begin{aligned} \frac{\partial \phi}{\partial \mathbf{Q}} &= \frac{1}{2}(\mathbf{d}_\mathbf{x}\,\mathbf{x}^{\star\top} + \mathbf{x}^\star \mathbf{d}_\mathbf{x}^\top) & \frac{\partial \phi}{\partial \mathbf{q}} &= \mathbf{d}_\mathbf{x} \\ \frac{\partial \phi}{\partial \mathbf{A}} &= \mathbf{d}_\nu\,\mathbf{x}^{\star\top} + \nu^\star \mathbf{d}_\mathbf{x}^\top & \frac{\partial \phi}{\partial \mathbf{b}} &= -\mathbf{d}_\nu \\ \frac{\partial \phi}{\partial \mathbf{G}} &= \textsc{Diag}(\lambda^\star)\,\mathbf{d}_\lambda\,\mathbf{x}^{\star\top} + \lambda^\star \mathbf{d}_\mathbf{x}^\top & \frac{\partial \phi}{\partial \mathbf{h}} &= -\textsc{Diag}(\lambda^\star)\,\mathbf{d}_\lambda \end{aligned} \tag{171}$$

9.4 Linear Complementary Problem

Given the LCP:

find $\mathbf{x} \in \mathbb{R}^n$ such that:

$$\mathbf{M}\mathbf{x} + \mathbf{q} \geq 0$$
$$\mathbf{x} \geq 0$$
$$\mathbf{x}^\top(\mathbf{M}\mathbf{x} + \mathbf{q}) = 0$$

and given $\frac{\partial \phi}{\partial \mathbf{x}^\star}$, we seek $\frac{\partial \phi}{\partial \mathbf{M}}, \frac{\partial \phi}{\partial \mathbf{q}}$.
We introduce the slack variable $\mathbf{w} = \mathbf{M}\mathbf{x} + \mathbf{q}$ and define the two implicit constraints:

$$\begin{aligned}\mathbf{F}_1(\mathbf{x}^\star, \mathbf{w}^\star, \mathbf{M}, \mathbf{q}) &= \mathbf{M}\mathbf{x}^\star + \mathbf{q} - \mathbf{w}^\star = 0 \\ \mathbf{F}_2(\mathbf{x}^\star, \mathbf{w}^\star) &= \textsc{Diag}(\mathbf{x}^\star)\mathbf{w}^\star = 0\end{aligned}$$

Then $\mathbf{J}$ is:

$$\frac{\partial \mathbf{F}}{\partial [\mathbf{x}^\star, \mathbf{w}^\star]} = \mathbf{J} = \begin{bmatrix} \mathbf{M} & -\mathbf{I} \\ \textsc{Diag}(\mathbf{w}^\star) & \textsc{Diag}(\mathbf{x}^\star) \end{bmatrix} \tag{172}$$

The adjoint vector is obtained by solving the linear system:

$$\begin{bmatrix} \mathbf{M}^\top & \textsc{Diag}(\mathbf{w}^\star) \\ -\mathbf{I} & \textsc{Diag}(\mathbf{x}^\star) \end{bmatrix} \begin{bmatrix} \mathbf{d}_\mathbf{x} \\ \mathbf{d}_\mathbf{w} \end{bmatrix} = \begin{bmatrix} (\frac{\partial \phi}{\partial \mathbf{x}^\star})^\top \\ 0 \end{bmatrix} \tag{173}$$

And the gradients with resoect to the input are:

$$\frac{\partial \phi}{\partial \mathbf{M}} = -\mathbf{d}_\mathbf{x} {\mathbf{x}^\star}^\top \qquad \frac{\partial \phi}{\partial \mathbf{q}} = -\mathbf{d}_\mathbf{x} \tag{174}$$

10 Differentiable Cloth Simulation for Inverse Problems [5]

Building on Sections 8 and 9 we will build a forward and backward simulation methodology very similar to the one proposed in the Paper [5].

Algorithm 15

function $(\mathbf{x}^-, \mathbf{v}^-, \mathbf{M}, h)$
  $\hat{\mathbf{M}}, \hat{\mathbf{f}} \leftarrow$ BuildLinearSystem$(\mathbf{x}^-, \mathbf{v}^-, \mathbf{M}, h)$ ▷ Eq. 9
  Solve $\hat{\mathbf{M}}\Delta\mathbf{x} = \hat{\mathbf{f}}$
  $\bar{\mathbf{x}} \leftarrow \mathbf{x}^- + \Delta\mathbf{x}$
  $\mathbf{G}, \mathbf{h} \leftarrow \mathrm{CCD}(\mathbf{x}^-, \bar{\mathbf{x}})$ ▷ Algo. 13
  $\mathbf{x}^+ \leftarrow$ SolveQP$(\bar{\mathbf{x}}, \mathbf{M}, \mathbf{G}, \mathbf{h})$ ▷ Algo. 14
  $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$
end function

11 Projective Dynamics (PD) [2]

We start with the variational form of implicit Euler integration. Given the current state, the inertial estimate $\tilde{\mathbf{x}} = \mathbf{x}^- + h\mathbf{v}^- + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}$ predicts where each vertex would land under momentum and external forces alone. The next positions $\mathbf{x}^+$ balance this inertial trajectory against elastic resistance:

$$\mathbf{x}^+ = \arg\min_{\mathbf{x}} \underbrace{\frac{1}{2h^2}(\mathbf{x}-\tilde{\mathbf{x}})^\top \mathbf{M}\,(\mathbf{x}-\tilde{\mathbf{x}})}_{\text{momentum potential}} + \underbrace{\sum_i U_i(\mathbf{x})}_{\text{elastic potential}} \tag{175}$$

The momentum potential penalizes deviation from the inertial trajectory, weighted by mass. The elastic potential penalizes deformation away from the rest configuration. Their balance determines the motion.

11.1 Constraint-Based Energy Potentials

The key idea of Projective Dynamics is to replace each elastic potential $U_i(\mathbf{x})$ with an augmented form involving an auxiliary variable $\mathbf{p}_i$:

$$U_i(\mathbf{x}, \mathbf{p}_i) = \frac{k_i}{2}\|\mathbf{A}_i\mathbf{S}_i\mathbf{x} - \mathbf{B}_i\mathbf{p}_i\|_F^2 + \delta_i(\mathbf{p}_i) \tag{176}$$

where $\mathbf{S}_i$ is a constant selection matrix extracting the vertices involved in constraint i from the global position vector $\mathbf{x}$, the matrices $\mathbf{A}_i$ and $\mathbf{B}_i$ are constant differential operators determined by the rest configuration (encoding, e.g., the discrete gradient operator and rest edge geometry), k_i is the constraint stiffness, and:

$$\delta_i(\mathbf{p}_i) = \begin{cases} 0 & \text{if } \mathbf{p}_i \in \mathcal{M}_i \\ +\infty & \text{otherwise} \end{cases} \tag{177}$$

is an indicator function that forces $\mathbf{p}_i$ to lie on the constraint manifold $\mathcal{M}_i$, the set of admissible configurations. The distance $\|\mathbf{A}_i\mathbf{S}_i\mathbf{x} - \mathbf{B}_i\mathbf{p}_i\|_F^2$ is *quadratic* in both $\mathbf{x}$ and $\mathbf{p}_i$, while all geometric nonlinearity is captured by the projection onto $\mathcal{M}_i$.
Minimizing over $\mathbf{p}_i$ recovers the original elastic potential: $U_i(\mathbf{x}) = \min_{\mathbf{p}_i} U_i(\mathbf{x}, \mathbf{p}_i)$.

11.2 The Augmented Problem

Substituting the constraint-based potentials, the full problem becomes:

$$\min_{\mathbf{x},\mathbf{p}} \frac{1}{2h^2}(\mathbf{x}-\tilde{\mathbf{x}})^\top \mathbf{M}\,(\mathbf{x}-\tilde{\mathbf{x}}) + \sum_i \frac{k_i}{2}\|\mathbf{A}_i\mathbf{S}_i\mathbf{x} - \mathbf{B}_i\mathbf{p}_i\|_F^2 + \delta_i(\mathbf{p}_i) \tag{178}$$

This is minimized via alternating optimization (block coordinate descent) over $\mathbf{p}$ and $\mathbf{x}$:

Local Step. Fix $\mathbf{x}$, minimize over each $\mathbf{p}_i$ independently:

$$\mathbf{p}_i^* = \arg\min_{\mathbf{p}_i \in \mathcal{M}_i} \frac{k_i}{2} \|\mathbf{A}_i \mathbf{S}_i \mathbf{x} - \mathbf{B}_i \mathbf{p}_i\|_F^2 \tag{179}$$

This is a projection of $\mathbf{A}_i \mathbf{S}_i \mathbf{x}$ onto the constraint manifold $\mathcal{M}_i$. Each constraint is independent, enabling massive parallelism.

Global Step. Fix all $\mathbf{p}_i$, minimize over $\mathbf{x}$. Since the objective is quadratic in $\mathbf{x}$, the minimizer satisfies:

$$\underbrace{\left[\frac{1}{h^2}\mathbf{M} + \sum_i k_i \mathbf{S}_i^\top \mathbf{A}_i^\top \mathbf{A}_i \mathbf{S}_i\right]}_{\mathbf{L}} \mathbf{x} = \underbrace{\frac{1}{h^2}\mathbf{M}\,\tilde{\mathbf{x}} + \sum_i k_i \mathbf{S}_i^\top \mathbf{A}_i^\top \mathbf{B}_i\, \mathbf{p}_i}_{\mathbf{b}} \tag{180}$$

The system matrix $\mathbf{L}$ is symmetric positive definite and *constant*, it depends only on the mass distribution, time step, constraint stiffnesses, and rest geometry, none of which change during simulation. It can therefore be prefactored (e.g., via sparse Cholesky) once at initialization. Each global step then requires only back-substitution with the updated right-hand side $\mathbf{b}$.

11.3 Strain Constraints for Cloth

Consider a codimensional thin shell (cloth) discretized as a triangle mesh. Each triangle has vertices defined in the 2D material (rest) space as $\bar{\mathbf{x}}_0, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2 \in \mathbb{R}^2$ and in the 3D world (deformed) space as $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^3$.

Deformation gradient. The deformation gradient $\mathbf{F} \in \mathbb{R}^{3\times 2}$ maps edge vectors from material space to world space. Define the edge matrices:

$$\mathbf{D}_M = [\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_0 \mid \bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_0] \in \mathbb{R}^{2\times 2}, \qquad \mathbf{D}_S = [\mathbf{x}_1 - \mathbf{x}_0 \mid \mathbf{x}_2 - \mathbf{x}_0] \in \mathbb{R}^{3\times 2}$$

Since $\mathbf{D}_M$ depends only on the rest configuration, it is computed and inverted once at initialization. The deformation gradient is then:

$$\mathbf{F} = \mathbf{D}_S\, \mathbf{D}_M^{-1} \tag{181}$$

Expanding with $\mathbf{D}_M^{-1} = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ reveals the linear dependence on vertex positions:

$$\mathbf{F} = \left[\, a(\mathbf{x}_1 - \mathbf{x}_0) + c(\mathbf{x}_2 - \mathbf{x}_0) \;\middle|\; b(\mathbf{x}_1 - \mathbf{x}_0) + d(\mathbf{x}_2 - \mathbf{x}_0) \,\right] \tag{182}$$

Since every entry of $\mathbf{F}$ is a linear combination of vertex coordinates with constant coefficients, we can write:

$$\mathrm{vec}(\mathbf{F}_i) = \mathbf{G}_i\, \mathbf{x} = \mathbf{A}_i\, \mathbf{S}_i\, \mathbf{x} \tag{183}$$

where $\mathbf{S}_i \in \mathbb{R}^{9\times 3n}$ is a sparse selection matrix extracting the three vertex positions involved in triangle i, and $\mathbf{A}_i \in \mathbb{R}^{6\times 9}$ encodes the edge differencing and multiplication by $\mathbf{D}_M^{-1}$, producing the vectorized deformation gradient $\mathrm{vec}(\mathbf{F}_i) \in \mathbb{R}^6$.

Strain potential. For each triangle i with rest area A_i, the augmented strain potential is:

$$U_i(\mathbf{x}, \mathbf{T}_i) = \frac{k_i}{2} A_i \|\mathbf{F}_i - \mathbf{T}_i\|_F^2 + \delta_{\mathcal{M}_i}(\mathbf{T}_i) \tag{184}$$

The auxiliary variable $\mathbf{T}_i \in \mathbb{R}^{3\times 2}$ represents the closest *admissible* deformation gradient. The area weighting A_i arises from discretizing the continuous strain energy integral $\int_S \|\cdot\|^2 \, dA$ and ensures mesh-independent behavior under refinement.

Local step. Fix $\mathbf{x}$ and project each deformation gradient onto the constraint manifold:

$$\mathbf{T}_i^* = \arg\min_{\mathbf{T}\in\mathcal{M}_i} \|\mathbf{F}_i - \mathbf{T}\|_F^2 \tag{185}$$

Compute the SVD $\mathbf{F}_i = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ and construct $\mathbf{T}_i^*$ based on $\mathcal{M}_i$:

- **Rigid motion** ($\mathcal{M}_i = \mathrm{SO}$): no stretching is permitted. The closest rotation is:
$$\mathbf{T}_i^* = \mathbf{U}\mathbf{V}^\top \tag{186}$$
- **Strain limiting** ($\mathcal{M}_i = \{\mathbf{T} : \sigma_{\min} \le \sigma_j(\mathbf{T}) \le \sigma_{\max}\}$): stretching is bounded. Clamp each singular value independently:
$$\mathbf{T}_i^* = \mathbf{U}\mathbf{\Sigma}^*\mathbf{V}^\top, \qquad \Sigma_{jj}^* = \textsc{Clamp}(\Sigma_{jj},\, \sigma_{\min},\, \sigma_{\max}) \tag{187}$$

All projections are independent across triangles and can be computed in parallel.

Global step. Fix all $\mathbf{T}_i$ and solve for $\mathbf{x}$. With strain constraints only, the linear system is:

$$\underbrace{\left[\frac{1}{h^2}\mathbf{M} + \sum_i k_i A_i \mathbf{G}_i^\top \mathbf{G}_i\right]}_{\mathbf{L}} \mathbf{x} = \underbrace{\frac{1}{h^2}\mathbf{M}\tilde{\mathbf{x}} + \sum_i k_i A_i \mathbf{G}_i^\top \mathrm{vec}(\mathbf{T}_i)}_{\mathbf{b}} \tag{188}$$

The system matrix $\mathbf{L}$ is constant and prefactored once. Only the right-hand side $\mathbf{b}$ is recomputed at each iteration as the projections $\mathbf{T}_i$ update.

11.4 PD with Dry Frictional Contact [13]

11.4.1 Velocity-Based PD

The Signorini-Coulomb law is expressed in terms of velocities, not positions. To incorporate frictional contact into PD's local/global iteration, we reformulate the global step with velocity as the primary unknown.
Starting from the position-based global step:

$$\mathbf{L}\,\mathbf{x}^+ = \mathbf{b}$$

with

$$\mathbf{L} = \mathbf{M} + h^2 \sum_i k_i \mathbf{G}_i^\top \mathbf{G}_i$$
$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum_i k_i \mathbf{G}_i^\top \mathbf{p}_i$$

We substitute $\mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^+$ and rearrange:

$$\mathbf{L}\,(\mathbf{x}^- + h\mathbf{v}^+) = \mathbf{b} \qquad \Rightarrow \qquad \mathbf{L}\,\mathbf{v}^+ = \underbrace{\tfrac{1}{h}(\mathbf{b} - \mathbf{L}\mathbf{x}^-)}_{\tilde{\mathbf{b}}}$$

so that we can write the velocity base global step of the PD framework as:

$$\mathbf{L}\,\mathbf{v}^+ = \tilde{\mathbf{b}} \qquad \text{with} \qquad \tilde{\mathbf{b}} = \frac{1}{h}(\mathbf{b} - \mathbf{L}\mathbf{x}^-) \tag{189}$$

The system matrix $\mathbf{L}$ is unchanged, the same prefactored Cholesky applies. Only the right-hand side differs. After solving for $\mathbf{v}^+$, positions are recovered as $\mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^+$.

11.4.2 Signorini–Coulomb Law

Each vertex can be in contact. Each contact j is described by the local frame $\mathbf{R}_j = (\hat{\mathbf{n}}_j, \hat{\mathbf{t}}_j', \hat{\mathbf{t}}_j'')$, which acts as a rotation matrix from world space to local contact space. For any world-space vector $\mathbf{p}_j \in \mathbb{R}^3$, the normal and tangential components are:

$$\mathbf{p}_{j|\mathbf{n}} = \mathbf{p}_j^\top \mathbf{R}_j^\top \hat{\mathbf{n}}_j \qquad \in \mathbb{R}$$
$$\mathbf{p}_{j|\mathbf{t}} = \mathbf{p}_j - \mathbf{p}_{j|\mathbf{n}} \mathbf{R}_j^\top \hat{\mathbf{n}}_j \qquad \in \mathbb{R}^3$$

For each contact j, the Signorini–Coulomb law constrains the relative velocity $\mathbf{u}_j \in \mathbb{R}^3$ and local contact force $\mathbf{r}_j \in \mathbb{R}^3$ to satisfy $(\mathbf{r}_j, \mathbf{u}_j) \in \mathcal{C}_{\mu_j}$, defined by three mutually exclusive and exhaustive cases:

- Take-off: $\mathbf{r}_j = \mathbf{0}$ and $\mathbf{u}_{j|\mathbf{n}} \geq 0$
- Stick: $\|\mathbf{r}_{j|\mathbf{t}}\| \leq \mu_j\, \mathbf{r}_{j|\mathbf{n}}$ and $\mathbf{u}_j = \mathbf{0}$
- Slip: $\|\mathbf{r}_{j|\mathbf{t}}\| = \mu_j\, \mathbf{r}_{j|\mathbf{n}}, \quad \mathbf{u}_{j|\mathbf{n}} = 0, \quad$ and $\exists\, \alpha > 0$ such that $\mathbf{r}_{j|\mathbf{t}} = -\alpha\, \mathbf{u}_{j|\mathbf{t}}$

Adding frictional contact to the velocity-based PD framework yields:

$$\begin{cases} \mathbf{L}\,\mathbf{v}^+ = \tilde{\mathbf{b}} + \mathbf{J}^\top \mathbf{r}^+ \\ \mathbf{u}^+ = \mathbf{J}\,\mathbf{v}^+ \\ (\mathbf{r}_j^+, \mathbf{u}_j^+) \in \mathcal{C}_{\mu_j} \qquad \forall\, j = 1, \ldots, N_c \end{cases} \tag{190}$$

where $\mathbf{J}$ is the contact Jacobian mapping nodal velocities to local contact velocities. For nodal contact, each row-block of $\mathbf{J}$ contains a single 3×3 rotation $\mathbf{R}_j^\top$ at the column corresponding to the contacting node, and zeros elsewhere.

11.4.3 Local Update of the Frictional Contact Forces

We split $\mathbf{L}$ into its mass and elastic contributions:

$$\mathbf{L} = \mathbf{M} + \underbrace{h^2 \sum_i k_i \mathbf{G}_i^\top \mathbf{G}_i}_{\mathbf{C}} = \mathbf{M} + \mathbf{C}$$

where $\mathbf{M}$ is a diagonal positive matrix and $\mathbf{C}$ is positive semi-definite. Using a Jacobi-like split, treating $\mathbf{M}$ implicitly and lagging $\mathbf{C}$ to the previous iterate, the global equation becomes:

$$\mathbf{M}\,\mathbf{v}^{(k+1)} = \underbrace{\tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^{(k)}}_{\mathbf{f}} + \mathbf{J}^\top \mathbf{r}^{(k+1)} \tag{191}$$

The vector $\mathbf{f}$ is fully computable from previous-iteration data. Exploiting the diagonal structure of $\mathbf{M}$, we write the per-node equation:

$$M_i\,\mathbf{v}_i^{(k+1)} = \begin{cases} \mathbf{f}_i & \text{if } i \text{ not in contact} \\ \mathbf{f}_i + \mathbf{R}_j\,\mathbf{r}_j^{(k+1)} & \text{if } i \text{ in contact } j \end{cases} \tag{192}$$

Rotating into the local contact frame and using $\mathbf{u}_j = \mathbf{R}_j^\top \mathbf{v}_i$, the contacting-node equation becomes (dropping the iteration superscript for readability):

$$M_i\,\mathbf{u}_j = \underbrace{\mathbf{R}_j^\top \mathbf{f}_i}_{\mathbf{d}_j} + \mathbf{r}_j \qquad (\mathbf{r}_j, \mathbf{u}_j) \in \mathcal{C}_{\mu_j} \tag{193}$$

This is a 3D equation with six scalar unknowns ($\mathbf{u}_j$ and $\mathbf{r}_j$), pinned to a unique solution by the Signorini–Coulomb law. The normal component of $\mathbf{d}_j$ determines contact vs. take-off, and its tangential magnitude relative to the friction-cone threshold $\mu_j\,|\mathbf{d}_{j|\mathbf{n}}|$ determines stick vs. slip:

- If $\mathbf{d}_{j|\mathbf{n}} \geq 0$ (take-off): the node separates freely.

$$\mathbf{r}_j = \mathbf{0}$$

- If $\mathbf{d}_{j|\mathbf{n}} < 0 \;\wedge\; \|\mathbf{d}_{j|\mathbf{t}}\| \leq -\mu_j\,\mathbf{d}_{j|\mathbf{n}}$ (stick): the required friction fits inside the cone.

$$\mathbf{r}_j = -\mathbf{d}_j$$

- If $\mathbf{d}_{j|\mathbf{n}} < 0 \;\wedge\; \|\mathbf{d}_{j|\mathbf{t}}\| > -\mu_j\,\mathbf{d}_{j|\mathbf{n}}$ (slip): friction saturates at the cone boundary, opposing the sliding direction.

$$\mathbf{r}_{j|\mathbf{n}} = -\mathbf{d}_{j|\mathbf{n}} \qquad \mathbf{r}_{j|\mathbf{t}} = -\mu_j\,\mathbf{r}_{j|\mathbf{n}}\,\frac{\mathbf{d}_{j|\mathbf{t}}}{\|\mathbf{d}_{j|\mathbf{t}}\|}$$

Algorithm 16 Projective Dynamics step with dry frictional contact.

function PDDryFrictionalContactStep($\mathbf{x}^-, \mathbf{v}^-, \mathbf{M}, h$)
  $\mathbf{x}^0 \leftarrow \mathbf{x}^- + h\mathbf{v}^- + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}$
  $\mathbf{v}^0 \leftarrow \mathbf{v}^-$
  $\mathbf{R}_j, \mathbf{J} \leftarrow$ DetectContacts($\mathbf{x}^0$)
  for $k \in [1, 2, \ldots, N_{\max}]$ **do**
    for elastic constraint i **do**
      $\mathbf{p}_i^{(k+1)} \leftarrow$ Project$_{\mathcal{M}_i}(\mathbf{x}^{(k)})$
    $\mathbf{f} \leftarrow \tilde{\mathbf{b}}(\mathbf{p}^{(k+1)}) - \mathbf{C}\mathbf{v}^{(k)}$
    for contact j **do**
      $\mathbf{d}_j \leftarrow \mathbf{R}_j^\top \mathbf{f}_i$
      $\mathbf{r}_j^{(k+1)} \leftarrow$ UpdateContactForce($\mathbf{d}_j, \mathbf{R}_j, M_i, \mu_j$)
    SOLVE $\mathbf{L}\mathbf{v}^{(k+1)} = \tilde{\mathbf{b}}(\mathbf{p}^{(k+1)}) + \mathbf{J}^\top \mathbf{r}^{(k+1)}$
    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + h\mathbf{v}^{(k+1)}$
  $\mathbf{x}^+ \leftarrow \mathbf{x}^{N_{\max}}$
  $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$
end function

function UpdateContactForce($\mathbf{d}_j, \mathbf{R}_j, M_i, \mu_j$)
  $\mathbf{d}_{j|\mathbf{n}}, \mathbf{d}_{j|\mathbf{t}} =$ Decompose($\mathbf{d}_j, \mathbf{R}_j$)
  if $\mathbf{d}_{j|\mathbf{n}} \geq 0$ **then**
    return 0
  if $\|\mathbf{d}_{j|\mathbf{t}}\| \leq -\mu_j \mathbf{d}_{j|\mathbf{n}}$ **then**
    return $-\mathbf{d}_j$
  else
    $\mathbf{r}_{j|\mathbf{n}} = -\mathbf{d}_{j|\mathbf{n}}$
    $\mathbf{r}_{j|\mathbf{t}} = -\mu_j \mathbf{r}_{j|\mathbf{n}} \frac{\mathbf{d}_{j|\mathbf{t}}}{\|\mathbf{d}_{j|\mathbf{t}}\|}$
    return Assemble($\mathbf{r}_{j|\mathbf{n}}, \mathbf{r}_{j|\mathbf{t}}, \mathbf{R}_j$)
end function

11.5 Differentiable PD

11.5.1 Backpropagation Through Implicit Time Integration

Consider the variational objective from (175):

$$\mathbf{g}(\mathbf{x}) = \frac{1}{2h^2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}\,(\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x}) \tag{194}$$

The implicit Euler solution $\mathbf{x}^+$ satisfies the optimality condition $\nabla \mathbf{g}(\mathbf{x}^+) = \mathbf{0}$, regardless of the solver used (Newton, PD, or otherwise). A standard Newton solver would find this critical point via the iterative update:

$$\nabla^2 \mathbf{g}(\mathbf{x}^{(k)})\, \Delta \mathbf{x} = -\nabla \mathbf{g}(\mathbf{x}^{(k)}), \qquad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta \mathbf{x}$$

where the gradient and Hessian of g are:

$$\nabla \mathbf{g} = \frac{1}{h^2}\mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + \nabla U(\mathbf{x}), \qquad \nabla^2 \mathbf{g} = \frac{1}{h^2}\mathbf{M} + \nabla^2 U(\mathbf{x}) \tag{195}$$

The adjoint method. Given a scalar loss $\phi(\mathbf{x})$ defined on the simulation output, suppose we know $\frac{\partial \phi}{\partial \mathbf{x}}$ and wish to compute $\frac{\partial \phi}{\partial \tilde{\mathbf{x}}}$. At convergence, $\mathbf{x}$ and $\tilde{\mathbf{x}}$ are implicitly related by $\nabla \mathbf{g}(\mathbf{x}) = \mathbf{0}$. Differentiating this condition with respect to $\tilde{\mathbf{x}}$:

$$\nabla^2 \mathbf{g}\, \frac{\partial \mathbf{x}}{\partial \tilde{\mathbf{x}}} - \frac{1}{h^2}\mathbf{M} = \mathbf{0} \qquad \Longrightarrow \qquad \frac{\partial \mathbf{x}}{\partial \tilde{\mathbf{x}}} = \frac{1}{h^2}[\nabla^2 \mathbf{g}]^{-1}\, \mathbf{M} \tag{196}$$

Applying the chain rule:

$$\frac{\partial \phi}{\partial \tilde{\mathbf{x}}} = \frac{\partial \phi}{\partial \mathbf{x}}\, \frac{\partial \mathbf{x}}{\partial \tilde{\mathbf{x}}} = \frac{1}{h^2}\, \underbrace{\frac{\partial \phi}{\partial \mathbf{x}}\, [\nabla^2 \mathbf{g}]^{-1}}_{\mathbf{z}^\top}\, \mathbf{M} \tag{197}$$

Rather than forming the expensive product $[\nabla^2 \mathbf{g}]^{-1}\mathbf{M}$, we define the adjoint vector $\mathbf{z}$ by solving a single linear system:

$$\nabla^2 \mathbf{g}\, \mathbf{z} = \left(\frac{\partial \phi}{\partial \mathbf{x}}\right)^\top \tag{198}$$

after which $\frac{\partial \phi}{\partial \tilde{\mathbf{x}}} = h^{-2}\, \mathbf{z}^\top \mathbf{M}$ is a cheap matrix-vector product. This reduces backpropagation through one time step to a single linear solve with the Hessian $\nabla^2 \mathbf{g}$, the same matrix that appears in forward Newton solves. For a standard Newton-based simulator, this requires assembling and factorizing $\nabla^2 \mathbf{g}$ at each time step during backpropagation.

11.5.2 DiffPD: Exploiting PD Structure in Backpropagation [14]

In the PD framework, each energy potential is defined as a projection-based distance:

$$U_i(\mathbf{x}) = \min_{\mathbf{p}_i \in \mathcal{M}_i} \frac{k_i}{2} \|\mathbf{G}_i \mathbf{x} - \mathbf{p}_i\|_2^2, \qquad U(\mathbf{x}) = \sum_i U_i(\mathbf{x}) \tag{199}$$

where $\mathbf{p}_i(\mathbf{x})$ denotes the projection of $\mathbf{G}_i \mathbf{x}$ onto $\mathcal{M}_i$, which depends implicitly on $\mathbf{x}$. Recall from (180) that the forward global step solves:

$$\underbrace{\left[\frac{1}{h^2}\mathbf{M} + \sum_i k_i\, \mathbf{G}_i^\top \mathbf{G}_i\right]}_{\mathbf{A}} \mathbf{x} = \frac{1}{h^2}\mathbf{M}\, \tilde{\mathbf{x}} + \sum_i k_i\, \mathbf{G}_i^\top \mathbf{p}_i \tag{200}$$

Decomposing the Hessian. To apply the adjoint method (198), we need $\nabla^2 \mathbf{g}$. The gradient and Hessian of the elastic energy are:

$$\nabla U(\mathbf{x}) = \sum_i k_i \, \mathbf{G}_i^\top (\mathbf{G}_i \mathbf{x} - \mathbf{p}_i) \tag{201}$$

$$\nabla^2 U(\mathbf{x}) = \sum_i k_i \, \mathbf{G}_i^\top \mathbf{G}_i - \sum_i k_i \, \mathbf{G}_i^\top \frac{\partial \mathbf{p}_i}{\partial \mathbf{x}} \tag{202}$$

In the gradient, the dependence of $\mathbf{p}_i$ on $\mathbf{x}$ can be ignored by the envelope theorem. In the Hessian, this dependence must be retained.
Combining with the momentum term from (195):

$$\nabla^2 g(\mathbf{x}) = \underbrace{\frac{1}{h^2}\mathbf{M} + \sum_i k_i \, \mathbf{G}_i^\top \mathbf{G}_i}_{\mathbf{A}} \; - \; \underbrace{\sum_i k_i \, \mathbf{G}_i^\top \frac{\partial \mathbf{p}_i}{\partial \mathbf{x}}}_{\Delta \mathbf{A}} \; = \; \mathbf{A} - \Delta \mathbf{A} \tag{203}$$

The constant matrix $\mathbf{A}$ is exactly the PD system matrix, already prefactored during forward simulation. The remainder $\Delta \mathbf{A}$ captures the nonlinearity of the constraint projections.

Iterative adjoint solver. Substituting $\nabla^2 g = \mathbf{A} - \Delta \mathbf{A}$ into the adjoint system (198) and rearranging yields the fixed-point iteration:

$$\mathbf{A} \, \mathbf{z}^{(k+1)} = \Delta \mathbf{A} \, \mathbf{z}^{(k)} + \left(\frac{\partial \phi}{\partial \mathbf{x}} \right)^\top \tag{204}$$

This has the same local/global structure as forward PD. In the **local step**, the product $\Delta \mathbf{A} \, \mathbf{z}^{(k)}$ is assembled from independent per-constraint contributions $k_i \, \mathbf{G}_i^\top (\frac{\partial \mathbf{p}_i}{\partial \mathbf{x}}) \, \mathbf{z}^{(k)}$, computed in parallel. In the **global step**, the updated adjoint vector $\mathbf{z}^{(k+1)}$ is obtained via back-substitution using the *same* prefactored Cholesky decomposition of $\mathbf{A}$ from forward simulation. No additional matrix assembly or factorization is required.

11.5.3 DiffCloth: DiffPD with Dry Frictional Contact [15]

For each contact j, the forward solver identifies which Signorini–Coulomb branch applies and enforces three equality constraints $\mathbf{C}_j(\mathbf{r}_j, \mathbf{u}_j) = \mathbf{0}$:

- Take-off:

$$\mathbf{C}_j(\mathbf{r}_j, \mathbf{u}_j) = \begin{bmatrix} \mathbf{r}_{j|\mathbf{t}'} \\ \mathbf{r}_{j|\mathbf{t}''} \\ \mathbf{r}_{j|\mathbf{n}} \end{bmatrix} = \mathbf{0} \tag{205}$$

- Stick:

$$\mathbf{C}_j(\mathbf{r}_j, \mathbf{u}_j) = \begin{bmatrix} \mathbf{u}_{j|\mathbf{t}'} \\ \mathbf{u}_{j|\mathbf{t}''} \\ \mathbf{u}_{j|\mathbf{n}} \end{bmatrix} = \mathbf{0} \tag{206}$$

- Slip:

$$\mathbf{C}_j(\mathbf{r}_j, \mathbf{u}_j) = \begin{bmatrix} \|\mathbf{r}_{j|\mathbf{t}}\| - \mu\, \mathbf{r}_{j|\mathbf{n}} \\ \mathbf{u}_{j|\mathbf{n}} \\ \mathbf{r}_{j|\mathbf{t}'}\, \mathbf{u}_{j|\mathbf{t}''} - \mathbf{r}_{j|\mathbf{t}''}\, \mathbf{u}_{j|\mathbf{t}'} \end{bmatrix} = \mathbf{0} \tag{207}$$

Stacking all per-contact constraints, the forward system becomes:

$$\begin{cases} \mathbf{v}^+ = \mathbf{v}^- + h\, \mathbf{M}^{-1}[\mathbf{f}_{\text{int}}(\mathbf{x}^- + h\mathbf{v}^+) + \mathbf{f}_{\text{ext}} + \mathbf{J}^\top \mathbf{r}^+] \\ \mathbf{C}(\mathbf{r}^+,\, \mathbf{J}\mathbf{v}^+) = \mathbf{0} \end{cases} \tag{208}$$

where $\mathbf{C} : \mathbb{R}^{3N_c} \times \mathbb{R}^{3N_c} \to \mathbb{R}^{3N_c}$ stacks all per-contact constraint functions. The contact Jacobian $\mathbf{J}$ and the branch assignment for each contact are determined from the previous state $(\mathbf{x}^-, \mathbf{v}^-)$ and held fixed during the current timestep.

Backpropagation. Given $\frac{\partial \phi}{\partial \mathbf{v}^+}$ from downstream, we seek $\frac{\partial \phi}{\partial \mathbf{v}^-}$. Differentiating the first equation in (208) with respect to $\mathbf{v}^-$ and using $\mathbf{f}_{\text{int}} = -\nabla U$ yields:

$$\frac{\partial\, \mathbf{v}^+}{\partial\, \mathbf{v}^-} = \left[\mathbf{M} + h^2 \nabla^2 U - \underbrace{h\, \mathbf{J}^\top \frac{\partial\, \mathbf{r}^+}{\partial\, \mathbf{v}^+}}_{\Delta \mathbf{R}} \right]^{-1} \mathbf{M} \tag{209}$$

Applying the chain rule and defining the adjoint vector $\mathbf{z}$:

$$\frac{\partial\, \phi}{\partial\, \mathbf{v}^-} = \frac{\partial\, \phi}{\partial\, \mathbf{v}^+} \frac{\partial\, \mathbf{v}^+}{\partial\, \mathbf{v}^-} = \mathbf{M} \underbrace{\frac{\partial\, \phi}{\partial\, \mathbf{v}^+} \left[\underbrace{\mathbf{M} + h^2 \nabla^2 U}_{\mathbf{A} - \Delta \mathbf{A}} - \Delta \mathbf{R} \right]^{-1}}_{\mathbf{z}^\top} \tag{210}$$

The adjoint vector satisfies:

$$(\mathbf{A} - \Delta \mathbf{A} - \Delta \mathbf{R})\, \mathbf{z} = \left(\frac{\partial\, \phi}{\partial\, \mathbf{v}^+} \right)^\top \tag{211}$$

Reusing the prefactored Cholesky of $\mathbf{A}$ (the same PD system matrix from forward simulation), this is solved iteratively:

$$\mathbf{A}\, \mathbf{z}^{(k+1)} = (\Delta \mathbf{A} + \Delta \mathbf{R})\, \mathbf{z}^{(k)} + \left(\frac{\partial\, \phi}{\partial\, \mathbf{v}^+} \right)^\top \tag{212}$$

Each iteration has three steps: an elastic local step computing $\Delta \mathbf{A}\, \mathbf{z}^{(k)}$ per constraint in parallel (identical to DiffPD), a contact local step computing $\Delta \mathbf{R}\, \mathbf{z}^{(k)}$ per contact in parallel, and a global step via back-substitution with the prefactored $\mathbf{A}$.

11.6 Differentiable Mass-Spring Systems via PD

11.6.1 Forward PD

Spring potential. For each spring i connecting particles at indices i_1 and i_2, the augmented constraint potential is:

$$U_i(\mathbf{x}, \mathbf{p}_i) = \frac{k_i}{2}\|\mathbf{x}_{i_1} - \mathbf{x}_{i_2} - \mathbf{p}_i\|^2 = \frac{k_i}{2}\|\mathbf{G}_i\,\mathbf{x} - \mathbf{p}_i\|^2 \tag{213}$$

where $\mathbf{p}_i$ is an auxiliary variable representing the constrained direction, and k_i is the spring stiffness.

Selection matrix $\mathbf{G}_i$. The matrix $\mathbf{G}_i \in \mathbb{R}^{3\times 3n}$ extracts and differentiates the two particle positions:

$$\mathbf{G}_i = \begin{bmatrix} \mathbf{0} & \cdots & \mathbf{I} & \cdots & -\mathbf{I} & \cdots & \mathbf{0} \end{bmatrix} \qquad \mathbf{G}_i\,\mathbf{x} = \mathbf{x}_{i_1} - \mathbf{x}_{i_2}$$

with $\mathbf{I}_{3\times 3}$ at particle i_1 and $-\mathbf{I}_{3\times 3}$ at particle i_2. The resulting Hessian $\mathbf{H}_i = \mathbf{G}_i^\top \mathbf{G}_i$ has positive diagonal blocks at $[i_1, i_1]$ and $[i_2, i_2]$, and negative coupling blocks at $[i_1, i_2]$ and $[i_2, i_1]$.
For example:

$$\mathbf{G} = \begin{bmatrix} \mathbf{0} & \mathbf{I} & -\mathbf{I} & \mathbf{0} \end{bmatrix} \qquad \mathbf{G}^\top\mathbf{G} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & -\mathbf{I} & \mathbf{0} \\ \mathbf{0} & -\mathbf{I} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}$$

Local step. Fix $\mathbf{x}$ and project the current edge vector onto the constraint manifold, enforcing the rest length l_i:

$$\mathbf{p}_i^* = l_i \cdot \frac{\mathbf{x}_{i_1} - \mathbf{x}_{i_2}}{\|\mathbf{x}_{i_1} - \mathbf{x}_{i_2}\|} \tag{214}$$

This is an independent per-spring operation.

Global step. Fix all $\mathbf{p}_i$ and solve for $\mathbf{x}$. The objective is quadratic, yielding the symmetric positive definite system:

$$\mathbf{L}\,\mathbf{x} = \mathbf{b} \tag{215}$$

where the system matrix and right-hand side are:

$$\mathbf{L} = \frac{1}{h^2}\mathbf{M} + \sum_i k_i\,\mathbf{G}_i^\top\mathbf{G}_i$$

$$\mathbf{b} = \frac{1}{h^2}\mathbf{M}\tilde{\mathbf{x}} + \sum_i k_i\,\mathbf{G}_i^\top\mathbf{p}_i^*$$

- The left-hand side matrix $\mathbf{L}$ is constant, depending only on spring stiffness, rest configuration, and mass distribution.
- The right-hand side $\mathbf{b}$ combines the inertial trajectory $\tilde{\mathbf{x}}$ with constraint forces: each term $\mathbf{G}_i^\top \mathbf{p}_i^*$ is a global $3n$-vector distributing the spring force $\mathbf{p}_i^*$ as $+\mathbf{p}_i^*$ at particle i_1 and $-\mathbf{p}_i^*$ at particle i_2, representing equal and opposite spring forces. For example, with 4 particles and a spring between particles 1 and 2:

$$\mathbf{G}^\top \mathbf{p}^* = \begin{bmatrix} \mathbf{p}^* \\ -\mathbf{p}^* \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}$$

Single-particle spring. When a free particle i is connected to a pinned vertex fixed at $\bar{\mathbf{x}}_i \notin \mathbf{x}$, the two-particle spring reduces to a single-particle constraint, since the pinned position is a constant and not a degree of freedom:

$$U_i(\mathbf{x}, \mathbf{p}_i) = \frac{k_i}{2} \|\mathbf{x}_i - \bar{\mathbf{x}}_i - \mathbf{p}_i\|^2 \tag{216}$$

The selection matrix and local projection simplify accordingly:

$$\mathbf{G}_i = \begin{bmatrix} \mathbf{0} & \cdots & \mathbf{I} & \cdots & \mathbf{0} \end{bmatrix} \qquad \mathbf{G}_i \, \mathbf{x} = \mathbf{x}_i$$

$$\mathbf{p}_i^* = l_i \cdot \frac{\mathbf{x}_i - \bar{\mathbf{x}}_i}{\|\mathbf{x}_i - \bar{\mathbf{x}}_i\|} \tag{217}$$

where $\bar{\mathbf{x}}_i$ enters only the local step and never the global system. The Hessian contribution $\mathbf{H}_i = \mathbf{G}_i^\top \mathbf{G}_i$ has a single non-zero block $\mathbf{I}_3$ at $[i, i]$, and the RHS contribution $\mathbf{G}_i^\top \mathbf{p}_i^*$ places $\mathbf{p}_i^*$ only at particle i.

11.6.2 Backward PD

The backward pass requires solving for the adjoint vector $\mathbf{z}$ using the true Hessian of the objective, which decomposes as:

$$\nabla^2 g(\mathbf{x}) \;=\; \mathbf{L} - \Delta\mathbf{L}$$

where $\mathbf{L}$ is the same constant matrix from the forward pass, and $\Delta\mathbf{L}$ captures the nonlinearity of the local projections:

$$\Delta\mathbf{L} = \sum_i k_i \, \mathbf{G}_i^\top \frac{\partial \, \mathbf{p}_i}{\partial \, \mathbf{x}}$$

Spring Local Derivative For a two-particle spring, the local projection $\mathbf{p}_i^*$ depends on $\mathbf{x}$ only through the edge vector $\mathbf{e}$, so by the chain rule:

$$\frac{\partial \, \mathbf{p}_i^*}{\partial \, \mathbf{x}} = \frac{\partial \, \mathbf{p}_i^*}{\partial \, \mathbf{e}} \frac{\partial \, \mathbf{e}}{\partial \, \mathbf{x}} = \frac{l_i}{\|\mathbf{e}\|} \mathbf{P}_{\mathbf{e}}^{\perp} \mathbf{G}_i \tag{218}$$

$$\mathbf{e} = \mathbf{x}_{i_1} - \mathbf{x}_{i_2} \qquad \mathbf{P}_{\mathbf{e}}^{\perp} = \mathbf{I} - \frac{\mathbf{e}\mathbf{e}^{\top}}{\|\mathbf{e}\|^2} \tag{219}$$

where $\mathbf{P}_{\mathbf{e}}^{\perp}$ projects out the component along $\mathbf{e}$, encoding how the projection onto the rest-length sphere varies with position. The contribution to $\Delta\mathbf{L}$ from a single two-particle spring constraint is therefore:

$$\frac{k_i l_i}{\|\mathbf{e}\|}\mathbf{G}_i^{\top}\mathbf{P}_{\mathbf{e}}^{\perp}\mathbf{G}_i \tag{220}$$

The same derivation applies to the single-particle spring, with $\mathbf{e} = \mathbf{x}_i - \bar{\mathbf{x}}_i$, yielding an identical contribution to $\Delta\mathbf{L}$ with a single non-zero 3×3 block at $[i, i]$.

11.6.3 Implementation

Algorithm 17 Construct initial simulation state from rest mesh and parameters.

function ConstructObject($\mathbb{M} = (\mathbf{V}, \mathbf{X}, \mathbf{E}), \mathbf{P}, m_{\text{tot}}, k$)
  $\mathbf{V}_{\text{free}} \leftarrow \mathbf{V} \setminus \mathbf{P}$
  $n \leftarrow |\mathbf{V}_{\text{free}}|$
  $\text{DOF}_{\text{map}} \leftarrow \{v : i \text{ for } i, v \in \text{ENUMERATE}(\mathbf{V}_{\text{free}})\}$
  $\mathbf{M} \leftarrow m_{\text{tot}}/n \cdot \mathbf{I}_{3n}$
  $\mathbf{x}_0 \leftarrow [\mathbf{X}[v] : v \in \mathbf{V}_{\text{free}}]$
  $\mathbf{v}_0 \leftarrow \mathbf{0}_{3n}$
  $\mathbb{C} \leftarrow \{\}$
  for $(v_1, v_2) \in \mathbf{E}$ **do**
    $l_0 \leftarrow \|\mathbf{X}[v_1] - \mathbf{X}[v_2]\|$
    if $v_1, v_2 \in \mathbf{V}_{\text{free}}$ **then**
      $i_1 \leftarrow \text{DOF}_{\text{map}}[v_1]$
      $i_2 \leftarrow \text{DOF}_{\text{map}}[v_2]$
      $\mathbb{C} \leftarrow \mathbb{C} \cup \{\texttt{spring2}, k, l_0, (i_1, i_2)\}$
    else if $v_1 \in \mathbf{V}_{\text{free}}, v_2 \in \mathbf{P}$ **then**
      $i \leftarrow \text{DOF}_{\text{map}}[v_1]$;
      $\bar{\mathbf{x}} \leftarrow \mathbf{X}[v_2]$
      $\mathbb{C} \leftarrow \mathbb{C} \cup \{\texttt{spring1}, k, l_0, \bar{\mathbf{x}}, i\}$
    else if $v_1 \in \mathbf{P}, v_2 \in \mathbf{V}_{\text{free}}$ **then**
      $i \leftarrow \text{DOF}_{\text{map}}[v_2]$;
      $\bar{\mathbf{x}} \leftarrow \mathbf{X}[v_1]$
      $\mathbb{C} \leftarrow \mathbb{C} \cup \{\texttt{spring1}, k, l_0, \bar{\mathbf{x}}, i\}$
  return $(\mathbf{M}, \mathbf{x}_0, \mathbf{v}_0, \mathbb{C})$
end function

The function ConstructObject, shown in Algorithm 17, initializes the full simulation state from a rest-pose mesh $\mathbb{M} = (\mathbf{V}, \mathbf{X}, \mathbf{E})$, where $\mathbf{V}$ is the set of vertex indices, $\mathbf{X}$ maps each vertex to its rest position in $\mathbb{R}^3$, and $\mathbf{E}$ is the set of edges encoding the spring topology. The set $\mathbf{P} \subseteq \mathbf{V}$ specifies pinned vertices, m_{tot} is the total object mass distributed uniformly across free particles, and k is a uniform stiffness applied to all constraints. The function returns the mass matrix $\mathbf{M}$, initial positions $\mathbf{x}_0$ and velocities $\mathbf{v}_0$ defined only over free particles $\mathbf{V}_{\text{free}} = \mathbf{V} \setminus \mathbf{P}$, and the constraint set $\mathbb{C}$ built from $\mathbf{E}$: edges between two free particles become double particle spring constraints (`spring2`), edges between a free and a pinned particle become single particle spring constraints (`spring1`) anchored at the pinned rest position.

Algorithm 18 Construct constant system matrix (LHS) for PD mass-spring system.

function ConstructLHS($\mathbb{C}, \mathbf{M}, h$)
  $\mathbf{L} \leftarrow \frac{1}{h^2}\mathbf{M}$
  for constraint $\mathbb{C}_i \in \mathbb{C}$ **do**
    if type($\mathbb{C}_i$) = `spring2` **then**
      $\{\_, k_i, \_, (i_1, i_2)\} \leftarrow \mathbb{C}_i$
      $\mathbf{L}_{[i_1,i_1]} \mathrel{+}= k_i\mathbf{I}_3$
      $\mathbf{L}_{[i_2,i_2]} \mathrel{+}= k_i\mathbf{I}_3$
      $\mathbf{L}_{[i_1,i_2]} \mathrel{-}= k_i\mathbf{I}_3$
      $\mathbf{L}_{[i_2,i_1]} \mathrel{-}= k_i\mathbf{I}_3$
    else if type($\mathbb{C}_i$) = `spring1` **then**
      $\{\_, k_i, \_, \_, i\} \leftarrow \mathbb{C}_i$
      $\mathbf{L}_{[i,i]} \mathrel{+}= k_i\mathbf{I}_3$
  return L
end function

Algorithm 19 RHS assembly for PD mass-spring system.

function ConstructRHS($\mathbb{C}, \mathbf{x}, \mathbf{b}_{\text{inertia}}$)
  $\mathbf{b} \leftarrow \mathbf{b}_{\text{inertia}}$
  for constraint $\mathbb{C}_i \in \mathbb{C}$ **do**
    if type($\mathbb{C}_i$) = `spring2` **then**
      $\{\_, k_i, l_i, (i_1, i_2)\} \leftarrow \mathbb{C}_i$
      $\mathbf{e} \leftarrow \mathbf{x}_{i_1} - \mathbf{x}_{i_2}$
      $\mathbf{p}_i^* \leftarrow l_i \cdot \frac{\mathbf{e}}{\|\mathbf{e}\|}$
      $\mathbf{b}_{i_1}$ += $k_i\, \mathbf{p}_i^*$
      $\mathbf{b}_{i_2}$ −= $k_i\, \mathbf{p}_i^*$
    else if type($\mathbb{C}_i$) = `spring1` **then**
      $\{\_, k_i, l_i, \bar{\mathbf{x}}_i, i\} \leftarrow \mathbb{C}_i$
      $\mathbf{e} \leftarrow \mathbf{x}_i - \bar{\mathbf{x}}_i$
      $\mathbf{p}_i^* \leftarrow l_i \cdot \frac{\mathbf{e}}{\|\mathbf{e}\|}$
      $\mathbf{b}_i$ += $k_i\, (\bar{\mathbf{x}}_i + \mathbf{p}_i^*)$
  return b
end function

Algorithm 20 PD mass-spring simulation loop.

function PD($\mathbf{x}_0, \mathbf{v}_0, \mathbf{M}, h, \mathbb{C}$)
  $\mathbf{L} \leftarrow$ ConstructLHS($\mathbb{C}, \mathbf{M}, h$)
  $\mathbf{U} \leftarrow$ SparseCholesky($\mathbf{L}$)
  $\mathbf{x}, \mathbf{v} \leftarrow \mathbf{x}_0, \mathbf{v}_0$
  while simulating **do**
    $\mathbf{x}_{\text{prev}} \leftarrow \mathbf{x}$
    $\tilde{\mathbf{x}} \leftarrow \mathbf{x} + h(\mathbf{v} + h\mathbf{M}^{-1}\mathbf{f}_{\text{ext}})$
    $\mathbf{b}_{\text{inertia}} = \frac{1}{h^2}\mathbf{M}\tilde{\mathbf{x}}$
    $\mathbf{x} \leftarrow \tilde{\mathbf{x}}$
    for $k \in [1, 2, \ldots, N_{\text{iter}}]$ **do**
      $\mathbf{b} \leftarrow$ ConstructRHS($\mathbb{C}, \mathbf{x}, \mathbf{b}_{\text{inertia}}$)
      $\mathbf{x} \leftarrow$ SolveCholesky($\mathbf{U}, \mathbf{b}$)
    $\mathbf{v} \leftarrow \frac{1}{h}(\mathbf{x} - \mathbf{x}_{\text{prev}})$
  end while
end function

12 Incremental Potential Contact (IPC) [3]

Algorithm 21 Collision culling algorithm.

function ComputeCollisionSet($\mathbf{x}, \hat{\mathbf{d}}$)
  $\mathbb{C} \leftarrow \{\ \}$
  $\mathbb{PT} \leftarrow$ PTBroadPhaseFilter($\mathbf{x}, \hat{\mathbf{d}}$)
  for (vertex i, triangle j) $\in \mathbb{PT}$ **do**
    $\mathbf{x}_p, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3 \leftarrow$ PTGetPoints($\mathbf{x}, i, j$)
    $\beta_1, \beta_2 \leftarrow$ ProjectPointOnTriangle($\mathbf{x}_p, \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}$)
    if $\beta_1 > 0 \wedge \beta_2 > 0 \wedge \beta_1 + \beta_2 < 1$ **then**
      $t_{ype} \leftarrow$ PT
      $d \leftarrow$ PTDistance($\mathbf{x}_p, \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\}$)
    else if one of $\{\beta_1, \beta_2, 1 - \beta_1 - \beta_2\}$ is 0 and other are in $(0, 1)$ **then**
      $t_{ype} \leftarrow$ PE
      $a, b \leftarrow$ IdentifyEdge(β_1, β_2)
      $d \leftarrow$ PEDistance($\mathbf{x}_p, \{\mathbf{x}_a, \mathbf{x}_b\}$)
    else
      $t_{ype} \leftarrow$ PP
      $a \leftarrow$ IdentifyVertex(β_1, β_2)
      $d \leftarrow$ PPDistance($\mathbf{x}_p, \mathbf{x}_a$)
    if $d < \hat{\mathbf{d}}$ **then**
      $\mathbb{C} \leftarrow \mathbb{C} \cup \{(i, j), \text{PT}, t_{ype}, d, (\beta_1, \beta_2), (a, b)\}$
  $\mathbb{EE} \leftarrow$ EEBroadPhaseFilter($\mathbf{x}, \hat{\mathbf{d}}$)
  for (edge i, edge j) $\in \mathbb{EE}$ **do**
    $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4 \leftarrow$ EEGetPoints($\mathbf{x}, i, j$)
    $\gamma_1, \gamma_2 \leftarrow$ ClosestPointsOnEdges($\{\mathbf{x}_1, \mathbf{x}_2\}, \{\mathbf{x}_3, \mathbf{x}_4\}$)
    if $\gamma_1 \in (0, 1) \wedge \gamma_2 \in (0, 1)$ **then**
      $t_{ype} \leftarrow$ EE
      $d \leftarrow$ EEDistance($\{\mathbf{x}_1, \mathbf{x}_2\}, \{\mathbf{x}_3, \mathbf{x}_4\}$)
    else if one of $\{\gamma_1, \gamma_2\}$ is at 0 or 1, other is in $(0, 1)$ **then**
      $t_{ype} \leftarrow$ PE
      $c, (a, b) \leftarrow$ IdentifyPointEdge(γ_1, γ_2)
      $d \leftarrow$ PEDistance($\mathbf{x}_c, \{\mathbf{x}_a, \mathbf{x}_b\}$)
    else
      $t_{ype} \leftarrow$ PP
      $a, b \leftarrow$ IdentifyVertices(γ_1, γ_2)
      $d \leftarrow$ PPDistance($\mathbf{x}_a, \mathbf{x}_b$)
    if $d < \hat{\mathbf{d}}$ **then**
      $\mathbb{C} \leftarrow \mathbb{C} \cup \{(i, j), \text{EE}, t_{ype}, d, (\gamma_1, \gamma_2), (a, b, c)\}$
  return $\mathbb{C}$
end function

Algorithm 22 .

function CCDStepSizeUpperBound($\mathbf{x}, \mathbf{p}, \mathbb{C}$)
  $\mathbb{C}^+ \leftarrow$ AugmentCollisionSet($\mathbf{x}, \mathbf{p}, \mathbb{C}$)
  $t_{\min} \leftarrow 1$
  for (vertex i, triangle j) $\in \mathbb{C}^+_{\text{PT}}$ **do**
    $\mathbf{x}_p, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3 \leftarrow$ PTGetPoints($\mathbf{x}, i, j$)
    $\mathbf{p}_p, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3 \leftarrow$ PTGetPoints($\mathbf{p}, i, j$)
    $a, b, c, d \leftarrow$ CoplanarityCubicCoefficients(
        $\mathbf{x}_p, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3,$
        $\mathbf{p}_p, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3$)
    for $t^* \in$ Roots(a, b, c, d) **do**
      if $t^* \notin [0, 1] \vee t^* \geq t_{\min}$ **then**
        continue
      $\bar{\mathbf{x}}_p, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2, \bar{\mathbf{x}}_3 = \{\mathbf{x}_p, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\} + t^* \cdot \{\mathbf{p}_p, \mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3\}$
      if PointIsInsideTriangle($\bar{\mathbf{x}}_p, \{\bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2, \bar{\mathbf{x}}_3\}$) **then**
        $t_{\min} = t^*$
  for (edge i, edge j) $\in \mathbb{C}^+_{\text{EE}}$ **do**
    $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4 \leftarrow$ EEGetPoints($\mathbf{x}, i, j$)
    $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4 \leftarrow$ EEGetPoints($\mathbf{p}, i, j$)
    $a, b, c, d \leftarrow$ CoplanarityCubicCoefficients(
        $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4,$
        $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4$)
    for $t^* \in$ Roots(a, b, c, d) **do**
      if $t^* \notin [0, 1] \vee t^* \geq t_{\min}$ **then**
        continue
      $\bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2, \bar{\mathbf{x}}_3, \bar{\mathbf{x}}_4 = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4\} + t^* \cdot \{\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4\}$
      if SegmentsIntersect($\{\bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2\}, \{\bar{\mathbf{x}}_3, \bar{\mathbf{x}}_4\}$) **then**
        $t_{\min} = t^*$
  return $t_{\min}$
end function

Algorithm 23 .

function IPCTimeStep($\mathbf{x}^-, \boldsymbol{\epsilon}$)
  $\mathbf{x} \leftarrow \mathbf{x}^-$
  $\mathbb{C} \leftarrow$ ComputeCollisionSet($\mathbf{x}, \hat{\mathbf{d}}$)
  $E^{(k-1)} \leftarrow B(\mathbf{x}, \hat{\mathbf{d}}, \mathbb{C})$
  $\mathbf{x}^{(k-1)} \leftarrow \mathbf{x}$
  do
    $\mathbf{J} \leftarrow \nabla_{\mathbf{x}} B(\mathbf{x}, \hat{\mathbf{d}}, \mathbb{C})$
    $\mathbf{H} \leftarrow$ PSDProjection($\nabla^2_{\mathbf{x}} B(\mathbf{x}, \hat{\mathbf{d}}, \mathbb{C})$)
    solve $\mathbf{Hp} = -\mathbf{J}$
    $\alpha =$ CCDStepSizeUpperBound($\mathbf{x}, \mathbf{p}, \mathbb{C}$)
    $\alpha =$ min(α, InversionStepSizeUpperBound($\mathbf{x}, \mathbf{p}$))
    do
      $\mathbf{x} \leftarrow \mathbf{x}^{(k-1)} + \alpha\,\mathbf{p}$
      $\mathbb{C} \leftarrow$ ComputeCollisionSet($\mathbf{x}, \hat{\mathbf{d}}$)
      $\alpha \leftarrow \alpha/2$
    while $B(\mathbf{x}, \hat{\mathbf{d}}, \mathbb{C}) > E^{(k-1)}$
    $E^{(k-1)} \leftarrow B(\mathbf{x}, \hat{\mathbf{d}}, \mathbb{C})$
    $\mathbf{x}^{(k-1)} \leftarrow \mathbf{x}$
  while $\frac{1}{h}\|\mathbf{p}\|_\infty > \boldsymbol{\epsilon}$
  return x
end function

12.1 Contact Barrier Energy

IPC enforces non-interpenetration by augmenting the elastic energy with a smooth barrier that activates when surface primitives approach within a threshold distance $\hat{d}$. For a primitive pair with unsigned distance $d(\mathbf{x})$, the barrier is:

$$b(d,\,\hat{d}) = \begin{cases} -\big(d(\mathbf{x}) - \hat{d}\big)^2 \ln\!\left(\dfrac{d(\mathbf{x})}{\hat{d}}\right) & \text{if } \; 0 < d(\mathbf{x}) < \hat{d} \\ 0 & \text{if } \; d(\mathbf{x}) \geq \hat{d} \end{cases} \tag{221}$$

This function is C^2 at $d = \hat{d}$, exactly zero beyond $\hat{d}$, and diverges as $d \to 0^+$. The barrier-augmented energy is then:

$$B(\mathbf{x}, \hat{d}, \mathbb{C}) = \Psi(\mathbf{x}) + \kappa \sum_{c \in \mathbb{C}} b\big(d_c(\mathbf{x}),\; \hat{d}\,\big) \tag{222}$$

where $\kappa > 0$ is an adaptively updated stiffness parameter, $\mathbb{C}$ is the culled constraint set, the subset of non-adjacent surface primitive pairs whose distance is currently below $\hat{d}$, and $\Psi(\mathbf{x})$ is the total internal elastic energy.

12.1.1 Collision Pair Distance Functions

All surface collisions reduce to two fundamental pair types: point–triangle and edge–edge. Depending on where the closest points lie on each primitive (determined by a constrained minimization over barycentric or edge parameters), the distance computation falls into one of four closed-form cases:

$$d^{\mathrm{PE}} = \|\mathbf{x}_1 - \mathbf{x}_2\| \tag{223}$$

$$d^{\mathrm{PE}} = \frac{\|(\mathbf{x}_1 - \mathbf{x}_p) \times (\mathbf{x}_2 - \mathbf{x}_p)\|}{\|\mathbf{x}_a - \mathbf{x}_b\|} \tag{224}$$

$$d^{\mathrm{PT}} = |(\mathbf{x}_p - \mathbf{x}_1) \cdot \hat{\mathbf{n}}|\,, \qquad \hat{\mathbf{n}} = \frac{(\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_3 - \mathbf{x}_1)}{\|(\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_3 - \mathbf{x}_1)\|} \tag{225}$$

$$d^{\mathrm{EE}} = |(\mathbf{x}_1 - \mathbf{x}_3) \cdot \hat{\mathbf{m}}|\,, \qquad \hat{\mathbf{m}} = \frac{(\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_4 - \mathbf{x}_3)}{\|(\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_4 - \mathbf{x}_3)\|} \tag{226}$$

Which formula applies and which vertices are taken in consideration is identified during the constraint-set computation and stored alongside each entry in $\mathbb{C}$.

12.2 Computing Energy Jacobians and SPD-Projected Hessians

We now examine the construction of the gradient vector and Hessian matrix at lines 7–8 of Algorithm 23. The barrier-augmented energy B is a sum of elastic, contact, and friction terms. We begin with the elastic energy.

12.2.1 Elastic Energy: First and Second Derivatives

IPC models 3D deformable bodies as tetrahedral finite-element meshes. Each tetrahedron e carries a hyperelastic energy density $\Psi_e(\mathbf{F}_e)$, which is a scalar function of the element's deformation gradient. The total elastic energy is

$$\Psi(\mathbf{x}) = \sum_e v_e \Psi_e\big(\mathbf{F}_e(\mathbf{x})\big), \tag{227}$$

where v_e is the rest-state volume of element e. Because each element's contribution depends only on its own four vertices, the global gradient and Hessian are assembled by summing independent per-element terms.

Deformation gradient. A tetrahedron is defined by four rest-state vertices $\bar{\mathbf{x}}_0, \bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2, \bar{\mathbf{x}}_3 \in \mathbb{R}^3$ and their deformed counterparts $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3 \in \mathbb{R}^3$. We form edge matrices from differences relative to the reference vertex:

$$\mathbf{D}_S = \big[\, \mathbf{x}_1 - \mathbf{x}_0 \;\big|\; \mathbf{x}_2 - \mathbf{x}_0 \;\big|\; \mathbf{x}_3 - \mathbf{x}_0 \,\big] \qquad \mathbf{D}_M = \big[\, \bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_0 \;\big|\; \bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_0 \;\big|\; \bar{\mathbf{x}}_3 - \bar{\mathbf{x}}_0 \,\big]$$

The deformation gradient is then

$$\mathbf{F} = \mathbf{D}_S\, \mathbf{D}_M^{-1} = \big[\, \mathbf{f}_0 \;\big|\; \mathbf{f}_1 \;\big|\; \mathbf{f}_2 \,\big] \in \mathbb{R}^{3\times 3}$$

Since $\mathbf{D}_M^{-1}$ depends only on the rest shape, it is precomputed once per element and remains constant throughout the simulation.
Two decompositions of $\mathbf{F}$ are used extensively. The *polar decomposition* $\mathbf{F} = \mathbf{RS}$ separates a rotation $\mathbf{R} \in \mathrm{SO}(3)$ from a symmetric stretch $\mathbf{S}$. The *singular value decomposition* $\mathbf{F} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$ yields the principal stretches $\sigma_1, \sigma_2, \sigma_3$ along the diagonal of $\boldsymbol{\Sigma}$.
We also make use of the *cofactor matrix* of $\mathbf{F}$,

$$\mathrm{cof}(\mathbf{F}) = \big[\, \mathbf{f}_1 \times \mathbf{f}_2 \;\big|\; \mathbf{f}_2 \times \mathbf{f}_0 \;\big|\; \mathbf{f}_0 \times \mathbf{f}_1 \,\big],$$

Stable Neo-Hookean (SNH). IPC defaults to the Stable Neo-Hookean model of [16], whose energy density is

$$\Psi_{\text{SNH}}(\mathbf{F}) = \frac{\mu}{2}\big(I_2 - 3\big) - \mu\big(I_3 - 1\big) + \frac{\lambda}{2}\big(I_3 - 1\big)^2, \tag{228}$$

where μ and λ are the Lamé parameters. The energy is expressed in terms of the Smith's invariants of $\mathbf{F}$ (defined first in [16]), defined through its singular values $\sigma_1, \sigma_2, \sigma_3$:

$$I_1 = \sigma_1 + \sigma_2 + \sigma_3 \quad = \mathrm{tr}(\boldsymbol{\Sigma}), \tag{229}$$

$$I_2 = \sigma_1^2 + \sigma_2^2 + \sigma_3^2 \quad = \|\mathbf{F}\|_F^2, \tag{230}$$

$$I_3 = \sigma_1\,\sigma_2\,\sigma_3 \quad = \det(\mathbf{F}). \tag{231}$$

First derivative (Jacobian). We now follow the notation of [7]. For each element, the derivative of the energy density with respect to its four vertex positions $\mathbf{x} \in \mathbb{R}^{12}$ is obtained via the chain rule:

$$\frac{\partial\,\Psi}{\partial\,\mathbf{x}} = \frac{\partial\,\mathbf{F}}{\partial\,\mathbf{x}} : \frac{\partial\,\Psi}{\partial\,\mathbf{F}} \tag{232}$$

This separates the computation into two independent factors. The first, $\frac{\partial \mathbf{F}}{\partial \mathbf{x}}$, is a third-order tensor that depends only on the rest geometry through $\mathbf{D}_M^{-1}$; it is constant, precomputed, and shared across all material models. The second, $\frac{\partial \Psi}{\partial \mathbf{F}}$, is the *first Piola–Kirchhoff stress tensor* (PK1), commonly denoted $\mathbf{P}(\mathbf{F})$. This term encodes the material response and differs for each constitutive model.
Per-element contributions are computed independently for each tetrahedron's four vertices and then scattered into the global gradient vector during assembly.

Second derivative (Hessian). The Hessian of the elastic energy with respect to vertex positions is obtained by a second application of the chain rule:

$$\frac{\partial^2\,\Psi}{\partial\,\mathbf{x}^2} = \frac{\partial\,\mathbf{F}^\top}{\partial\,\mathbf{x}}\,\frac{\partial^2\,\Psi}{\partial\,\mathbf{F}^2}\,\frac{\partial\,\mathbf{F}}{\partial\,\mathbf{x}}.$$

The outer factors $\frac{\partial \mathbf{F}}{\partial \mathbf{x}}$ are again constant and precomputed. The material-dependent core is the 9×9 Hessian in $\mathbf{F}$-space, for which [7] gives a generic

formula valid for any energy written in terms of the Smith invariants:

$$\operatorname{vec}\left(\frac{\partial^2 \Psi}{\partial \mathbf{F}^2}\right) = \sum_{i=1}^{3} \frac{\partial^2 \Psi}{\partial I_i^2}\, \mathbf{g}_i \mathbf{g}_i^\top + \frac{\partial \Psi}{\partial I_i}\, \mathbf{H}_i \,, \tag{233}$$

where $\mathbf{g}_i = \operatorname{vec}(\frac{\partial I_i}{\partial \mathbf{F}})$ and $\mathbf{H}_i = \operatorname{vec}(\frac{\partial^2 I_i}{\partial \mathbf{F}^2})$ are the flattened gradient and Hessian of each invariant. These are the same for every material model; only the scalar coefficients $\frac{\partial \Psi}{\partial I_i}$ and $\frac{\partial^2 \Psi}{\partial I_i^2}$ change. The invariant terms are:

$$\begin{aligned}
&\mathbf{g}_1 = \operatorname{vec}(\mathbf{R}), && \mathbf{H}_1 = \sum_{j=0}^{2} \lambda_j \operatorname{vec}(\mathbf{Q}_j) \operatorname{vec}(\mathbf{Q}_j)^\top, \\
&\mathbf{g}_2 = \operatorname{vec}(2\mathbf{F}), && \mathbf{H}_2 = 2\,\mathbf{I}_{9\times 9}, \\
&\mathbf{g}_3 = \operatorname{vec}\big(\operatorname{cof}(\mathbf{F})\big), && \mathbf{H}_3 = \begin{bmatrix} \mathbf{0} & -\mathbf{f}_2^\times & \mathbf{f}_1^\times \\ \mathbf{f}_2^\times & \mathbf{0} & -\mathbf{f}_0^\times \\ -\mathbf{f}_1^\times & \mathbf{f}_0^\times & \mathbf{0} \end{bmatrix}.
\end{aligned} \tag{234}$$

Here $\mathbf{R}$ is the rotation from the polar decomposition of $\mathbf{F}$, $\mathbf{f}_i^\times$ denotes the 3×3 skew-symmetric (cross-product) matrix of column $\mathbf{f}_i$, and λ_j, $\mathbf{Q}_j$ are the eigenvalues and eigenmatrices of the rotation gradient $\frac{\partial \mathbf{R}}{\partial \mathbf{F}}$, given by [7]:

$$\begin{aligned}
\lambda_0 &= \frac{2}{\sigma_x + \sigma_y}, \quad \mathbf{Q}_0 = \frac{1}{\sqrt{2}}\,\mathbf{U} \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \mathbf{V}^\top, \\
\lambda_1 &= \frac{2}{\sigma_y + \sigma_z}, \quad \mathbf{Q}_1 = \frac{1}{\sqrt{2}}\,\mathbf{U} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} \mathbf{V}^\top, \\
\lambda_2 &= \frac{2}{\sigma_x + \sigma_z}, \quad \mathbf{Q}_2 = \frac{1}{\sqrt{2}}\,\mathbf{U} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ -1 & 0 & 0 \end{bmatrix} \mathbf{V}^\top.
\end{aligned} \tag{235}$$

Hessian PSD projection. The per-element Hessian $\operatorname{vec}(\frac{\partial^2 \Psi}{\partial \mathbf{F}^2})$ can become indefinite under compression or large deformation. To guarantee that the assembled global Hessian is positive semi-definite (as required by the Newton solver), IPC projects each element's 9×9 Hessian to be PSD independently before assembly. This requires the eigendecomposition of $\operatorname{vec}(\frac{\partial^2 \Psi}{\partial \mathbf{F}^2})$, which, as shown by [16], admits a closed-form solution: all isotropic energies expressed through the Smith invariants share the same nine eigenmatrices, only the eigenvalues differ. They decompose into three groups.

- **Scaling modes** ($i = 0, 1, 2$): These eigenmatrices are linear combinations

of three basis matrices $\mathbf{D}_0, \mathbf{D}_1, \mathbf{D}_2$:

$$\mathbf{Q}_i = \sum_{j=0}^{2} z_j \, \mathbf{D}_j, \qquad \begin{cases} z_0 = \sigma_x \sigma_z + \sigma_y \lambda_i, \\ z_1 = \sigma_y \sigma_z + \sigma_x \lambda_i, \\ z_2 = \lambda_i^2 - \sigma_z^2. \end{cases}$$

with

$$\mathbf{D}_0 = \mathbf{U} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \mathbf{V}^\top \; \mathbf{D}_1 = \mathbf{U} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \mathbf{V}^\top \; \mathbf{D}_2 = \mathbf{U} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \mathbf{V}^\top$$

The corresponding eigenvalues $\lambda_{0,1,2}$ are energy-dependent.

- **Twist modes** ($i = 3, 4, 5$): these are antisymmetric matrices rotated by $\mathbf{U}$ and $\mathbf{V}$:

$$\mathbf{Q}_3 = \frac{1}{\sqrt{2}} \mathbf{U} \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \mathbf{V}^\top \qquad \mathbf{Q}_4 = \frac{1}{\sqrt{2}} \mathbf{U} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} \mathbf{V}^\top$$

$$\mathbf{Q}_5 = \frac{1}{\sqrt{2}} \mathbf{U} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ -1 & 0 & 0 \end{bmatrix} \mathbf{V}^\top$$

- **Flip modes** ($i = 6, 7, 8$): these are the symmetric counterparts of the twist modes:

$$\mathbf{Q}_6 = \frac{1}{\sqrt{2}} \mathbf{U} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \mathbf{V}^\top \qquad \mathbf{Q}_7 = \frac{1}{\sqrt{2}} \mathbf{U} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \mathbf{V}^\top$$

$$\mathbf{Q}_8 = \frac{1}{\sqrt{2}} \mathbf{U} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \mathbf{V}^\top$$

Stable Neo-Hookean: PK1, Hessian and it eigendecomposition. Applying the scalar derivatives of Ψ_{SNH} with respect to the Smith invariants to the generic formulas above yields closed-form expressions for the PK1 stress and Hessian. The flattened PK1 is:

$$\operatorname{vec}\left(\frac{\partial \, \Psi_{\text{SNH}}}{\partial \, \mathbf{F}}\right) = \mu \, \operatorname{vec}(\mathbf{F}) + \big(\lambda(I_3 - 1) - \mu\big) \, \operatorname{vec}\big(\operatorname{cof}(\mathbf{F})\big), \tag{236}$$

and the flattened 9×9 Hessian is:

$$\operatorname{vec}\left(\frac{\partial^2 \, \Psi_{\text{SNH}}}{\partial \, \mathbf{F}^2}\right) = \mu \, \mathbf{I}_{9\times 9} + \lambda \, \mathbf{g}_3 \, \mathbf{g}_3^\top + \big(\lambda(I_3 - 1) - \mu\big) \, \mathbf{H}_3 \, . \tag{237}$$

The scalar derivatives of Ψ_{SNH} with respect to each Smith invariant are:

$$\frac{\partial \Psi}{\partial I_1} = 0, \qquad \frac{\partial^2 \Psi}{\partial I_1^2} = 0,$$

$$\frac{\partial \Psi}{\partial I_2} = \frac{\mu}{2}, \qquad \frac{\partial^2 \Psi}{\partial I_2^2} = 0,$$

$$\frac{\partial \Psi}{\partial I_3} = \lambda(I_3 - 1) - \mu, \qquad \frac{\partial^2 \Psi}{\partial I_3^2} = \lambda.$$

Substituting into the generic formula and simplifying yields the PK1 and Hessian expressions above.

The SNH Hessian can become indefinite under large deformation. When this occurs, it is projected to PSD before assembly. The eigenmatrices are those presented above, shared by all isotropic energies defined through the Smith invariants. The eigenvalues specific to SNH are:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \qquad \begin{aligned} a_{ii} &= \mu + \lambda \frac{I_3^2}{\sigma_i^2} \\ a_{ih} &= \sigma_{??} \left(\lambda(2I_3 - 1) - \mu\right) \end{aligned}$$

$$\begin{aligned} \lambda_0, \lambda_1, \lambda_2 &= \text{eigenvalues}(\mathbf{A}) \\ \lambda_3 &= \mu + \sigma_3(\lambda(I_3 - 1) - \mu) \\ \lambda_4 &= \mu + \sigma_1(\lambda(I_3 - 1) - \mu) \\ \lambda_5 &= \mu + \sigma_2(\lambda(I_3 - 1) - \mu) \\ \lambda_6 &= \mu - \sigma_3(\lambda(I_3 - 1) - \mu) \\ \lambda_7 &= \mu - \sigma_1(\lambda(I_3 - 1) - \mu) \\ \lambda_8 &= \mu - \sigma_2(\lambda(I_3 - 1) - \mu) \end{aligned} \tag{238}$$

12.2.2 Contact Barrier: First and Second Derivatives

For $0 < d(\mathbf{x}) < \hat{d}$, the barrier derivatives follow by the chain rule through the distance function:

$$\frac{\partial b}{\partial \mathbf{x}} = \frac{\partial b}{\partial d} \frac{\partial d}{\partial \mathbf{x}} \tag{239}$$

$$\frac{\partial^2 b}{\partial \mathbf{x}^2} = \frac{\partial^2 b}{\partial d^2} \frac{\partial d}{\partial \mathbf{x}} \frac{\partial d}{\partial \mathbf{x}}^\top + \frac{\partial b}{\partial d} \frac{\partial^2 d}{\partial \mathbf{x}^2} \tag{240}$$

where the scalar barrier derivatives are:

$$\frac{\partial\, b}{\partial\, d} = -2\big(d - \hat{d}\big)\ln\left(\frac{d}{\hat{d}}\right) - \frac{(d-\hat{d})^2}{d} \tag{241}$$

$$\frac{\partial^2 b}{\partial\, d^2} = -2\ln\left(\frac{d}{\hat{d}}\right) - \frac{4(d-\hat{d})}{d} + \frac{(d-\hat{d})^2}{d^2} \tag{242}$$

The distance derivatives $\frac{\partial\, d}{\partial\, \mathbf{x}}$ and $\frac{\partial^2 d}{\partial\, \mathbf{x}^2}$ depend on the active distance type (PP, PE, PT, or EE) for each pair in $\mathbb{C}$. Each barrier Hessian is restricted to the stencil of its primitive pair and projected to PSD before global assembly.

References

[1] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. "XPBD: position-based simulation of compliant constrained dynamics". In: *Proceedings of the 9th International Conference on Motion in Games*. 2016, pp. 49–54.

[2] Sofien Bouaziz et al. "Projective dynamics: Fusing constraint projections for fast simulation". In: *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*. 2023, pp. 787–797.

[3] Minchen Li et al. "Incremental Potential Contact: Intersection- and Inversion-free Large Deformation Dynamics". In: *ACM Trans. Graph. (SIGGRAPH)* 39.4 (2020).

[4] Miles Macklin et al. "Primal/dual descent methods for dynamics". In: *Computer Graphics Forum*. Vol. 39. 8. Wiley Online Library. 2020, pp. 89–100.

[5] Junbang Liang, Ming Lin, and Vladlen Koltun. "Differentiable cloth simulation for inverse problems". In: *Advances in neural information processing systems* 32 (2019).

[6] David Baraff and Andrew Witkin. "Large steps in cloth simulation". In: *Proceedings of the 25th annual conference on Computer graphics and interactive techniques* (1998), pp. 43–54.

[7] Theodore Kim and David Eberle. "Dynamic deformables: implementation and production practicalities". In: *Acm siggraph 2020 courses*. 2020, pp. 1–182.

[8] Theodore Kim. "A finite element formulation of baraff-witkin cloth". In: *Computer Graphics Forum*. Vol. 39. 8. Wiley Online Library. 2020, pp. 171–179.

[9] Huamin Wang and Yin Yang. "Descent methods for elastic body simulation on the GPU". In: *ACM Transactions on Graphics (TOG)* 35.6 (2016), pp. 1–10.

[10] Tuur Stuyck and Hsiao-yu Chen. "Diffxpbd: Differentiable position-based simulation of compliant constraint dynamics". In: *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 6.3 (2023), pp. 1–14.

[11] David Harmon et al. "Robust treatment of simultaneous collisions". In: *ACM SIGGRAPH 2008 papers*. 2008, pp. 1–4.

[12] Brandon Amos and J Zico Kolter. "Optnet: Differentiable optimization as a layer in neural networks". In: *International conference on machine learning*. PMLR. 2017, pp. 136–145.

[13] Mickaël Ly et al. "Projective dynamics with dry frictional contact". In: *ACM Transactions on Graphics (TOG)* 39.4 (2020), pp. 57–1.

[14] Tao Du et al. "Diffpd: Differentiable projective dynamics". In: *ACM Transactions on Graphics (ToG)* 41.2 (2021), pp. 1–21.

[15] Yifei Li et al. “Diffcloth: Differentiable cloth simulation with dry frictional contact”. In: *ACM Transactions on Graphics (TOG)* 42.1 (2022), pp. 1–20.

[16] Breannan Smith, Fernando De Goes, and Theodore Kim. “Analytic eigensystems for isotropic distortion energies”. In: *ACM Transactions on Graphics (TOG)* 38.1 (2019), pp. 1–15.